

Fundamentos de Programación

Copyright (c) 2021 Carlos Obregón

Este trabajo está licenciado bajo la Universal Permissive License (UPL), versión 1.0. Puedes encontrar una copia de la licencia en <https://opensource.org/licenses/UPL-1.0>.

El llamado a la Aventura

¡Bienvenida aventurera o aventurero!

¡La programación no es particularmente difícil, pero exige consistencia!

A continuación algunos consejos para tu travesía:

- ¡Descansa bien! Se ha estudiado que mientras descansas, el cerebro sigue aprendiendo pero de una manera diferente, que complementa las otras formas de aprender. No te esfuerces por estudiar hasta muy tarde, ya que esto no será muy productivo.
- En general, si tienes dificultades con un concepto, no temas poner cierta distancia con él. Aléjate de la pantalla, estira las piernas, haz algo completamente diferente y luego regresa. Este tiempo será utilizado por tu cerebro y podría ser suficiente para tener un momento "¡ajá!" que te ayudará con tu estudio.
- ¡Comunícate! Cuando hablas, otras partes de tu cerebro se activan, lo que podría ser suficiente para comprender un tema. ¡No te sorprendas si mientras estás explicando una dificultad, de repente encuentras la respuesta o contestas tus propias preguntas!
- No tengas miedo de hacer preguntas o pregunta si lo que entendiste es correcto ¡a veces solo necesitas otra perspectiva para que todo encaje en su lugar!
- ¡Toma notas escritas a mano! La mejor forma de tomar notas es a mano, ya que esto también hace que funcionen otras partes del cerebro. Las notas digitales están bien, pero intenta escribirlas primero a mano y luego podrás ponerlas en formato digital. Además, ¡no tengas miedo de dibujar diagramas, mamarrachos, flechas o cualquier cosa que encuentres útil para un tema! Estas pistas visuales son muy importantes para el cerebro.
- La repetición espaciada es muy importante para aprender. Por ejemplo, si vas a dedicar 10 horas a la semana es mejor tener cinco bloques de 2 horas que tener uno solo de 10 o dos de 5. Idealmente utiliza un espacio diario, por pequeño que sea, para al menos repasar un concepto o intentar un ejercicio.

En cuanto al material, ten en cuenta esto:

- Algunos capítulos tienen secciones llamadas "misiones secundarias". Intenta leer estas secciones, pero si no las entiendes de inmediato, continúa y vuelve a ellas más tarde. No te las saltes por completo ya que te enseñarán algo valioso, pero está bien si después de una primera lectura, las pospones por un tiempo.

- Los ejercicios tienen respuestas seleccionadas. Igual de importante que hacer los ejercicios es echarles un vistazo cuando hayas hecho la tuya. Leer código es una habilidad tan importante como escribirlo y probablemente aprenderás mucho viendo cómo otros resuelven el mismo problema que tú. No des un ejercicio por completo hasta que hayas comprendido estas soluciones.
- Algunos ejercicios están marcados con 1 o 2 tazas de café, que indican su nivel de dificultad. No por ser difíciles deberías evitarlos. Aún si no puedes completar el código, trata de esbozar una idea de cómo los podrías resolver. Sí definitivamente sientes que no encuentras cómo resolverlos, busca la solución y estúdiala. Luego, más adelante, vuelve a intentarlos. Es importante que puedas llegar a escribir una solución de cada ejercicio.

Convenciones a tener en cuenta:

- Cuando una palabra se escribe dentro de `< y >` es el identificador de un elemento.
- Si al final de un nombre ves `()` es una función.



Las secciones azules tienen anécdotas históricas.



Las secciones grises tienen conocimientos de los que debes tomar nota. Por lo general, es un enlace a la documentación oficial o información sobre cuándo una función estuvo disponible por primera vez en Java.



Las secciones amarillas hablan de cosas a tener en cuenta para que tus programas se comporten correctamente. Esto podría ayudarte a evitar problemas sutiles cuando su programa se ejecuta pero no hace lo que se espera.



Las secciones rojas muestran errores del compilador. No hay nada más frustrante que intentar ejecutar un programa y ver errores del compilador, por lo que estas secciones intentarán ayudarte a evitarlos.



Las secciones moradas explican mejores prácticas: la programación no se trata sólo de hacer que tu programa "funcione", sino que es igualmente importante otras características. Estos consejos te ayudarán a destacar entre otros programadores.



Las secciones verdes hacen preguntas abiertas. Intenta pensar en ellas, sabiendo que no hay respuestas incorrectas. ¡Discute tus respuestas con otros y ve otras perspectivas!

¡Ten cuidado con la forma en que utilizas Internet para ayudarte en tu viaje! Hay mucha información en línea, alguna está anticuada y otra simplemente es incorrecta.

Es posible que encuentres diferentes estilos de codificar: este material te explicará porqué elegimos el nuestro. A medida que adquieras confianza, es posible que quieras usar estilos diferentes al que se usa aquí y eso está bien, pero creemos que el estilo que se muestra aquí es un buen estilo para comenzar a aprender.

¿Qué debo hacer si mi código no funciona?

Todos escribimos código que no funciona como esperamos, no importa cuánta experiencia tengamos. Aquí hay algunas estrategias que puedes utilizar, sin ningún orden en particular:

- ¡Comprueba tu comprensión de lo que quieres lograr! Antes de realizar cambios, asegúrate de entender lo que quieres hacer; si no, empieza por ahí.
- Si tu código utiliza funciones que no escribiste, asegúrate de comprender cómo usarlas correctamente.
- Vuelve a leer el código que escribiste y verifica que cada línea haga lo que esperas. A veces escribimos código y pasamos por alto un pequeño detalle, por lo que volver a leer el código detenidamente puede ser suficiente para ver lo que se pasó por alto la primera vez.
- Si tienes un error del compilador:
 - Lee el mensaje de error del compilador. Algunos de ellos son un poco esotéricos pero otros son muy claros. Una vez que resuelvas el error, tendrás una idea de cómo corregir errores futuros con el mismo mensaje de error.
- Si no es un error de compilador
 - Prueba diferentes entradas y mira cómo se comporta tu función. A veces puedes entender cuál es la línea de código que se comporta mal simplemente verificando cuáles son las entradas que dan una respuesta inesperada.
- Resiste la tentación de simplemente hacer correcciones aleatorias. Cada código fallido debe ser una oportunidad para profundizar tu comprensión de la programación. Si te

tomas el tiempo para solucionarlo, te ahorrarás muchas horas de frustración en el futuro.

- Intenta eliminar tu código y escribe la versión más simple de la funcionalidad que deseas lograr. Comprueba si esa versión funciona y luego escribe un poco más de código, lo suficiente para ampliar un poco su funcionalidad. Escribir código complejo en pequeños incrementos iterativos suele ser la mejor manera de evitar errores.

Como mencionamos, inevitablemente en algún momento necesitarás pedir ayuda, ya sea en línea o a personas que conozcas, asegúrate de tener en cuenta estos puntos:

1. Antes que nada, explícale a la persona lo que quieres lograr, p.j. "Estoy intentando escribir una función que haga X"
2. Luego muestra el código que has escrito hasta ahora. Si escribiste varias versiones también explica estos otros acercamientos.
3. Luego muestra el mensaje de error que tienes o explica el error en el comportamiento que estás viendo, con ejemplos: p.j. "Si ingreso el número 4 y 3 lo que está retornando es 1 pero yo espero que regrese 7"

La programación es un desafío, así que asegúrese de dedicar la cantidad de tiempo adecuada para aprenderla. ¡Si eres constante podrás entender cualquier tema!

Pero sobre todo ¡disfruta del viaje!

JShell

Nota: Lee ligeramente este capítulo antes del capítulo 1, pero a medida que leas otros capítulos es posible que desees volver a leerlo.

JShell es una herramienta que forma parte de todas las instalaciones de Java desde Java 9 en adelante.

Para iniciar JShell, ve a una terminal y escribe jshell. La aplicación comenzará con un mensaje parecido a este:

```
| Welcome to JShell -- Version 11.0.16.1
| For an introduction type: /help intro

jshell> █
```

Una vez iniciado, todo lo que escribas se interpretará como código Java. Para salir escribe "/exit"

```
jshell> /exit
| Goodbye
```

Para tener todo organizado, crea un directorio en tu computadora específicamente para el Bootcamp e inicia la sesión de JShell en ese directorio.

Ahora, cuando el material te presenta un código como:

01	String Saludos() {
02	return "Hola Mundo!";
03	}

Debes escribir el texto que está en la columna de la derecha, en una sesión de JShell activa. *No copies y pegues el código ya que escribirlo te sirve para reforzar el aprendizaje.*

JShell es lo suficientemente inteligente como para que, si está escribiendo un código de varias líneas, sepa cuándo has terminado y cuándo no, y te mostrará "...>" para informarte que el intérprete espera más código para escribir. Antes de terminar, el código anterior quedaría así:

```
jshell> String greetings() {  
    ...>     return "Hello World!";  
    ...> }
```

Y cuando hayas terminado, el intérprete imprimirá una línea explicando cómo interpreta lo escrito. Para el ejemplo anterior se vería así:

```
jshell> String greetings() {  
    ...>     return "Hello World!";  
    ...> }  
| created method greetings()  
  
jshell>
```

Si ves en el material algo como esto:

> saludos()	Hola Mundo!
-------------	-------------

Significa que si ya definiste el método en JShell, al llamarlo, usando la sintaxis que está a la derecha (ignorando el >), deberías obtener la respuesta a la derecha.

En la terminal, se vería de una manera parecida a:

```
[jshell> greetings()  
$2 ==> "Hello World!"  
  
jshell>
```

Si deseas recorrer los comandos que has ingresado previamente en tu sesión, puedes usar las flechas hacia arriba y hacia abajo de tu teclado. Luego podrás realizar cambios en tus programas para corregir errores o probar cosas diferentes:

```
jshell> String greetings() {  
    ...>     return "Hello Java!";  
[    ...> }  
| modified method greetings()  
  
jshell> █
```

Consejo: si vas a editar una función de varias líneas, probablemente agregando más líneas de código, primero elimina la última línea de código, que siempre es un "}", y comienza a editar la función. Dado que eliminaste la última línea, el intérprete te permitirá realizar modificaciones sin pensar que has terminado. Cuando termines de editar el método, coloca el "}" que eliminaste en la última línea de la función.

Eres libre de definir tantos métodos o variables como desees, si deseas saber qué está definido en la sesión activa escribe: `/list`.

```
[jshell> /list  
  
2 : greetings()  
3 : String greetings() {  
    return "Hello Java!";  
}  
  
jshell> █
```

Algunos de los ejercicios de un capítulo determinado pueden utilizar métodos que se definieron como ejemplos, por lo que antes de intentar los ejercicios asegúrate de tener todos los códigos de ejemplo disponibles.

Finalmente, puedes guardar lo que escribiste en una sesión como un archivo y puedes iniciar una sesión usando ese archivo en lugar de volver a escribir el código. Para crear un archivo escribe: `/save [filepath]`, por ejemplo, para guardar la sesión en el directorio de trabajo en un archivo llamado "hola-mundo" escribe: `/save hola-mundo`

Si tienes una sesión JShell activa y deseas utilizar el estado guardado en un archivo determinado, escribe: `/open [filepath]`

Hay más características de JShell, pero esta introducción es suficiente para comenzar tu viaje de aprendizaje.

El documento oficial para JShell está aquí:

<https://docs.oracle.com/javase/9/jshell/introduction-jshell.htm>

¿Qué es Programar?

Programar es crear instrucciones para que un computador las haga. ¿Por qué nos interesa que las haga un computador? Resulta que los computadores eléctricos son muy rápidos y eficientes. Y hoy en día gracias a muchos dispositivos que se han inventado para conectar a un computador, hay muchas cosas que un computador puede hacer, así que poder programar lo que queremos que haga abre muchísimas opciones para personas y para empresas.

Pero ¿cómo funciona? Primero hay que entender que los computadores eléctricos son binarios: todas las instrucciones las procesan como series de ceros y unos. ¿Por qué binarios? Porque con la electricidad solo se pueden modelar fácilmente 2 estados: prendido o apagado. Esto es poco ideal porque se necesitan códigos muy largos para representar las operaciones que un procesador sabe hacer y para codificar información de la vida real.

El procesador es la parte del computador que sabe como realizar operaciones. Cada empresa que desarrolla un procesador inventa un *lenguaje de máquina* para interactuar con él. Al principio los programadores tenían que usar tarjetas perforadas para programar. Esto era engorroso y era muy fácil cometer errores que eran difíciles de reconocer y de corregir.

Otra idea importante fue la de los *sistemas operativos*. Estos son programas que están hechos para encargarse de todas las operaciones de muy bajo nivel en un computador. De modo que un programador solo tenga que preocuparse de sus instrucciones y dejar la interacción con todos los elementos al sistema operativo.

Muy pronto la industria se dio cuenta que lo más óptimo era crear lenguajes de programación, que tuvieran una sintaxis que fuera un mezcla de palabras en algún lenguaje, usualmente inglés, y símbolos para representar las instrucciones que queríamos usar. Pero entonces necesitamos un traductor: *el compilador*. Este toma el código que escribimos en un lenguaje de programación y es capaz de entenderlo, produce un ejecutable para el sistema operativo.

De esta manera ejecutar un programa es al menos un proceso de 3 pasos:

1. Un programador escribe un programa utilizando un lenguaje de programación. En el lenguaje de programación se abstraen las operaciones de modo que un mismo código puede ser usado para crear ejecutables de diferentes sistemas operativos.
2. El programador ejecuta un compilador para que tome su código y lo convierta en un ejecutable para ese sistema operativo. Si el compilador no puede entender el significado de nuestro programa, debemos volver al paso 1 y corregir los errores que nos haya señalado el compilador y volver a ejecutar el compilador hasta que la operación sea exitosa.
3. El programador ejecuta el programa, de modo que el sistema operativo lo ejecuta.

En la medida que los computadores se fueron volviendo más eficientes, algunos lenguajes de programación, como Java, optaron por un proceso distinto con un nuevo elemento: una máquina virtual.

El proceso es el siguiente:

1. El programador escribe el programa en un lenguaje de programación.
2. El programador ejecuta un compilador que toma el código y lo transforma en un lenguaje intermedio que ya no es fácil de leer para un humano pero que tampoco es lenguaje de máquina. Usualmente a este lenguaje intermedio se le conoce como *bytecode*.
3. El programador inicia la máquina virtual y le pide que interprete el código.
4. En la medida que la máquina virtual va interpretando el código, lo transforma en lenguaje de máquina y le pide al sistema operativo que lo ejecute.

Este paso adicional permite que la máquina virtual genere algunas optimizaciones que no podría hacer de otra manera.

No hay ningún modelo que sea infinitamente superior al otro. Y esto se puede generalizar a los lenguajes de programación, no hay uno que sea superior a todos los demás. Usualmente cada lenguaje de programación tiene fortalezas y debilidades que lo hacen una buena elección para ciertos campos y quizás una elección no tan buena para otros.

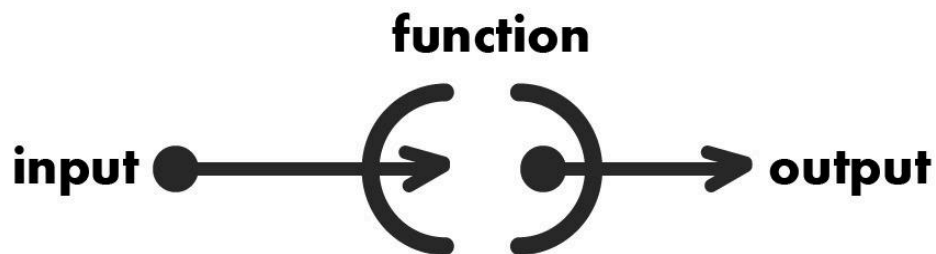
Herramientas como JShell simplifican estos procesos y generan un ciclo de retroalimentación inmediato que es muy útil para aprender a programar.

Más adelante usaremos programas llamados IDE (integrated development environment) que son acercan al proceso completo de programación al tiempo que nos permiten usar herramientas adicionales que nos ayudan a aprender conceptos más complejos.

1. ¡Hola Mundo!

Las **funciones** son un concepto fundacional en la programación. Tan importantes son que vale la pena que sean lo primero que aprendamos.

Una función toma unos parámetros o *input* y produce un valor de retorno o *output*. Los pasos que se siguen para esa transformación reciben el nombre de **algoritmo**.



Ejemplos sencillos de una función son:

- Un procedimiento que calcule el precio que el consumidor debe pagar, teniendo como *input* el precio base y el valor del impuesto a cobrar, y como *output* el precio final.
- Un procedimiento que devuelva sinónimos de una palabra, teniendo como *input* la palabra que nos interesa, y como *output* una lista de palabras.
- Un procedimiento que indique si un año es bisiesto, teniendo como *input* el número del año y como *output* un predicado *verdadero* si es bisiesto o *falso* en caso contrario.
- Una función que resuelva una sopa de letras, teniendo como *input* una cuadrícula cuadrada con letras mayúsculas y una lista de palabras a buscar, y como *output* una sopa de letras donde las palabras de la lista aparezcan destacadas.



Los algoritmos son secuenciales, llevamos a cabo un primer paso antes de llevar a cabo un segundo paso, etc. hasta que finalmente somos capaces de devolver el *output* esperado.

Veamos una función muy sencilla:

Código 1 - Hola Mundo

```
01 String saludos() {  
02     return "Hola Mundo!";  
03 }
```

Si lo invocas:

```
> saludos()
```

```
Hola Mundo!
```

¡Has escrito tu primera función! Felicitaciones.

El Código 1 muestra una **función** en Java que retorna una cadena de caracteres, llamadas String, en este caso "Hola Mundo".



El libro "The C Programming Language", de 1978, escrito por Dennis Ritchie y Brian Kernighan, es considerado uno de los libros más influyentes en nuestra industria. El primer ejemplo de código en ese libro es un programa que imprime "¡Hola mundo!" en la pantalla y se ha vuelto costumbre que todos los tutoriales de programación hagan lo mismo. Esta referencia ha entrado en la cultura popular e incluso se ha convertido en sinónimo de dar un ejemplo con los elementos más simples de un concepto.

La línea 01 tiene 3 aspectos importantes de todas las funciones, en orden:

1. **Valor de retorno** u **output**, en nuestro ejemplo String. El valor de retorno es importante para saber qué vamos a obtener después de llamar a nuestra función.
2. El **identificador** de la función, en nuestro caso saludos. El identificador es importante para poder distinguir las diferentes funciones que tengamos.
3. Y los **parámetros** o **input**, que siempre están entre paréntesis, "(" y ")". En este caso no hay parámetros, nuestra función siempre va a retornar el mismo valor.

Estos 3 elementos se llaman la **firma** de una función.

Al final de la línea 01 encontramos una llave de apertura "{" y en la línea 03 encontramos una llave de cierre "}", ellas forman un **bloque**. Los bloques nos ayudan a agrupar las instrucciones de nuestra función y de esta forma reconocer rápidamente una instrucción a qué función pertenece.

La línea 02 tiene una **sentencia** fácilmente identificable por el punto y coma, ";", al final. En este caso es una **sentencia return**, indicada por la palabra clave return usada en ella, que nos indica cuál es el output de la función. Toda función debe terminar en una sentencia return implícita o explícita. Luego viene un String, indicado por comillas dobles "", que rodean la cadena de caracteres "¡Hola Mundo!".

Leyendo el código podemos ver que al llamar la función saludos() obtendremos la cadena de caracteres "¡Hola Mundo!"

Esta es una de las funciones más simples que podemos hacer, con el pasar de los capítulos iremos aprendiendo cómo hacer funciones más complejas.

Revisemos cada concepto en más detalle.

Valor de Retorno

El valor de retorno del código 01 es String, que representa una cadena de caracteres. Sin embargo las funciones pueden retornar otros tipos de datos, como números, fechas o incluso tipos de datos que nosotros mismos definamos.

Identificadores

El identificador del código 01 es <saludos>. El uso de un buen identificador es muy importante para nuestra capacidad de entender el código que nosotros escribimos o que escriben otras personas. Sin embargo no todos los identificadores son válidos, hay unas reglas básicas. Aquí las más importantes:

✗mi saludo maravilloso	Los identificadores no pueden tener espacios
✗101saludos	Los identificadores pueden tener números, pero no pueden empezar con ellos.
✓SopaDeLetras	Los identificadores pueden llevar mayúsculas y minúsculas.
✓CONSTANTE_GRAVITACIONAL	Los identificadores pueden usar "_", por ejemplo para separar palabras en mayúsculas.



Si usas un identificador que no sea válido, tu programa no va a compilar y a veces tendrá un mensaje un poco críptico. Afortunadamente hay una manera sencilla de evitar estos errores: evita los espacios y los números como caracteres iniciales, lo demás es un poco intuitivo.



Las mayúsculas y minúsculas son importantes en los identificadores que uses. Una función que se llama saludos es diferente de una que se llame Saludos. Si no usas el identificador exacto de una función obtendrás un error de compilación.



Hay algunas convenciones en el mundo de la programación que conviene seguir. La más usada en lenguajes como Java es el uso de *Camel Case*: Escribe primordialmente en minúsculas pero si un identificador usa más de una palabra entonces haz que la letra inicial esté en mayúsculas. Algunos ejemplos serían: *SopaDeLetras*, *miSaludo*, *unIdentificadorMuyLargo*, etc. Si además la primera letra es también mayúscula se llama *Uppercase Camel Case*, sino se llama *Lowercase Camel Case*. En Java la convención es que las funciones y las variables utilicen *Lowercase Camel Case*.

Parámetros

Los parámetros siempre van después del identificador de una función y van entre paréntesis. Si no hay nada entre los paréntesis es porque la función no tiene parámetros. Las funciones sin parámetros suelen ser poco comunes, pero a veces son útiles. Pronto veremos funciones con parámetros.

Bloques

En el código 01, vemos un **bloque** delimitado por las llaves, la de apertura en la línea 01 y la de cierre en la línea 03. Un bloque siempre es una manera de agrupar elementos de nuestro código. Si tenemos muchas líneas de código es importante poder entender rápidamente a dónde pertenecen. Por ejemplo la línea 02 sabemos que pertenece a la función saludos porque está definida dentro del bloque de esa función. Además de funciones, hay otros elementos que usan bloques y en todos cumplen la misma función: agrupar.



El uso de caracteres específicos no es la única manera en la que los lenguajes de programación agrupan cosas. He aquí 2 ejemplos adicionales de cómo lo hacen, el primero en el lenguaje Python y el segundo en Basic:

```
def saludos() :  
    return "¡Hola Mundo!"
```

```
Function saludos()  
    return "Hello World!"  
End Function
```

En el primer ejemplo, se utilizan los niveles de tabulación, un elemento pertenece al elemento que tiene un nivel menos de tabulación. En el segundo, en vez de utilizar un carácter arbitrario, se utilizan palabras en inglés para indicar el inicio y el fin de un elemento.

¿Cuáles crees que son las ventajas y desventajas de los 3 métodos?

Sentencias

Finalmente en la línea 02 hay una sentencia. Una sentencia indica una instrucción que se le da al computador. Una sentencia siempre termina en ";".

Todas las funciones deben terminar en una sentencia return, es decir una sentencia que inicia con la palabra clave **return**. Una sentencia return indica el valor que se va a retornar una vez que la función termina de ser ejecutada. En la función <saludos> siempre retornamos la misma cadena de caracteres: "¡Hola Mundo!"

Pronto escribiremos funciones más complejas. Las funciones pueden tener múltiples sentencias, cada una haciendo un paso intermedio para calcular el valor final. Lo más importante a tener en cuenta es que una función se ejecuta una línea a la vez, en el orden que aparecen.



Uno de los errores más comunes al aprender a programar en Java es olvidar poner el ";" al final de una sentencia. ¡No olvides tus ";"!



Los ";" son importantes porque permiten al compilador identificar dónde termina una sentencia, de igual manera que los humanos necesitamos un "." para saber dónde termina una oración. Hay otros lenguajes que usan el caracter de final de línea para indicar el final de una sentencia. Por lo que no se puede poner más una sentencia por línea de código, algo que antes los programadores veían como inconveniente aunque hoy en día es mucho más aceptado. Por lo tanto los lenguajes deben tomar una decisión: permitir más de una sentencia por línea, necesitando un caracter como ";" para separarlas, o permitir una sola sentencia por línea y no usar ningún delimitador.



Históricamente las funciones se han asociado a la reutilización de código, pero, el verdadero poder de las funciones vienen de la legibilidad que agregan a nuestro código. Asignando un nombre a un algoritmo hacemos nuestro código más fácil de entender. Deberías siempre escribir una *función* incluso si no esperas reusarla.

La reutilización es un bono adicional a la legibilidad.

¡Hola Mundo! ¡Hola Java!

Código 02 - ¡Saludos con parámetros!

```
01 String saludosPara(String quien) {  
02     return String.format("¡Hola %s!", quien);  
03 }
```

> saludosPara("Mundo")	¡Hola Mundo!
> saludosPara("Java")	¡Hola Java!

La función `saludosPara()` es muy similar a la función `saludos()` pero ahora tiene un **parámetro**. Un parámetro agrega flexibilidad a una función, dado que una función sin parámetros retornaría siempre el mismo valor. ¿Qué sucede si queremos saludar, no solo al mundo, sino a otras personas o cosas? ¡Parámetros al rescate!

En la línea 01 se define un parámetro dentro de los paréntesis. Un parámetro tiene 2 partes: un tipo de dato, como lo tienen los valores de retorno, y un identificador. Para el Código 02 el parámetro también es un String y decidimos usar el identificador "quien" para ayudar a recordar el papel que tiene dentro de la función. Podríamos haber usado cualquier identificador y la

función seguiría retornando el valor esperado, pero usar un buen identificador hace que nuestro código sea más legible.



Vas a escuchar que en muchos lugares vamos a hablar sobre cómo algunas recomendaciones hacen el código más legible. ¿Si una máquina va a ejecutar el código, por qué nos preocupamos por que sea legible para un humano? Antes de que un código sea ejecutado, los humanos tienen que leerlo y entenderlo para poder corregir errores en ellos, para poder extenderlos e incluso para aprender de ellos. De hecho es algo que hace unos años no se mencionaba en la industria y había una cierta mentalidad de que escribir código críptico era chévere porque las personas se sentían orgullosas de entender código que otros no podían. Pero lo malo es que esas mismas personas olvidaban rápidamente la razón por la cuál su código críptico funcionaba. Trata de dedicar un poco de tiempo a pensar un buen identificador para los elementos de tu código. Es difícil hacerlo y quizás no te salga en tu primer intento pero no te sientas desalentado, con la práctica se vuelve más sencillo.

Una función puede tener varios parámetros, separados por comas, y de cualquier tipo. Por ahora solo usaremos Strings porque son fáciles de usar y de hecho son bastante importantes en la programación.

La línea 02 usa la función `String.format()`, ¿de dónde salió? Resulta que todos los lenguajes de programación incluyen una librería estándar con funciones generales. De esta manera sin haber escrito una línea de código, ya tienes la capacidad de hacer muchas cosas.



"No reinventar la rueda" es una frase muy común en programación. Es muy importante conocer la librería estándar del lenguaje de programación que utilices porque te hará más eficiente en el código que escribas.

`String.format()` permite crear una cadena de caracteres combinando diferentes partes. La función necesita al menos 2 parámetros: un formato y un **argumento**. En este caso tenemos un formato de saludo y tenemos a quien queremos saludar. El formato necesita un marcador de posición para saber cómo formar el String resultante, en `String.format()` este es `"%s"`. La función va a tomar los argumentos y va a ir reemplazando los marcadores de posición con ellos.

Podemos pensar la ejecución de `saludosPara()` en 3 reemplazos:

- `saludosPara("Mundo")`
- `String.format(";Hola %s!", "Mundo")`
- `";Hola Mundo!"`

Si utilizamos diferentes argumentos, "Java" en vez de "Mundo", obtendremos diferentes saludos. Ahora podemos crear cualquier tipo de saludo que necesitemos.



Invocar una función con menos o más argumentos que su número de parámetros es un error de compilación. Adicionalmente usar un argumento de diferente tipo también es error de compilación, por ejemplo no pasar un String cuando el parámetro esperado es un String.

¡Genial! Ya podemos crear saludos a todo lo que se nos ocurra.



La documentación oficial de `String.format()` está aquí: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#format\(java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#format(java.lang.String,java.lang.Object...))

Puede parecer un poco desalentador leer ahora, pero sentirse cómodo leyendo documentación oficial es una habilidad muy importante que sería bueno empezar a practicar. ¡No importa tus años de experiencia, leer la documentación oficial es algo que seguirás haciendo!

¡Hola Mundo! versión 2.0

Código 03 - Hola Mundo 2.0

```
01 String saludosPara(String quien) {  
02     return String.format("¡Hola Mundo!",quien);  
03 }  
04  
05 String saludos() {  
06     return saludosPara("Mundo!");  
07 }
```

<pre>> saludos() > saludosPara("Mundo")</pre>	<pre>¡Hola Mundo! ¡Hola Mundo!</pre>
---	--

Cuando escribimos la función `saludosPara()` ¿no se te hizo algo familiar? De alguna manera podemos ver como `saludos()` y `saludosPara()` tienen alguna similitud. De hecho si lo piensas bien, ¡`saludos()` es una versión específica de `saludosPara()`! Es una versión de `saludosPara()` que siempre saluda al Mundo.

Esta relación la podemos poner de manifiesto en el código, y lo que podemos hacer es invocar a la función `saludosPara()` dentro de `saludos()`. Esto es una manera de reutilización de código que es una muy buena práctica y una muy importante en programación, de hecho antes hemos visto que tiene un lema “No reinventar la rueda”. Y de una manera anexa, también es una manera de reducir la repetición en nuestro código que es igualmente una muy importante práctica.



DRY es una sigla para Don't Repeat your Code. Si tenemos repetición en nuestro código, cuando queramos ampliarlo o corregirlo tenemos que tener cuidado de hacer los cambios en todas las partes. Si olvidamos hacerlo entonces tendremos un sistema inconsistente y a veces eso genera errores sutiles difíciles de encontrar. Al evitar la repetición nos aseguramos de evitar este tipo de problemas.

Con el Código 03, si más adelante queremos cambiar el formato del saludo, entonces solo tendremos un lugar donde hacer el cambio, pero ambas funciones verán el cambio.

El Código 03 también nos reitera que dentro de una función podemos usar funciones dentro de funciones incluso las que hemos definido nosotros. No solo es permitido sino que es algo que debemos fomentar. En un sentido, la programación trata de hacer pequeñas partes y después usar esas pequeñas para crear partes más complejas.

Es muy importante que empieces a ejercitar el hábito de escribir pequeñas funciones y utilizarlas como partes de otra función. Lo inverso es también importante, si vas a crear una función, piensa cómo puedes dividir el algoritmo en partes más pequeñas y escribir una función para cada una.



Cuando estés escribiendo una función, piensa en cómo puedes implementarla en términos de funciones más pequeñas. Esto te dará la oportunidad de reutilizarlas más adelante además hará que tus funciones sean más fáciles de leer y de probar.

Espacio en blanco

Código 04 - saludos() con mucho espacio en blanco

```
String saludos
(String quien)
{
    return String.format("¡Hola %s!", quien);
}
```

Código 05 - saludos() con no mucho espacio en blanco

```
String saludos(String quien){return String.format("Hola %s!",quien);}
```

El espacio en blanco es cualquier caracter que parezca un espacio, esto incluye espacios, tabulaciones y saltos en línea. El espacio en blanco no tiene ninguna afectación en el funcionamiento de una función, es netamente una opción para mejorar la legibilidad para los seres humanos. Cuánto espacio en blanco es el exacto para mejorar la legibilidad de un código es una cuestión subjetiva, pero parece claro que el Código 04 tiene mucho y el Código 05 tiene muy poco. Por lo tanto, busca escribir código con una cantidad de espacio en blanco que esté entre esos dos estilos, pero sobre todo no temas agregar un poco de espacio en blanco para darle espacio a respirar a los elementos que componen una sentencia para hacerlos más fáciles de entender.



Después de escribir una función, dale un vistazo y mira si se siente densa. En caso de duda, pide una segunda opinión. Agrega un poco de espacio en blanco para hacerla más sencilla a los ojos.

Comentarios

Código 06 - Comentarios

```
01 String saludosPara(String quien) {
02     return String.format("¡Hola Mundo!",quien);
03 }
04
05 String saludos() {
06     return saludosPara("Mundo!"); // evitar repetición
07 }
```

La línea 06 tiene un comentario de una sola línea, empieza con “//” y termina en el siguiente salto de línea. Los comentarios son ignorados por el compilador, son realmente para dejar notas para las personas que puedan leer el código.

Dejar notas en tu código puede ayudar a que sea más fácil de entender. Pero ten cuidado de que el comentario no se desactualice si cambias el código.



También existen los comentarios de múltiples líneas, que empiezan con /* y terminan con */ y como su nombre lo indica pueden englobar múltiples líneas. No es necesario, pero tiende a usarse, que cada línea intermedia empiece con un * para darle un aspecto estético. Este es un comentario de múltiples líneas:

```
/*  
 * Este es un comentario de múltiples líneas  
*/
```

Los comentarios de una sola línea son usualmente preferidos porque es muy sencillo comentar mucho código, y después descomentar partes de él. Sin embargo es importante conocerlos porque los vas a encontrar en muchas partes.

Resumen

Las funciones son la piedra angular de la programación. Una función encapsula un algoritmo; una serie de pasos que toman unos parámetros (input) y lo transforman en un valor de retorno (output), por ejemplo tomar 2 números y transformarlos en su suma.

Una función puede llamar otras funciones. De hecho una habilidad importante es poder tomar un algoritmo, dividirlo en pequeños pasos y crear subfunciones para esos pasos y luego escribir una función que las articule.

Mucho del tiempo de un programador se gasta leyendo código, nuestro o de compañeros, por lo que hay que preocuparse por escribir código fácil de leer. Escoger buenos identificadores, utilizar una buena cantidad de espacio blanco e incluso agregar un comentario aquí y allá, ayudar a que podamos entender código sin tener el autor del mismo a nuestro lado (que incluso podemos ser nosotros en el pasado)

Ejercicios

Errores del compilador y de ejecución

Escribe los siguientes códigos, lee el error que te devuelve el compilador o el ambiente de ejecución. Después escribe el código correcto de modo que al ejecutarlo ya no salgan errores.

E1.1

```
err1() {  
    return "¿Dónde está el error?";  
}
```

E1.2

```
string err2() {  
    return "¿String o string?";  
}
```

E1.3

```
String err3(who) {  
    return String.format("%s, cuál es el error?", who);  
}
```

E1.4

```
String err4() {  
    return "Este es un error sutil"  
}
```

E1.5

```
String err5() {  
    "Es es un poco menos sutil";  
}
```

E1.6

```
String err6 {  
    return "¡Hola Error!";  
}
```

E1.7

```
String err7(String quien) {  
    return format("%s ¿Un poco mañoso, no?", quien);  
}
```

Ejercicios de Codificación

E1.7

Llena los huecos para crear una función que dados 3 Strings que sean emojis, los devuelva en formato "[%s, %s, %s]" por ejemplo al invocar la función así: adivinaLaPelicula("❄️", "🚢", "💕"), retornaría "[❄️, 🚢, 💕]". Para cada hueco con una letra, escoge el reemplazo utilizando una de las opciones numeradas. Una vez que utilices una opción numerada ya no la puedes usar en otro reemplazo.

```
String _A_(String emoji1 _B_ String emoji2 _C_ String emoji3) _D_  
    return _E_("[%s, %s, %s]" _F_ emoji1 _G_ emoji2 _H_ emoji3);  
_I_
```

1. ,
2. ,
3. ,
4. ,
5. ,
6. }
7. {
8. adivinaLaPelicula
9. String.format

E1.8

Llena los huecos, para crear una función que concatene la cadena "こんにちは" (hola en japonés) y un parámetro dado. Para cada hueco con una letra, escoge el reemplazo utilizando una de las opciones numeradas. Una vez que utilices una opción numerada ya no la puedes usar en otro reemplazo.


```

A B {
C String.format("こんにちは D ", E) F
}

```

1. return
2. ;
3. dare
4. konnichiwa(String dare)
5. %s
6. String

E1.9

Escribe una función `separador()` que retorne el String "<<<>>>"

> <code>separador()</code>	"<<<>>>"
----------------------------	----------

E1.10

Escribe una función `vocales()` que retorne el String "aeiou"

> <code>vocales()</code>	"aeiou"
--------------------------	---------

E1.11

Escribe una función `tituloAdornado()` que tome un String como parámetro y retorne el mismo String precedido por "<<< " y antecedido por " >>>"

> <code>tituloAdornado("Java")</code>	"<<< Java >>>"
> <code>tituloAdornado(">>> <<<")</code>	"<<< >>> <<< >>>"
> <code>tituloAdornado("A + B")</code>	"<<< A + B >>>"

Ejercicios de Corrección de Código

Para estos problemas, mira los ejemplos de uso de una función y luego revisa el código inicial que no está funcionando como se espera. Ejecútalo y luego corrígelo para coincidir con los ejemplos.

E1.12

<code>> adornar("Java")</code>	<code>"<=Java=>"</code>
<code>> adornar("Mundo")</code>	<code>"<=Mundo=>"</code>

```
String adornar(String frase) {  
    return String.format("<=%s=>", frase);  
}
```

E1.13

<code>> concatenar("Hola", "Mundo")</code>	<code>"[Hola+Mundo]"</code>
<code>> concatenar("A", "Z")</code>	<code>"[A+Z]"</code>

```
String concatenar(String s1, String s2) {  
    return String.format("[%s+%s]", s2, s1);  
}
```

2. Operadores

Código 1 - hipotenusa, usando tipos doubles y algunos operadores

01	<code>double hipotenusa(double a, double b) {</code>
02	<code> return Math.sqrt((a * a) + (b * b));</code>
03	<code>}</code>

<code>> hipotenusa(3.0, 4.0)</code>	5.0
<code>> hipotenusa(1.0, 1.0)</code>	1.4142135623730951

¿Recuerdas la fórmula para calcular el lado mayor de un triángulo rectángulo, su hipotenusa?

$$c = \sqrt{a^2 + b^2}$$

El Código 1 tiene una función que lo calcula.

Hasta ahora nos hemos ocupado principalmente de cadenas, pero para la hipotenusa presentamos nuestro primer tipo de dato numérico: double. Un double representa un número real (con algunas limitaciones) Para casos simples de números reales puedes usar un double. La función <hipotenusa> usa dos números reales que representan los lados del triángulo rectángulo y calcula la longitud del otro lado.

Para la implementación usamos algunos operadores y otra función de la librería estándar: Math.sqrt(). Esta función calcula la raíz cuadrada de un número. Recibe un double y retorna un double.



La documentación oficial de Math.sqrt:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#sqrt\(double\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#sqrt(double))

Como argumento utilizamos dos operadores: "*" para multiplicar números y "+" para sumar números. Estos operadores también están disponibles con otros tipos numéricos.

Para escribir valores necesitas usar literales: 3.0, 4.0 y 1.4142. Estos son literales para tipo doubles, dado que tienen un punto.



Cuando usas una expresión con más de un operador, ¿cuál se aplica primero? ¿Es `<a * a + b * b>` lo mismo que `<(a * a) + (b * b)>` o lo mismo que `<a * (a + b) * b>`? Cada lenguaje de programación tiene una tabla de precedencia de operadores que nadie recuerda de memoria, pero que es en su mayor parte intuitiva. En cualquier caso, evita tener operadores complejos en una línea y utilice siempre paréntesis para que sea más obvio cuál es el orden previsto de las operaciones.



Utiliza siempre un espacio antes y después de un operador binario, ya que hace que el código sea mucho más legible, no escribas `a+b`, prefiere escribir `a + b`.

Tipos Primitivos

Java viene con algunos tipos definidos que se denominan primitivos para representar tipos básicos: caracteres, números y predicados. Estos son los más importantes:

Palabra clave	Que representa	Ejemplo Literal
<code>char</code>	Caracteres Unicode	<code>'A'</code> , <code>'a'</code> , <code>'1'</code> , <code>' '</code> , <code>'ó'</code> , <code>'@'</code> , etc.
<code>boolean</code>	Predicados	<code>true</code> , <code>false</code>
<code>int</code>	Enteros	<code>1</code> , <code>-17</code> , <code>123_000</code> , etc.
<code>double</code>	Números Reales	<code>1.4142</code> , <code>-123.0</code> , <code>0.0</code> , etc.

Ya conocíamos los String que no son datos primitivos, pero que también tiene literales como `"Hola Mundo!"`



Definir una cadena usando comillas simples es un error del compilador. No utilices '¡Hola mundo!'.



Si usas un literal entero en lugar de un literal doble, por ejemplo llamando a la función hipotenusa como hipotenusa(1, 1), en muchos casos no obtendrás un error de compilación, pero puedes observar errores sutiles, errores lógicos. Ten cuidado con mezclar literales con tipos y sobre todo ten cuidado con los tipos que eliges y los literales que utilizas.

Operadores

Así como ya tenemos algunos tipos, Java tiene algunos operadores que podemos usar. Los operadores más comunes son:

Operadores Numéricos

Estos operadores se pueden utilizar con tipos numéricos como int y double. Toman dos valores del mismo tipo y devuelven un valor de ese mismo tipo, p.j. si sumas dos números enteros devuelven un número entero, si multiplicas dos double devuelve un double.

Simbolo	Operacion	Ejemplo
+	adicion	$a + b$, $x + 1$, etc.
-	sustraccion	$a - b$, $j - 2$, etc.
*	multiplicacion	$a * b$, $x * 10$, etc.
/	division	a / b , $m / 5$, etc.
%	modulo	$a \% b$, $n \% 2$, etc.

El operador de módulo devuelve el resto después de la división de los dos números.



Un error que cometemos al aprender a programar es no recordar que en lenguajes donde hay números enteros y números reales, la división de dos números enteros da como resultado un número entero. Esto significa que $1 / 2$ no es 0.5 sino 0, porque al retornar un entero la parte decimal se pierde. Cuando veamos variables, veremos qué podemos hacer si es importante que el resultado sea un double pero por ahora recuerda este punto: el resultado de estos operadores binarios es del mismo tipo de sus operandos.

Código 2 - Operador +

```
01  int siguiente(int número) {  
02      return número + 1;  
03  }
```

> siguiente(9)	10
> siguiente(33)	34

Código 3 - Operador *

```
01  int agregar0(int número) {  
02      return número * 10;  
03  }
```

> agregar0(55)	550
> agregar0(123)	1230

Código 4 - Operador -

```
01  int rangoInclusivo(int inicio, int fin) {  
02      // retorna la cantidad de números entre fin e inicio  
03      // incluyendo los extremos  
    return fin - inicio + 1;  
}
```

> rangoInclusivo(3, 10)	8
> rangoInclusivo(100, 102)	3



Si utilizas un operador con tipos incompatibles, obtendrás un error de compilación, pero si operas dos tipos numéricos diferentes, no. Por ejemplo, si operas un `int` con un `double`, obtendrás el resultado que podrías esperar, p.j. `<1 + 1.0>` devuelve `<2.0>` ¿Qué crees que hace el compilador para que esto suceda?

Operadores de Relación

Estos operadores se pueden utilizar con tipos numéricos como `int` y `double`. Toman dos números del mismo tipo y devuelven un valor booleano.

Símbolo	Operación	Ejemplos
<code><</code>	Menor que	<code>a < b</code> , <code>x < 0.0</code> , etc.
<code><=</code>	Menor o igual	<code>a <= b</code> , <code>length <= 1.0</code> , etc.
<code>></code>	Mayor que	<code>a > b</code> , <code>remainder > 0.0</code> , etc.
<code>>=</code>	Mayor o igual	<code>a <= b</code> , <code>(a + 1) <= b</code> , etc.
<code>==</code>	Igual	<code>a == b</code> , <code>size == 1</code> , etc.
<code>!=</code>	Diferente	<code>a != b</code> , <code>n != 0</code> , etc.



La mayoría de los lenguajes de programación usan los mismos símbolos para estas operaciones. JavaScript, otro lenguaje popular, tiene dos tipos de operador igual y no igual, también tiene `===` y `!==` pero estos no son necesarios en Java, así que no te confundas si ya conoces JavaScript o alguna vez has visto código escrito. en él.

Código 5 - Operador >=

01	<code>boolean esMayorDeEdad(int edad) {</code>
02	<code>// la mayoría de edad es a partir de los 18</code>
03	<code>return edad >= 18;</code>
04	<code>}</code>

<code>> esMayorDeEdad(17)</code>	<code>false</code>
<code>> esMayorDeEdad(18)</code>	<code>true</code>
<code>> esMayorDeEdad(31)</code>	<code>true</code>

Código 6 - Operador <

01	<code>boolean exentoDeImpuesto(double precio) {</code>
02	<code>// si el precio es menos de \$1.000.000 no paga impuesto</code>
03	<code>return precio < 1_000_000.0;</code>
04	<code>}</code>

<code>> exentoDeImpuesto(150_000)</code>	<code>true</code>
<code>> exentoDeImpuesto(2_300_000)</code>	<code>false</code>

Código 7 - Operador ==

01	<code>boolean esMultiplo(int a, int b) {</code>
02	<code>return (a % b) == 0;</code>
03	<code>}</code>

<code>> esMultiplo(30, 2)</code>	<code>verdadero</code>
<code>> esMultiplo(100, 3)</code>	<code>falso</code>

esMultiplo() es una función que toma dos números enteros y devuelve un valor booleano: verdadero si a es múltiplo de b o falso en caso contrario. Utiliza el operador del módulo para verificar si el resto de dividir a y b es 0, que es la definición de ser múltiplo. Podemos usar varios operadores en una misma expresión, así sea de tipos distintos.

El uso más común del operador % es para revisar si un número es múltiplo de otro.

Operadores Lógicos

La mayoría de estos operadores toman dos predicados y devuelven otro predicado. Sólo el operador negativo toma un solo predicado. Se utilizan para obtener predicados más complejos a partir de otros predicados. El resultado de estos operadores se muestra después de ellos:

Simbolo	Operacion	Ejemplo
&&	Logic AND	<code>a && b</code> , <code>a > 0 && a < 10</code> , etc.
	Logic OR	<code>a b</code> , <code>(a != b b == 10)</code> , etc.
!	negate	<code>!contains</code> , etc

Tabla lógica del operador AND

a	b	<code>a && b</code>
verdadero	verdadero	verdadero
verdadero	falso	falso
falso	verdadero	falso
falso	falso	falso

El resultado del AND lógico es sólo puede resultar en verdadero cuandoambos operandos son verdaderos.

Tabla Lógica del operador OR

a	b	<code>a b</code>
verdadero	verdadero	verdadero
verdadero	falso	verdadero
falso	verdadero	verdadero
falso	falso	falso

El resultado del OR lógico sólo puede resultar en falso cuando ambos operandos son falsos.

Tabla lógica del operador negación

n	!n
verdadero	falso
falso	verdadero

Código 8 - Operadores && y !

01	<code>boolean esMayorDeEdad(int edad) {</code>
02	<code> return edad >= 18;</code>
03	<code>}</code>
04	
05	<code>boolean esAdultoMayor(int edad) {</code>
06	<code> return edad >= 60;</code>
07	<code>}</code>
08	
	<code>boolean esAdulto(int edad) {</code>
	<code> return esMayorDeEdad(edad) && !esAdultoMayor(edad);</code>
	<code>}</code>

<code>> esAdulto(24)</code>	<code>true</code>
<code>> esAdulto(81)</code>	<code>false</code>

Código 9 - esAñoBisiesto, predicados complejos

01	<code>boolean esMúltiplo(int a, int b) {</code>
02	<code> return (a % b) == 0;</code>
03	<code>}</code>
04	
05	<code>boolean esAñoBisiesto(int year) {</code>
06	<code> return esMúltiplo(year, 4) &&</code>
07	<code> (esMúltiplo(year, 400) !esMúltiplo(year, 100));</code>
08	<code>}</code>

<code>> esAñoBisiesto(1900)</code>	<code>false</code>
<code>> esAñoBisiesto(2000)</code>	<code>true</code>
<code>> esAñoBisiesto(2024)</code>	<code>true</code>

esAñoBisiesto() toma un número entero y devuelve verdadero si ese año es o será bisiesto. Un año bisiesto debe ser múltiplo de 4 y tener una o dos condiciones más: ser múltiplo de 400 o no ser múltiplo de 100. Nuevamente, usamos esMúltiplo() para mejorar la legibilidad del código.

Operador Ternario

Código 4 - máximo usando el operador ternario

01	<code>double max(double a, double b) {</code>
02	<code> return (a > b) ? a : b;</code>
03	<code>}</code>

<code>> max(3.0, 5.0)</code>	<code>5.0</code>
---------------------------------	------------------

La mayoría de los operadores son binarios, lo que significa que operan sobre dos valores, como la suma. En matemáticas y lógica también existen operadores unarios que utilizan un solo valor. El operador negativo en programación es un operador unario. Resulta que en programación existe un operador ternario de esta forma:

<code><predicado> ? <valor1> : <valor2></code>
--

El resultado del operador se basa en el predicado, si la expresión es verdadera se evalúa como el "primer" valor, el que está entre "?" y ":", si es falso, se evalúa como el "segundo" valor, el que está después de ":".

Para <max> en el Código 4, el predicado es <a > b>. Si eso se evalúa como verdadero, entonces el valor de toda la expresión es el parámetro <a>; de lo contrario, el valor es el parámetro . El predicado puede ser tan complejo como se necesite, y las otras partes también pueden ser tan complejas como se desee, aunque probablemente sea mejor mantenerlas simples.

Podrías pensar que es extraño tener un operador como el operador ternario, pero resulta que hay muchas oportunidades para usarlo en lugares para hacer que tu código sea conciso pero muy legible.

Misión secundaria: consideraciones con números reales

Los números reales son difíciles de representar en las computadoras. A las variables se les asigna una cantidad de bits para ser representadas, pero muchos números reales tienen infinitos dígitos decimales como $\frac{1}{3}$. Dado que las computadoras sólo pueden tener una representación limitada de dichos números, debes tener cuidado al usarlos en tu código. Si tienes varias operaciones con números reales, es probable que el número sea ligeramente diferente de lo esperado. En algunos casos, estos podrían no ser importantes, pero en otros, como el dinero, sería inconveniente y se debería utilizar otra alternativa. Dichas alternativas se salen del alcance de este material, pero es bueno que conozcas las limitaciones de los tipos de datos primitivos en un lenguaje.

Normalmente se prefiere evitar el uso de un operador como == para comprobar números reales. Debido a las imprecisiones de las que hemos hablado, solemos comprobarlas con un delta. Por ejemplo para comprobar si una variable <n> tiene un valor equivalente a 1.0 puedes usar:

```
Math.abs(n - 1.0) < 1e-3
```

En este caso 1e-3 es otro posible literal para números reales, y es literal para expresar un número en notación científica y en este caso es igual a 0,001. Entonces, en el código anterior estás verificando que <n> sea igual a 1.0 hasta los primeros 2 números digitales. Esto suele aceptarse cuando se trata de longitudes, ya que las diferencias más pequeñas son imperceptibles para el ojo humano.

Diferentes dominios requerirán diferentes deltas, así que no asuma que siempre puede usar el mismo. Cuando utilices un dominio, aprende qué es importante en él y elige el delta con esa información. Como se indicó anteriormente, para las medidas 1e-3 probablemente sería suficiente.

Misión secundaria: Casting

Casting es una operación donde tomamos una variable de un tipo de dato y le pedimos al compilador que la trate como una variable de otro tipo de dato compatible, en primitivas solo funciona con variables numéricas. Por ejemplo para crear una variable int a partir de un valor double podemos escribir:

```
var n = (int) 2.0;
```

Casting tiene una consecuencia importante de entender y es que puede producir pérdida de información. En el código anterior la variable <n> va a tener el valor 2 que es equivalente a 2.0, pero por ejemplo:

```
var n2 = (int) 3.14;
```

<n2> tendrá el valor de 3, lo que significa que los valores decimales se perdieron. Así que cuando usemos casting tenemos que estar seguros que no vamos a perder información o que la pérdida es deliberada.

La razón principal para usar casting es para garantizar que una división de 2 ints dé el resultado esperado. Por ejemplo, ¿Qué valores crees que tendrán las variables <a> y ?

```
var totalElementos = 11;
var tamañoGrupos = 2;

var a = totalElementos / tamañoGrupos;
var b = (double) totalElementos / tamañoGrupos;
```

Lo primero importante es entender que <a> es de tipo int y es de tipo double. <a> es de tipo int porque es el resultado de la división de 2 valores int. Recordemos que la división siempre retorna un valor del tipo de los 2 operandos. es de tipo double, debido a que aplicamos un casting a totalElementos por lo tanto el compilador lo trata como un valor double aunque es de tipo int. Usamos ese valor en una división con un int pero como lo habíamos mencionado al operar un valor double con un int, obtenemos un valor de tipo double. Ahora que estamos seguros del tipo de dato debería ser sencillo llegar a la realización que <a> es 5 y es 5.5

Hay otro uso común para casting. Math es una clase que tiene muchos métodos interesantes. Antes vimos la función sqrt() pero hay muchas otras funciones interesantes. De hecho algunas de ellas evitarían que escribamos código más complicado de lo necesario. Dos de esas funciones son ceil() y floor(), ambas toman una variable double y retornan un

valor double con el menor número entero mayor que el valor dado, para el caso de ceil() y el mayor número que sea menor que el valor dado, para el caso de floor().

> Math.ceil(7.5)	8.0
> Math.ceil(3.14)	4.0
> Math.floor(7.5)	7.0
> Math.floor(3.14)	3.0

En algunos casos lo que queremos es el resultado de ceil() o floor() como un int y no como un double, así que debemos usar casting.

```
var niños = 5;
var porciones = 1.5;

var elementosParaComprar = (int) Math.ceil(niños * porciones);
```

Si tenemos un número de niños y tenemos que alimentarlos con una número determinado de porciones de un alimento, podemos calcular el número de alimentos para comprar multiplicando ambos valores. Pero ese número puede dar un número decimal, en el ejemplo anterior la multiplicación 7.5 pero en raros casos podemos comprar una cantidad no entera de alimentos. Por ejemplo manzanas, nadie nos vende 7.5 manzanas, así que usamos ceil() para estar seguros que obtendremos un número entero. Pero de nuevo ceil realmente retorna double así que el casting nos ayuda a almacenar en una variable int sabiendo que no vamos a tener pérdida de información.

Además de estos 2 casos, pueden haber otras situaciones donde el casting sea necesario, por eso no está de mal entender las reglas de casting.

Diversión con funciones

¿Dónde puedes usar una función que retorne? En cualquier lugar donde se espere un valor. De hecho, una función pura es en realidad otra forma de representar su valor devuelto.

Escribamos algunas funciones:

```
double adición(double a, double b) {
    return a + b;
}
```

```
double multiplicación(double a, double b) {  
    return a * b;  
}  
  
double cuadrado(double n) {  
    return multiplicación(n, n);  
}
```

Ahora podemos reimplementar la `hipotenusa()` como por ejemplo:

```
double hipotenusa(double a, double b) {  
    return Math.sqrt(adición(cuadrado(a), cuadrado(b)));  
}
```

Esa implementación tiene muchos paréntesis pero es una expresión válida. Diferentes programadores argumentarán que la última versión de la `hipotenusa` es más o menos legible que la que vimos inicialmente en este capítulo, pero lo importante es recordar que puedes llamar funciones en cualquier lugar donde puedas usar un valor.

Resumen

Los lenguajes de programación ofrecen diferentes tipos de datos primitivos. En Java los más comunes son:

- `boolean`: para predicados
- `char`: para caracteres Unicode
- `int` : números enteros
- `double`: números reales
- `String`: cadenas de caracteres

Si deseas representar valores de estos tipos de datos, puedes utilizar sus literales:

- `boolean`: `<true>`, `<false>`
- `char`: un solo caracter entre comillas sencillas `'`
- `int`: dígitos sin caracter de punto

- double: dígitos con un punto
- String: cero o más caracteres entre comillas dobles "

Además de funciones, los lenguajes incluyen operadores que puedes utilizar para operaciones que son muy comunes. Los operadores están definidos para algunos tipos y tienen un tipo de devolución específico. Lo más importante que debes recordar es que, si operas con dos valores, el resultado será del mismo tipo, ¡La división de dos enteros también será un entero!

Ejercicios

Errores del compilador y de ejecución

Escribe los siguientes códigos y lee el error que te devuelve el compilador o el ambiente de ejecución. Después escribe el código correcto.

E2.1

```
int multiplicación(String número1, String número2) {  
    return número1 * número2;  
}
```

E2.2

```
int suma(int número1, int número2) {  
    return + número1 número2;  
}
```

E2.3

```
boolean positivoMenorDe20(int número) {  
    return número > 0 && < 20;  
}
```

Ejercicios de Codificación

E2.4

Llena el hueco de la función truncar, de modo que dado un número entero, retorne el mismo número pero sin su último dígito. Revisa los ejemplos para más claridad.

```
int truncado(int n) {  
    return n  10;  
}
```

> truncado(123)	12
> truncado(456)	45
> truncado(9)	0
> truncado(0)	0

E2.5

Llena el hueco de la función últimoDígito, de modo que dado un número entero, retorne el último dígito de este. Revisa los ejemplos para más claridad.

```
int últimoDígito(int n) {  
    return n  10;  
}
```

> últimoDígito(123)	3
> últimoDígito(456)	6
> últimoDígito(9)	9
> últimoDígito(0)	0

E2.6

Llena los huecos, para crear una función que dado un número entero, devuelve verdadero si el número está dentro del rango de 0 a 9 y falso de lo contrario. Revisa los ejemplos para más claridad.

```
 únicoDígito(int n) {  
    return (n  0)  (n  10);  
}
```

> únicoDígito(0)	true
> únicoDígito(1)	true
> únicoDígito(5)	true
> únicoDígito(9)	true
> únicoDígito(10)	false
> únicoDígito(88)	false

E2.7

Llena los huecos, para crear una función que dado un número entero, devuelva el String "no-negativo" si es positivo o cero, o "negativo" si es negativo.

```
String signo(int n) {
    return (n >= 0)        "no-negativo"        "negativo";
}
```

E2.8

Escribe una función esPar() que verifique si un número es par. Un número es par si es múltiplo de 2.

> esPar(2)	true
> esPar(13)	false



Recuerda las funciones que introdujimos en este capítulo y mira si hay una que puedas reutilizar para este ejercicio.

E2.9

Escribe una función esImpar() que verifique si un número es impar. Un número es impar si no es par.

> esImpar(2)	false
> esImpar(13)	true

E2.10

Escriba una función, diferenciaDeCuadrados() que tome dos números enteros y devuelva el resultado de $a^2 - b^2$.

> diferenciaDeCuadrados(3, 2)	5
-------------------------------	---

> diferenciaDeCuadrados(1, 1)	0
-------------------------------	---

E2.11

Escribe una función, `tercerÁngulo()` que tome dos doubles que representan la medida de dos ángulos de un triángulo y devuelva la medida del tercero. Los 3 ángulos de un triángulo suman 180.0, por lo que si restas la suma de dos ángulos a 180.0 deberías obtener el tercer ángulo.

> tercerÁngulo(60.0, 60.0)	60.0
> tercerÁngulo(45.0, 90.0)	45.0
> tercerÁngulo(35.0, 50.0)	95.0

E2.12

Escribe una función que tome un número entero y devuelva un predicado que sea verdadero si es un múltiplo positivo de siete menor que 1000.

> validarMúltiploDe7(-7)	false
> validarMúltiploDe7(0)	false
> validarMúltiploDe7(91)	true
> validarMúltiploDe7(9100)	false

E2.13

Escribe una función `validar()` que tome tres números enteros: inicio, mitad y tamaño y retorne verdadero si <inicio> es mayor o igual a 0, <inicio> es menor que <mitad> y <mitad> es menor que tamaño.

> validar(0, 1, 2)	true
> validar(1, 1, 2)	false
> validar(2, 1, 3)	false
> validar(0, 3, 3)	false
> validar(1, 3, 5)	true



E2.14

Un parqueadero abre a las 8:00 am y cierra a las 6:00 pm. Su sistema de cobro es el siguiente: el usuario paga \$10.0 por cada minuto de parqueo, pero tiene un cobro mínimo de \$100.0 y tiene un cobro máximo de \$3000.0. Haz una función `cobro()` que retorne un número double siendo el cobro a un usuario, dado un número entero que representa los minutos que estuvo su vehículo en el parqueadero.

Para hacer esta función puedes utilizar estos 2 métodos de la clase Math:

- [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#min\(double,double\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#min(double,double))
- [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#max\(double,double\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html#max(double,double))

> cobro(1)	100.0
> cobro(10)	100.0
> cobro(15)	150.0
> cobro(90)	900.0
> cobro(1_000)	3000.0



Es una buena idea escribir código iterativamente. Haz una primera versión de la función que haga lo más sencillo posible. Luego agrega un poquito de funcionalidad y revisa que funciona como esperas y así sucesivamente.

Para este ejercicio la recomendación sería:

- Haz una función que calcule el valor a cobrar solo teniendo en cuenta el tiempo.
- Luego agregar la funcionalidad para que la función nunca devuelva un valor menor de \$100.0.
- Finalmente agrega funcionalidad para que adicionalmente no devuelva un valor mayor de \$3 000.0

Ejercicios de Corrección de Código

Para estos problemas, mira los ejemplos de uso de una función y luego revisa el código inicial que no está funcionando como se espera. Ejecútalo y luego corrígelo para coincidir con los ejemplos.

E2.15

> inicio(6, 1)	0
> inicio(6, 2)	6
> inicio(6, 3)	12
> inicio(10, 2)	10

```
int inicio(int resultados, int página) {  
    return (resultados * página) - 1;  
}
```

3. Variables

Código 1 - hipotenusa sin variables

```
01 double hipotenusa(double a, double b) {  
02     return Math.sqrt((a * a) + (b * b));  
03 }
```

El Código 1 muestra la función hipotenusa() del capítulo anterior. Es una manera de escribirla que es válida, pero suceden muchas cosas en el argumento de Math.sqrt(). Si desea que esta expresión sea más expresiva, podemos agregar variables:

Código 2 - hipotenusa con variables

```
01 double hipotenusa(double a, double b) {  
02     var cuadradoDeA = a * a;  
03     var cuadradoDeB = b * b;  
04     var sumaDeCuadrados = cuadradoDeA + cuadradoDeB;  
05     return Math.sqrt(sumaDeCuadrados);  
06 }
```

Al crear una variable, le das un nombre a un valor. Esto generalmente ayuda a documentar tu código y puede ayudarte a reutilizar el valor sin volver a calcularlo. Pero así como las funciones no tienen nada que ver con la reutilización, las variables tampoco. Tanto las funciones como las variables deben usarse principalmente para mejorar la legibilidad de su código.

Para crear una variable utilizamos la palabra clave <var> y le da un identificador. Luego usa el operador de asignación "=" y por último un valor. Puede ser un literal, otra variable o una llamada a una función. Así como los parámetros tienen tipos, las variables también tienen tipos, de hecho los parámetros son variables. El tipo de una variable se toma del tipo de expresión que utiliza como valor. Por ejemplo: "var cuadradoDeA = a * a", cuadradoDeA tendría el mismo tipo del resultado de la multiplicación. Y el tipo de la multiplicación sería del mismo tipo de las variables <a> y , que son double. Entonces cuadradoDeA es de tipo double.



Desafortunadamente, hacer algo más legible es una cuestión subjetiva. Algunas personas probablemente dirían que el Código 2 no es de ninguna manera más legible que el Código 1. Sin embargo, en caso de duda, siempre opta por "demasiadas" y no por "muy pocas". Tener muchas variables o muchas funciones tiene un impacto negativo mínimo y en algunos casos nulo, en el rendimiento de sus programas. La programación es siempre una negociación

entre fuerzas contrarias: ¿qué es más importante unos milisegundos de rendimiento o la legibilidad de un código?



En Java, una vez que creas una variable, no puedes cambiar de tipo. Por ejemplo, si creas una variable de tipo int, no podrás asignarle después un String.

Código legado: código sin var

Código 3 - hipotenusa usando tipos de datos en lugar de var.

```
double hipotenusa(double a, double b) {  
    double cuadradoDeA = a * a;  
    double cuadradoDeB = b * b;  
    double sumaDeCuadrados = cuadradoDeA + cuadradoDeB;  
    return Math.sqrt(sumaDeCuadrados);  
}
```

El Código 3 era la única manera de escribir código en versiones muy anteriores de Java. Al usar explícitamente el tipo de dato de la variable, el código no cambia realmente en su funcionamiento. ¿Por qué usamos <var>? Cuando hagamos código más complejo, con muchos tipos de datos, usar <var> ayuda a la legibilidad. Usualmente al leer código, cuando ves el valor asignado, el tipo de dato es obvio, así que repetirlo al declarar la variable no ayuda mucho.

En este material usaremos exclusivamente <var>



La palabra clave var se introdujo en Java 10. Es posible que encuentres código en otros lugares o en proyectos antiguos que no lo utilizan.



La forma recomendada de definir variables es con var. Si defines un montón de variables con var, sus nombres estarán alineados, lo que ayuda mucho con la legibilidad.

Cuidados con literales y variables

Escribamos una declaración que parece inofensiva:

```
double uno = 1;
```

¿Ves algo raro? Bueno, el tipo de variable y el literal no coinciden. Recuerda qué `<1>` es un literal entero de tipo `int`. Y la variable `<uno>` es de tipo `<double>`. Entonces, el compilador promueve el literal `<1>` y lo trata como un `<double>` para que la asignación pueda realizarse correctamente. Entonces después de eso `<uno>` tendrá el valor de `<0.0>`

Ahora veamos este código:

```
double uno = 1;
double mitad = uno / 2;
```

Esto produce el valor esperado de `<0,5>`, pero ¿qué sucede si cambia la declaración de variable con `<var>`?

```
var uno = 1;
var mitad = uno / 2;
```

Ahora la mitad es 0. Debido a que `<uno>` es de tipo `int`, porque `<1>` es de tipo `int`, y luego `<uno/2>` opera dos enteros, el resultado es un `int`, por lo que `<0.5>` se convierte en `<0>`.

Pero si escribes:

```
var uno = 1.0;
var mitad = uno / 2.0;
```

Ahora la `<mitad>` es `<0.5>`

Ten mucho cuidado con los valores que utiliza para inicializar las variables para que obtengan los valores esperados.

Números Mágicos

Código 4 - números mágicos


```

01 double tercerÁngulo(double alfa, double beta) {
02     // la suma de los 3 ángulos de un triángulo es siempre 180.0
03     var tercerÁngulo = 180.0 - (alfa + beta);
04     return tercerÁngulo;
05 }

```

En el código anterior, ¿qué es 180.0? Podrías pensar que es "obvio", pero la mayoría de los literales utilizados en las expresiones parecerían "números mágicos", literales que tienen un significado que no queda claro solo por su valor. Podemos agregar un comentario para explicar su significado pero un mejor enfoque sería utilizar una variable para darle un nombre a este valor:

Código 5 - usar una variable para evitar números mágicos

```

01 double tercerÁngulo(double alfa, double beta) {
02     var sumaDeTodoslosÁngulos = 180.0;
03     var sumaDeÁngulosConocidos = alfa + beta;
04     var tercerÁngulo = sumaDeTodoslosÁngulos - sumaDeÁngulosConocidos;
05     return tercerÁngulo;
06 }

```

Con una variable, queda claro qué es lo que el valor intenta representar, mejorando la legibilidad del código.



La mejor forma de documentación es con el código. Usar variables, con buenos nombres, ayuda a evitar los comentarios y aumenta la legibilidad del código.

Reasignaciones

Código 6 - forma caótica de implementar el tercer ángulo

```

double tercerÁngulo(double alfa, double beta) {
    var ángulo = 180.0;
    ángulo = ángulo - alfa; // actualiza el valor del ángulo
                          // restando alfa del valor actual
}

```

```
ángulo = ángulo - beta; // nuevamente, actualiza el ángulo
return ángulo;
}
```

Cuando crea una variable, puede reasignarle un valor, a menos que haga que la variable sea final, lo cual veremos más adelante. Las reasignaciones suelen ser malas, luego veremos un ejemplo que necesita reasignación, pero en general es mejor evitarlas. Si lo piensas bien, una variable le da un nombre a un valor, por lo que si estás asignando, entonces el significado del nombre probablemente no estará relacionado con el nuevo valor. En ese caso, es mejor crear una nueva variable.

Sin embargo, a lo largo de la historia, ¡se ha escrito mucho código con reasignaciones! ¿Porqué es eso? Hubo una época en la que los recursos informáticos eran escasos por lo que había casos en los que era importante no crear demasiadas variables. Hoy en día las computadoras y los compiladores son mucho mejores, por lo que esto ya no es una preocupación.

Resumen

Si intenta empaquetar una gran cantidad de código en una sola línea, afectará la legibilidad del código. En su lugar, puede utilizar variables para dar nombres a ciertos valores. La antigua forma de definir variables es:

```
int n = 1;
```

Pero la forma más nueva es usar var:

```
var n = 1;
```

Las variables definidas con <var> tienen un tipo y se inferirán por el tipo de valor en la asignación.

Cuando utilice literales en su expresión, compruebe si la legibilidad del código se puede mejorar creando una variable para él.

Ejercicios

Errores del compilador y en tiempo de ejecución

Escribe los siguientes códigos y lee el error devuelto por el compilador o el entorno de ejecución. Luego escribe el código correcto.

E3.1

```
int duplicar(double a) {  
    var foo = a * 2;  
    return foo;  
}
```

Ejercicios de Codificación

E3.2

```
 mod(int p, int q) {  
    var d = p / q;  
     r = d * q;  
     m = p - r;  
    return m;  
}
```

> mod(3, 2)	1
> mod(4, 2)	0

E3.3

```
int diferenciaDeCuadrados(int a, int b) {  
    var  = a * a;  
    var cuadradoDeB  b * b;  
    var diferenciaDeCuadrados = cuadradoDeA - cuadradoDeB;  
     diferenciaDeCuadrados;  
}
```

> diferenciaDeCuadrados(3, 4)	-7
> diferenciaDeCuadrados(2, 2)	0



Recuerda que el uso o no uso de variables, en muchos casos es una decisión de estilo. Si te parece mejor escribir una función sin variables, hazlo. Si al terminar sientes que el código es suficientemente claro, entonces no está mal que no las hayas usado. Si por el contrario sientes que la lectura se hace complicada, trata de introducir variables para hacer el código más legible.

E3.4

Escribe una función que calcule la distancia entre dos puntos. Cada punto está representado por 2 doubles; la función toma 4 parámetros: x1, y1, x2 e y2; los dos primeros son las coordenadas x e y del primer punto y los dos últimos son las coordenadas x e y del segundo punto. La fórmula de la distancia es:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

> distancia(0.0, 0.0, 1.0, 1.0)	1.4142135623730951
> distancia(0.0, 3.0, 4.0, 0.0)	5.0



Con las herramientas que tenemos no hay una mejor manera de hacer este código, pero mira que solo mirando el código no es claro que <x1> e <y1> están relacionadas, más allá del nombre, y de hecho tampoco es claro que representan un punto. Igual con <x2> e <y2>. ¿Cómo se te ocurre que esto se podría mejorar?

E3.5

Escribe una función que, dada una temperatura medida en Fahrenheit como double, devuelva el valor equivalente en Celsius. Utiliza la fórmula: $C = (F - 32) * 5/9$

> fahrenheitACelsius(60.0)	15.555555555555557
> fahrenheitACelsius(32.0)	0.0

E3.6

Escribe una función que calcule la gravedad superficial de un planeta dada la masa y el radio de este. La fórmula para la gravedad superficial es:

$$\frac{G * masa}{radio^2}$$

en donde G es la constante gravitacional que es 6.67430×10^{-11} .



En muchos lenguajes de programación, Java incluido, hay una notación para escribir literales para números reales que hace muy práctico escribir números como el de la constante gravitacional. En esta notación, esta constante se escribe `6.67430e-11`

<pre>> gravedadSuperficial(5.972e24, 6.378e6)</pre>	9.798429966761134
<pre>> gravedadSuperficial(6.421e23, 3.3972e6)</pre>	3.713352371667296

E3.7

Escribe una función que calcule la gravedad superficial de la tierra. Para este ejercicio asume que la masa de la tierra en literal de doubles es `5.972e24` y el radio es `6.378e6`. Utiliza la fórmula del ejercicio anterior.

<pre>> gravedadSuperficialDeLaTierra()</pre>	9.798429966761134
---	-------------------

E3.8

Escribe una función que tome el peso de una persona en la tierra, y devuelva el peso de esa persona en Marte. Para eso, utiliza la fórmula:

$$Peso_{Marte} = \frac{GravedadSuperficial_{Marte}}{GravedadSuperficial_{Tierra}} * Peso_{Tierra}$$

Para este ejercicio asume que la masa y el radio de Marte son, respectivamente: `6.421e23` y `3.3972e6`

<pre>> pesoEnMarte(80.0)</pre>	30.31793774524261
<pre>> pesoEnMarte(65.0)</pre>	24.63332441800962



Estos ejercicios de física planetaria fueron adaptados de este artículo sobre Enums en Java:

<https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>

Nosotros hablaremos de Enums más adelante.



E3.9

Un Triángulo es un polígono cerrado de 3 lados. Cada lado tiene una longitud, pero ¿crees que dadas 3 longitudes aleatorias siempre puedes dibujar un triángulo con ellas? La respuesta es no, hay una propiedad de los 3 lados que debe ser cierta para formar un triángulo: el lado más largo debe ser menor que la suma de los otros 2 lados.

Escribe una función que dadas las longitudes de 3 lados, verifique si se puede formar un triángulo.

Hay una manera de escribir la función probando todas las combinaciones posibles, pero hay una manera ingeniosa de usar `Math.max(double,double)` y otras operaciones para hacer la verificación.



Nuevamente, la documentación oficial de `Math.max(double,double)`:
[https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html#max\(double,double\)](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Math.html#max(double,double))

```
> esTriángulo(3.0, 4.0, 5.0)
> esTriángulo(17.0, 10.0, 7.0)
```

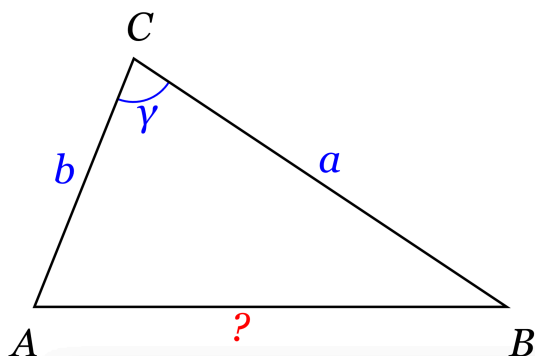
```
true
false
```



Algo importante a tener en cuenta, es que el ejercicio no pide saber cuál es el parámetro que tiene el valor más grande, solo pide que lo puedas comparar con la sumatoria de los más pequeños.

E3.10

Los triángulos han jugado un papel muy importante en la historia del hombre. Gracias a sus propiedades, es posible saber cuánto mide una montaña sin ningún instrumento sofisticado y sin tener que subirse a ella. Una de las fórmulas relacionadas con los triángulos es la llamada Ley de Cosenos que dice que para un triángulo cualquiera, si conozco las medidas de 2 lados, llamémoslos a y b , y el ángulo que forman:



Podemos calcular el lado desconocido con la fórmula:

$$c = \sqrt{a^2 + b^2 - 2 * a * b * \cos(\textit{gamma})}$$

Haz una función que reciba, la medida de *a*, de *b* y de *gamma*, medido en grados, y retorne el valor de *c*.

Para esto lee la documentación de la clase Math y busca:

- Un método para elevar al cuadrado o en su defecto para hallar potencias (power en inglés)
- Un método para convertir un ángulo medido en grados (degrees en inglés), a un ángulo medido en radianes (radians en inglés)
- Un método que dado un ángulo en radianes, devuelva su coseno (cosine en inglés)



La documentación oficial de Math:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Math.html>



Además de la documentación oficial también podrías usar buscadores en Internet para encontrar estas respuestas.

> leyCosenos(3.0, 4.0, 90.0)	5.0
> leyCosenos(11.0, 11.0, 60.0)	10.999999999999998

Ejercicios de Corrección de Código

Para estos problemas, mira los ejemplos de uso de una función y luego revisa el código inicial que no está funcionando como se espera. Ejecútalo y luego corrígelo para coincidir con los ejemplos.

E3.11

<code>promedio(11, 16)</code>	13.5
<code>promedio(2, 3)</code>	2.5

```
double promedio(int edadMenor, int edadMayor) {  
    var sumaEdades = edadMenor + edadMayor;  
    var promedio = sumaEdades / 2;  
    return promedio;  
}
```


4. String y StringBuilder

Código 1 - substring()

```
01 String teléfonoConFormato(String teléfono) {  
02     var primerosTres = teléfono.substring(0, 3);  
03     var siguientesTres = teléfono.substring(3, 6);  
04     var últimos = teléfono.substring(6);  
05     var conFormato = String.format("(%s) %s-%s", primerosTres, siguientesTres, últimos);  
06     return conFormato;  
07 }
```

> teléfonoConFormato("1234567890")	(123) 456-7890
------------------------------------	----------------

El primer tipo de dato que usamos fue String. Ahora vamos a profundizar en él.

Las Strings están presentes en casi todo lo que hacemos en programación, ¿se podría decir que están sobre utilizados! Como recordatorio, un String es una secuencia de caracteres: letras, dígitos, signos de puntuación, emojis, etc.

Un String no es un tipo de dato primitivo, es una clase. Más adelante profundizaremos en qué es una clase, y cómo crear clases nuestras. Una diferencia fundamental entre datos primitivos y clases es que estás últimas pueden definir métodos que se llaman de una manera diferente de la que hemos usado hasta ahora.

En la línea 2, llamamos el método substring() de la variable <teléfono>. Lo importante es que a diferencia de métodos como <Math.sqrt> o <String.format>, aquí el método se llama sobre una variable. Podemos pensarlo como que cada String creado, tiene un comportamiento que se accede mediante sus métodos.

Dado que un String es una cadena de caracteres, internamente se le asigna un índice a cada carácter para poder hacer referencia a ellos individualmente. Pero tiene algo extraño y es que el índice empieza en 0 y no en 1 como quizás uno esperaría.

H	o	l	a		M	u	n	d	o	!
0	1	2	3	4	5	6	7	8	9	10

La cadena "Hola Mundo!" que tiene 11 caracteres, tiene índices que empiezan en 0 y el último es el total de caracteres, su tamaño, menos 1. El carácter en el índice 0 es 'H', el carácter en el índice 4 es ' ' y en el índice 5 es 'M'. El índice del carácter 'a' es 3, mientras que el primer índice de 'o' es 1 y el último es 9.



El primer índice de una string es siempre 0 y el último índice es la longitud del String menos 1.

Uno de los aspectos más importantes con un String es poder extraer información de él y `substring()` es una de las formas más comunes de hacerlo. Como su nombre lo indica, devuelve solo una parte de un String, utilizando índices para indicar donde se hacen los cortes.

En la línea 02, `<teléfono.substring(0, 3)>` retornaría todos un String que consiste únicamente de los caracteres 0, 1 y 2. El último índice de `substring()` no está incluido en el String resultante.



Comenzar índices en 0 no es algo que se haya hecho sólo por sorprender, hay una razón muy práctica por la que se hizo así. Lo que hace el índice es indicar el desplazamiento que debes hacer desde el inicio de la cadena hasta el caracter deseado. El desplazamiento desde el inicio de la cadena hasta el primer caracter es 0 porque este mismo es el inicio de la cadena. ¿Crees qué fue error implementarlo de la manera más sencilla para las máquinas así resulte inicialmente poco intuitivo para los programadores?

Los parametros son de tipo `int` lo que quiere decir que puedes usar índices negativos, pero esto siempre producirá un error en tiempo de ejecución. Igualmente si usas un índice más grande que el máximo índice válido recibirás un error en tiempo de ejecución.



Si utilizas un índice que no sea válido, menor que 0 y mayor o igual al tamaño del String obtendrás un error en tiempo de ejecución.

Pero la clase String tiene 2 métodos con el mismo nombre `<subtring>` uno de ellos toma un `int`, y el otro toma dos. Esto, que 2 métodos se llamen igual pero tengan diferentes parámetros, se llama sobrecarga de métodos. Por ahora lo importante es entender qué diferencia hay entre ambos. La versión que solo toma 1, asume que el índice final es el final de la cadena.

Estas dos expresiones son equivalentes:

```
teléfono.substring(6); // todos los dígitos después del índice 6
```

Y

```
// todos los dígitos entre 6 y
// el índice que es 1 menor que la longitud del String
teléfono.substring(6, phone.length() - 1);
```

Como habrás adivinado, existe un método String para calcular su longitud, llamado apropiadamente `length()`, que resulta útil si deseas encontrar el índice del último carácter, ya que solo necesitas restar 1 a la longitud.

Esta no es una lista exhaustiva de los métodos de String, pero te brindará suficientes herramientas para poder descubrir cómo usar los otros métodos que encontrarás en la documentación.



Documentación oficial de String:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

String Format

Crear un String a partir de cierta información es una tarea muy común en programación. Como vimos anteriormente, `String.format()` es una forma práctica de hacerlo. Hay muchas cosas sobre ese método, que merecen que leas la documentación oficial o busques ejemplos. Pero vale la pena mostrar algunos ejemplos.

`String.format()` no solo funciona con Strings, funciona con otros tipos de datos:

<pre>// %d para ints String.format("%d", 123);</pre>	<pre>"123"</pre>
<pre>// %f para doubles String.format("%f", 3.14);</pre>	<pre>"3.140000"</pre>

Incluso podemos hacer conversión de números enteros a base 16:

<pre>// %x para dígitos en minúsculas String.format("%x", 123);</pre>	<pre>"7b"</pre>
<pre>// %X para dígitos en mayúsculas String.format("%X", 123);</pre>	<pre>"7B"</pre>

También podemos convertir Strings a mayúsculas:

<code>// %s no modifica el String String.format("%s", "Código");</code>	"Código"
<code>// %S transforma a mayúsculas String.format("%S", "Código");</code>	"CÓDIGO"

También podemos garantizar que el String resultante tenga un tamaño mínimo, utilizando un número entero entre el % y el indicador del tipo de dato:

<code>String.format("%s", "Casa");</code>	"Casa"
<code>String.format("%8s", "Casa");</code>	" Casa"
<code>String.format("%d", 6789);</code>	"6789"
<code>String.format("%6d", 6789);</code>	" 6789"

Para los números, puede indicar que en vez de espacios se utilicen ceros:

<code>String.format("%06d", 6789);</code>	"006789"
---	----------

Por último para números reales, podemos indicar cuántos número decimales queremos en el String resultante, utilizando un número antecedido por un punto. El resultado va a ser redondeado:

<code>String.format("%f", Math.sqrt(2.0));</code>	"1.414214"
<code>String.format("%.3f", Math.sqrt(2.0));</code>	"1.414"

Todas estas reglas pueden ser combinadas. Y recuerda que hay aún otras funcionales adicionales, aunque estás son posiblemente las que más puedas usar.

Métodos Destacados de String

Otros métodos que podrías utilizar con frecuencia:

- [charAt\(int\)](#)

<code>> "Hola".charAt(0)</code>	'H'
<code>> "Hola".charAt(3)</code>	'a'

- [contains\(String\)](#)

> "casa".contains("as")	true
> "casa".contains("Ca")	false
> "casa".contains("sas")	false
> "casa".contains("me")	false

- [equals\(Object\)](#)

> "hola".equals("hola")	true
> "hola".equals("adios")	false
> !("hola".equals("adios"))	true

- [indexOf\(String\)](#)

> "hola".indexOf('h')	0
> "hola".indexOf('a')	3
> "hola".indexOf('z')	-1
> "hola".indexOf('H')	-1

- [indexOf\(String, int\)](#)

> "casa".indexOf('a')	1
> "casa".indexOf('a', 2)	3

- [lastIndexOf\(String\)](#)
- [lastIndexOf\(String, int\)](#)

> "casa".lastIndexOf('a')	3
> "casa".lastIndexOf('a', 2)	1

Métodos relacionados con Strings en otras clases:

- [Integer.parseInt\(String\)](#)

> Integer.parseInt("56")	56
--------------------------	----



¿Si usamos == y != para primitivas, también podemos usarlo con String? ¡No! Al no ser un dato primitivo, esos operadores tienen un significado diferente, que a veces pueden dar el resultado correcto. Como está en la lista anterior, equals() y equals() son la manera correcta de revisar que 2 String sean o no iguales.

Código 3 - Parsear un String para extraer tiempos

```
01 // Un String de tiempo en formato ISO 8601
02 // p.j. 2021-11-20T13:15:00 pero también 08:05:00
03 int tiempoEnSegundos(String fecha) {
04     var separador = fecha.indexOf('T'); // puede ser -1
05     var tiempo = (separador != -1) ? fecha.substring(separador + 1) : fecha;
06
07     var horas = Integer.parseInt(tiempo.substring(0, 2));
08     var minutos = Integer.parseInt(tiempo.substring(3, 5));
09     var segundos = Integer.parseInt(tiempo.substring(6));
10
11     var SEGUNDOS_POR_MINUTO = 60;
12     var MINUTOS_POR_HORA = 60;
13     var SEGUNDOS_POR_HORA = SEGUNDOS_POR_MINUTO * MINUTOS_POR_HORA;
14
15     return SEGUNDOS_POR_HORA * horas + SEGUNDOS_POR_MINUTO * minutos + segundos;
16 }
```

```
> tiempoEnSegundos("2023-04-20T00:00:15")
> tiempoEnSegundos("2023-04-20T00:01:00")
> tiempoEnSegundos("2023-04-20T00:10:00")
> tiempoEnSegundos("2023-04-20T01:00:00")
> tiempoEnSegundos("00:00:32")
> tiempoEnSegundos("00:02:00")
> tiempoEnSegundos("10:00:00")
```

En la línea 04 utilizamos el método indexOf() para encontrar el índice en el que se encuentra el caracter 'T'. El estándar ISO 8601 indica que un tiempo que incluye una fecha y un tiempo, utiliza ese caracter para separar los datos de la fecha del tiempo. Pero si el String solo indica un tiempo entonces no hay 'T', así que nuestro primer paso es garantizar que el String no incluya la parte de la fecha.

El método indexOf() retorna el menor índice en el que se encuentra un caracter dado. El estándar es claro que si hay un 'T', solo hay uno así que buscar el primero, equivale a buscar el único.

La primera acción que queremos hacer es quitar la información de la fecha si es que existe. Utilizamos el operador ternario para crear una nueva variable <tiempo> que haga un substring() si existe el caracter 'T', y en caso contrario sea una copia de la variable <fecha>

Ya garantizado que la variable <tiempo> no incluye información sobre una fecha, entonces queremos extraer los números que se encuentran entre caracteres ':', utilizando nuevamente substring(), pero usando el valor de retorno de estos como argumento de Integer.parseInt() un método de una clase diferente de String, pero muy relacionada, porque convierte un número entero dentro de un String, en un int. Lo importante de esta operación es que nos va a permitir multiplicar y sumar ese valor.

Finalmente sabemos que hay 60 minutos en una hora y 60 segundos en un minuto, por lo tanto podemos hacer una fórmula para terminar de encontrar cuántos segundos desde el inicio del día han transcurrido hasta la hora consignada en el String.

StringBuilder

Código 2 - StringBuilder

```
01 String phoneFormatter(String teléfono) {
02     var primerosTres = teléfono.substring(0, 3);
03     var siguientesTres = teléfono.substring(3, 6);
04     var últimos = teléfono.substring(6);
05
06     var builder = new StringBuilder();
07     builder.append("(");
08     builder.append(primerosTres);
09     builder.append(") ");
10     builder.append(siguientesTres);
11     builder.append(" - ");
12     builder.append(últimos);
13     var formattedPhone = builder.toString();
14     return formattedPhone;
15 }
```

El código 2 muestra una implementación alternativa de phoneFormatter utilizando StringBuilder. Puede que esta no sea la mejor implementación, pero nos permite hablar de un tipo de datos importante: String Builder.

String.format() es una de las formas de construir strings, ¡pero no es la única! String.format() es muy bueno cuando tienes un patrón para String y quieres interpolar algunas variables. Pero tal vez quieras construir un String paso a paso, y eso es lo que StringBuilder hace mejor.

Un `StringBuilder` es un tipo que permite crear un `String` agregándole datos.

La línea 05 es cómo se crea un `StringBuilder` vacío. Los detalles de qué <nuevo> se discutirán más adelante, por ahora recuerda que lo necesitas para crear este tipo. Con un `StringBuilder` puedes llamar a `append` tantas veces como quieras y usar cualquier tipo de variable que tengas para crear el `String`, paso a paso.

La línea 06 agrega el literal `String` `"("`. Luego la línea 07 agrega la variable `firstThree`, que es un `String`. Pero no importaría si fuera de otro tipo, ya que `append()` puede aceptar cualquier tipo. Luego las siguientes 4 líneas agregan la información restante necesaria para crear la string. Después de agregar todos los datos necesarios, puede llamar a `toString()` para convertir `StringBuilder` en una string en la línea 12, y finalmente devolver la string resultante en la línea 13.

`StringBuilder` tiene más métodos para ayudarle a construir `String` pero no son importantes por ahora, `append()` probablemente sería el método más utilizado de todos modos.



Documentación oficial de `StringBuilder`:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuilder.html>

Misión secundaria: inmutabilidad

Código 3 - `immutable` vs `mutable`

```
01 String stringEsImmutable(String s) {  
02     s.substring(1);  
03     return s;  
04 }  
05  
06 StringBuilder Elconstructordestringsesmutable(StringBuilder builder) {  
07     builder.substring(1);  
08     return builder;  
09 }
```

```
> stringsAreImmutable("foo")  
> stringBuilderAreMutable(new StringBuilder("foo"))
```

Presentar `StringBuilder` nos ayuda a hablar sobre un gran tema en programación: la inmutabilidad. En Java, las strings son inmutables, mientras que los `StringBuilders` no lo son.

Los tipos inmutables una vez creados no se pueden cambiar, sin importar qué métodos los invoques. Para tipos mutables, normalmente su método cambia su valor.

En `stringsAreImmutable()` primero llamamos a `substring()` y luego lo devolvemos. Como las strings son inmutables, `substring()` no cambia la string, crea una nueva string con el valor actualizado. Recuerde que la `substring` toma 1 o 2 argumentos, si usa 1 devolverá la substring después del índice dado, y como el primer índice de una string es 0, `s.substring(1)` devuelve la substring de `s` comenzando desde el segundo carácter.

En `stringBuilderAreMutable()` también llamamos a `substring()` pero esta vez es el método de `StringBuilder` el que es mutable. Después de llamarlo, el valor del argumento cambia y después de devolverlo verás que ha cambiado. Para demostrarlo, necesita crear un `StringBuilder`, lo creamos antes cuando estaba vacío, pero primero puede crear un `StringBuilder` a partir de un `String` como un nuevo `StringBuilder` ("foo")

Como otro ejemplo, si revisa el Código 2, verá que cuando llamamos a métodos en una string, como en las líneas 02 a 05, necesitamos capturar el tipo de retorno del método en una variable. En las líneas 06 - 11 llamamos a los métodos sin necesidad de una variable porque después del método, se cambiará el valor del `StringBuilder`. Cuando utilice un tipo, preste mucha atención para ver si es inmutable o mutable.

La inmutabilidad es un tema importante cuando creas tus propios tipos. Cuando utilice tipos ya definidos, simplemente lea la documentación o estudie cómo funcionan sus métodos para saber si es mutable o no.

Resumen

¡Los Strings son muy importantes en la programación! Es importante saber cómo crearlos y manipularlos. Para crearlos, la forma recomendada es `String.format()` y `StringBuilder`. Se prefiere lo primero si se puede crear la string en un solo paso, utilizando los datos disponibles y proporcionando una plantilla. Se prefiere lo último si crea la string en pasos, cómo en un bucle que verá en el siguiente capítulo.

Hay muchos métodos en la API de Strings, probablemente los más utilizados son: `charAt`, `contains`, `compareTo`, `equals`, `indexOf` y `lastIndexOf`.

Debido a que las strings a veces contienen información confidencial, son inmutables: ¡eso significa que una vez que se crea una string no puedes modificarla! Entonces, los métodos que parecen hacerlo, en realidad, devuelven una nueva string con los estados actualizados. Es muy importante que se sienta cómodo manejando tipos inmutables antes de profundizar en un capítulo posterior.

Ejercicios

Errores del compilador y del tiempo de ejecución

Escribe los siguientes códigos y lee el error devuelto por el compilador o el entorno de ejecución. Luego escribe el código correcto.

E4.1

```
boolean empiezaConMinúscula(String s) {  
    var primerCaracter = String.charAt(s, 0);  
    return Character.isLowerCase(primerCaracter);  
}
```

> empiezaConMinúscula("manzana")	true
> empiezaConMinúscula("Manzana")	false



Método isLowerCase de Character:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isLowerCase\(char\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isLowerCase(char))

E4.2

```
String pluralizar(int conteo, String s) {  
    return (conteo < 2) ? s : s.format("%d %ss", conteo);  
}
```

> pluralizar(1, "código")	"código"
> pluralizar(2, "código")	"2 códigos"

E4.3

```
boolean esVocal(char letra) {  
    var letraMinúscula = Character.toLowerCase(letra);  
    return 'aeiou'.contains(Character.toString(letraMinúscula));  
}
```

> esVocal('a')	true
----------------	------

> esVocal('A')	true
> esVocal('c')	false
> esVocal('@')	false



Método toLowerCase de Character:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toLowerCase\(char\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toLowerCase(char))



Método contains de String:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#contains\(java.lang.CharSequence\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#contains(java.lang.CharSequence))



Método toString de Character:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toString\(char\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toString(char))

Ejercicios de codificación

E4.4

Escribe una función que tome un String y lo devuelva con la primera letra en mayúscula y las demás en minúsculas (en inglés esto se llama title case)

```
String titleCase(String texto) {
    var primeraLetra = texto.substring(0, 1);
    var restoDelTexto = texto.substring(1);
    return String.format("%S%s", primeraLetra, restoDelTexto.toLowerCase());
}
```

> titleCase("java")	"Java"
> titleCase("Java")	"Java"
> titleCase("JAVA")	"Java"

E4.5

Escribe una función terminaEnS() que verifique si una palabra termina en la letra S.

```
boolean terminaEnS(String texto) {  
    var últimoCaracter = texto.lastIndexOf('');  
    return (últimoCaracter == 's') || (últimoCaracter == 'S');  
}
```

> terminaEnS("miércoles")	true
> terminaEnS("jueves")	true
> terminaEnS("sábado")	false

E4.6

Es útil cuando se intenta escribir una función, codificar primero una versión más simple de la misma y extenderla desde allí. Escribe una función que verifique si un String de dos letras es un palíndromo. Un palíndromo es una oración que se lee igual independientemente de dónde se lea, p.j. "Anita, lava la tina".

> esPalindrome("ab")	false
> esPalindrome("ee")	true
> esPalindrome("Aa")	true

E4.7

Escribe una función esExterno() que tome una string que represente un enlace y devuelva verdadero si el enlace es externo. Para este ejercicio tenga en cuenta que los enlaces internos comienzan con "/" y los enlaces externos utilizan los protocolos "http" o "https".

> esExterno("/definetely-internal")	false
> esExterno("https://www.java.com")	true
> esExterno("http://dont-go.com")	true

E4.8

Escribe una función extensión() que tome un String que represente una ruta a un archivo y devuelva la extensión del archivo, todo lo que viene después del último punto del archivo que es parte del nombre del archivo. Las rutas pueden ser relativas o absolutas. Recuerda que es posible que un archivo no tenga extensiones.

> extension("debug.sh")	"sh"
> extension("/path/to/debug.sh")	"sh"
> extension("/the.dot/file.java")	"java"
> extension("no-extension")	""

E4.9

El análisis de Strings es muy común en la programación y generalmente existen algunos tipos o bibliotecas que ayudan con eso. Pero a veces solo necesitas una operación muy específica, por lo que puede tener sentido escribir una función básica en lugar de agregar una dependencia adicional. Escribe una función que tome una URL y devuelva el dominio, la parte que viene después del protocolo y termina en la siguiente barra o al final de la string si no hay ninguna. Puede suponer que todos los parámetros comenzarán con "https://", que es el protocolo.

> domain("https://www.java.com/en/")	"www.java.com"
> domain("https://jcp.org")	"jcp.org"

E4.10

¿Te has preguntado cómo se calcula la Semana Santa? El domingo de Pascua es el primer domingo después de la primera luna llena después del equinoccio de primavera. Dado que las lunas llenas se rigen por un ciclo distinto de los meses, cada año esta fecha cambia. Resulta que hay un algoritmo que puede ayudar a tener en cuenta todos estos sucesos y es el que se utiliza hoy en día para determinar esa fecha. El algoritmo, dado el año, es el siguiente:

<pre> var a = año % 19; var b = año / 100; var c = año % 100; var d = b / 4; var e = b % 4; var g = ((8 * b) + 13) / 25; var h = ((19 * a) + b - d - g + 15) % 30; var i = c / 4; var k = c % 4; var l = (32 + (2 * e) + (2 * i) - h - k) % 7; var m = (a + (11 * h) + (19 * l)) / 433; // n es el número del mes, 3 = marzo, 4 = abril var n = (h + l - (7 * m) + 90) / 25; // p es el día del mes var p = (h + l - (7 * m) + (33 * n) + 19) % 32; </pre>	
--	--

Por ejemplo, si el año fuera 2024, entonces $n = 3$, $p = 31$.

Escribe una función `domingoDePascua(int)` que dado un parámetro entero, el año a averiguar, retorne la fecha del domingo de pascua para ese año como un String con el formato "YYYY/mm/dd" es decir el año con sus 4 dígitos, el mes con 2 dígitos "03" para marzo y "04" para abril y el día también con 2 dígitos, si el día es menor de 10 con un 0 por delante, p.j. "05", "21", etc.

> domingoDePascua(2024)	"2024/03/31"
	"2025/04/20"

> domingoDePascua(2025)	"1961/04/02"
> domingoDePascua(1961)	



Regresar una fecha como un String no es la mejor opción aunque es la única de la que disponemos por ahora. ¿Qué dificultades crees que puede tener?

E4.11

Hay muchos métodos para representar colores en un computador. Uno de ellos es usando el modelo de color RGB en que los colores se describen utilizando diferentes intensidades de 3 colores: Rojo, Verde (Green en inglés) y Azul (Blue en inglés) Esta intensidad es un valor entre 0 y 255.

Aunque uno podría representar un color RGB como un conjunto de 3 números entre 0 y 255, la manera más común de representarlos es con un String de 6 dígitos, donde los primeros 2 representan la intensidad del rojo, los siguientes 2 la intensidad del verde y los últimos 2 la intensidad del azul pero escritos en base hexadecimal o base 16 (la razón siendo que todos los números entre 0 y 255 se pueden escribir máximo con 2 dígitos)

Escribe una función `hexString(int, int, int)` que reciba 3 números enteros, en el rango de 0 a 255, y retorne un String con todos los caracteres en mayúsculas representando el número RGB como un String.

> <code>hexString(0, 0, 0)</code>	"000000"
> <code>hexString(255, 255, 255)</code>	"FFFFFF"
> <code>hexString(12, 15, 77)</code>	"0C0F4D"



E4.12

Escribe una función `correoElectronicoValido()` que, dado un correo electrónico, devuelva verdadero si es un correo electrónico válido. A los efectos del ejercicio un correo electrónico válido tiene estas características:

- Tiene solo una @
- Tiene solo un "." después de la @
- Hay al menos un carácter antes de @
- Hay al menos un carácter entre @ y el último "."
- Hay al menos un carácter después del último "."

> correoElectronicoValido("abc@me.com")	true
> correoElectronicoValido("abc@me.")	false
> correoElectronicoValido("abc@me.com")	false
> correoElectronicoValido("abc@me@com")	false
> correoElectronicoValido("@me.com")	false
> correoElectronicoValido("a.bc@me.com")	true
> correoElectronicoValido("abc@me.co.m")	false
> correoElectronicoValido("a@c@me.com")	false

E4.13

En sitios web donde se muestra mucho texto en combinación con otros elementos, es usual tener que cortar un título por temas de espacio. Pero si cortamos un título, es importante que podamos transmitir al lector que está leyendo un título acortado y usualmente se hace esto agregando puntos suspensivos. A esta operación de acortar un título y agregar puntos suspensivos si el título fue acortado, lo llamaremos elipsificar (porque en inglés el caracter de los puntos suspensivos se llama ellipsis)

Escribe una función para elipsificar un String. Así es como debería funcionar:

1. Encuentra el primer espacio del título dado a partir de la longitud deseada.
2. Si existe, corta el título en ese espacio.
3. Si el título resultante es más corto que el inicial, agrega el carácter de puntos suspensivos "..."



Aunque en español el caracter se llama puntos suspensivos, es importante aclarar que no son 3 puntos uno detrás de otro. De hecho es un caracter específico.

> elipsificar("Java is fun", 4)	"Java..."
> elipsificar("Java is fun", 6)	"Java is..."
> elipsificar("Java is fun", 11)	"Java is fun"
> elipsificar("Java is fun", 20)	"Java is fun"

Para todos los ejemplos, ten en cuenta que "Java is fun" es un String de 11 caracteres. El primer ejemplo busca acortar el título para que tenga solo 4 caracteres. El primer espacio después del cuarto caracter es el que se encuentra después de "Java" que tiene una longitud menor que el original, por lo tanto el String retornado es "Java...". Para el segundo ejemplo, la longitud máxima es 6, pero no podemos partir ahí mismo porque partiríamos la palabra "is", el primer espacio después de ese espacio es precisamente al final de la palabra "is". El String retornado es "Java is...". Para los últimos 2 ejemplos, la cadena cuenta ya con menos o la misma cantidad de caracteres necesaria así que retornamos la cadena sin ningún cambio.

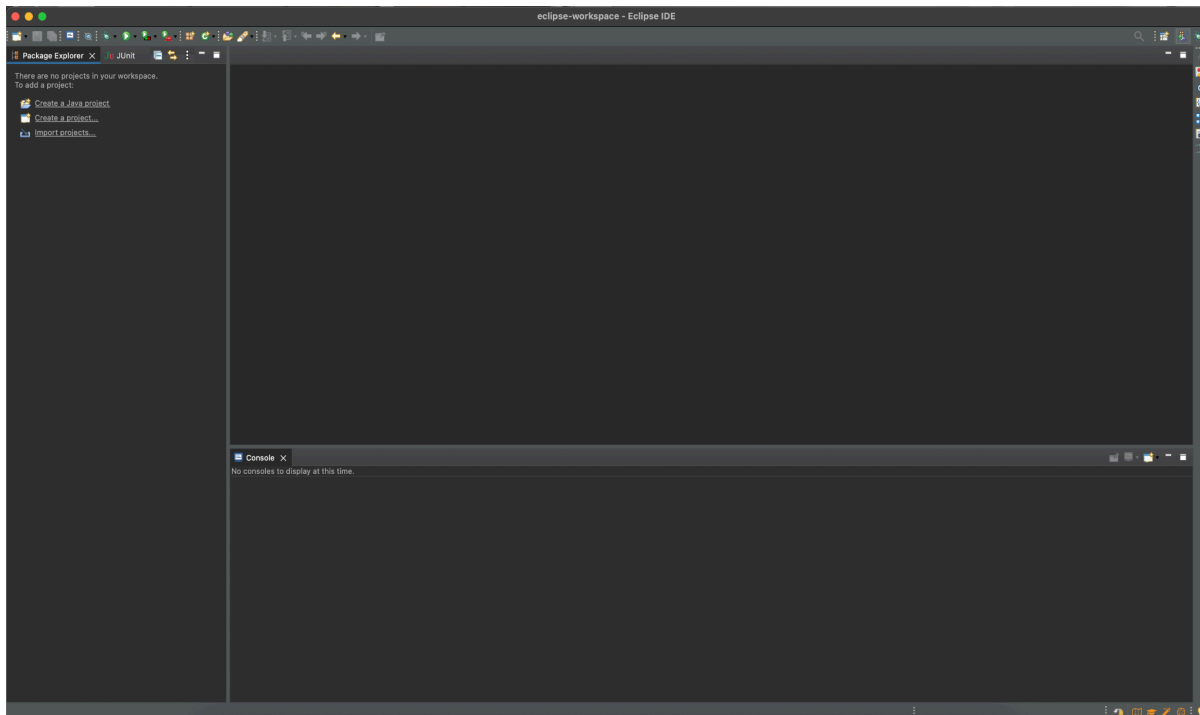
IDE

JShell es una excelente herramienta para la experimentación, perfecta a la hora de iniciar el aprendizaje de programar. Pero cuando intentas hacer programas más complejos necesitas un poco más. Sin embargo eso no significa que JShell no siga siendo útil, aún con nuevas herramientas el mejor lugar para experimentar sigue siendo ahí.

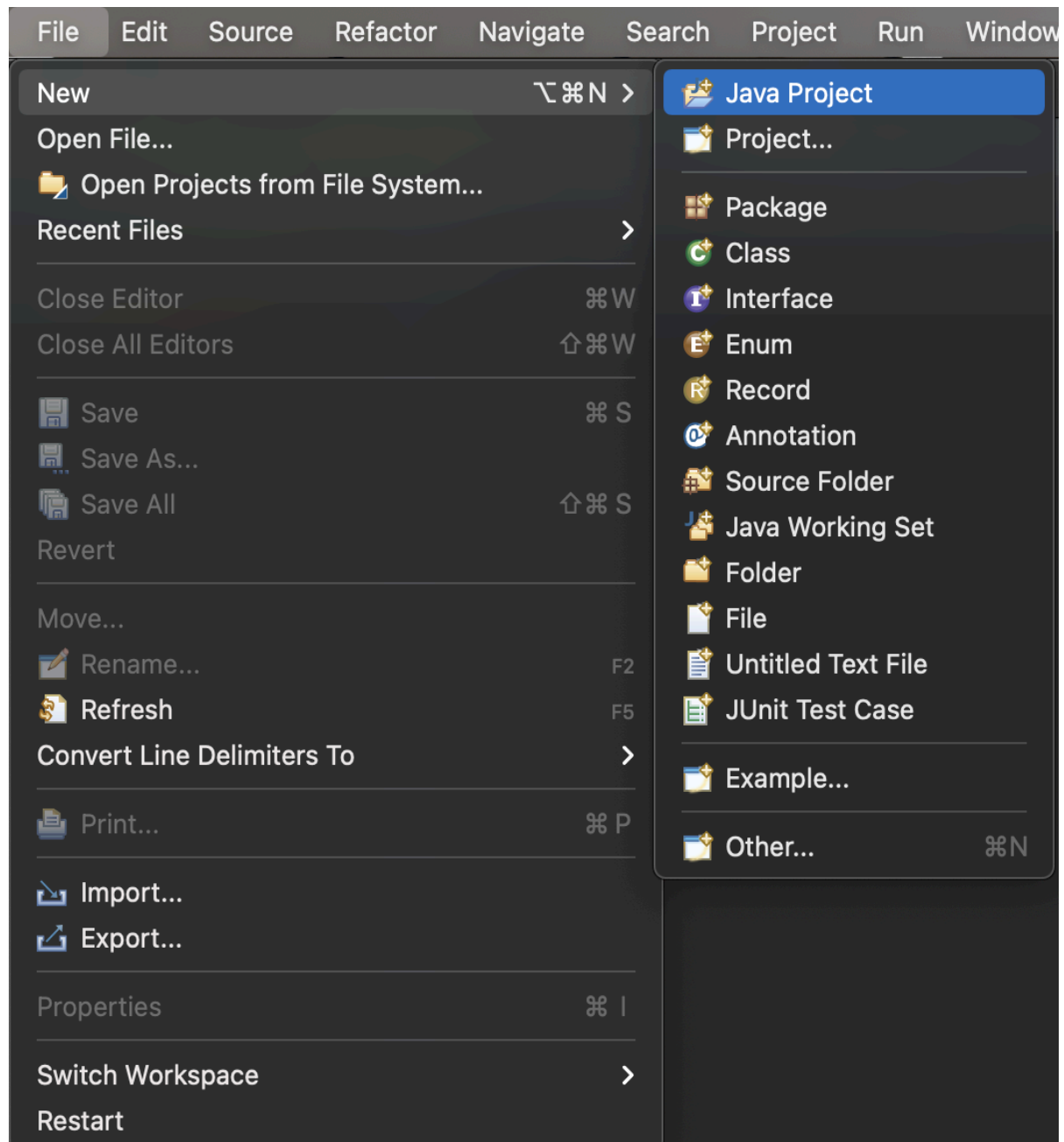
En el mercado existen muchos Entornos de Desarrollo Integrados, o IDE, pero vamos a utilizar el primero que fue importante para el lenguaje Java: Eclipse.

Puedes instalarlo aquí: <https://www.eclipse.org/downloads/>

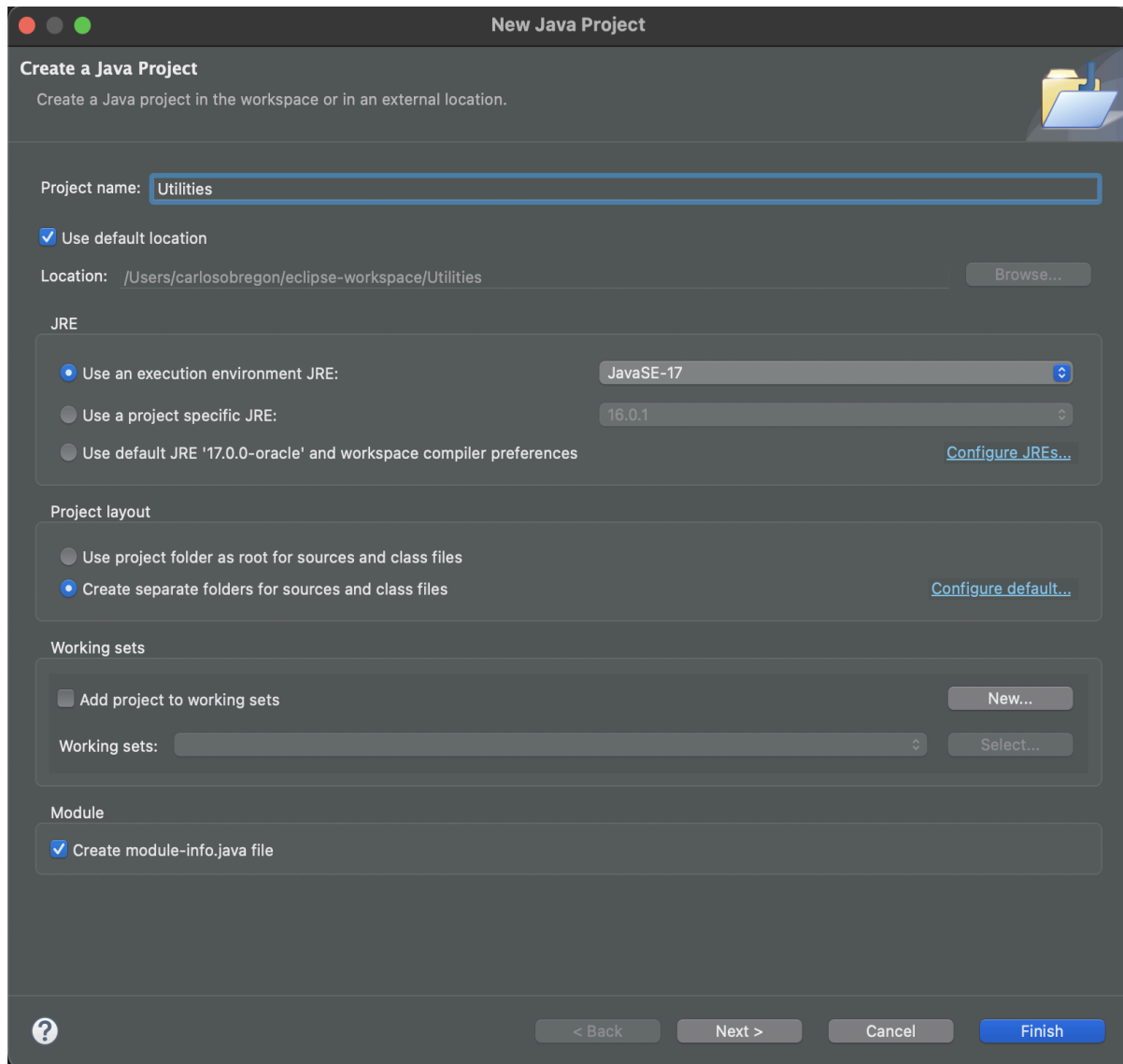
Cuando lo inicies obtendrás algo similar a (las imágenes muestran el modo oscuro):



Desde el menú Archivo, puedes elegir crear un nuevo Proyecto:



A continuación, debes elegir un nombre. Elige un nombre para el proyecto, algo como: Utilidades, Búsqueda de Palabras, etc. Para las demás opciones puedes usar sus valores predeterminados:



The screenshot shows the 'New Java Project' dialog box in Eclipse. The title bar says 'New Java Project'. Below the title bar, there's a subtitle 'Create a Java Project' and a description 'Create a Java project in the workspace or in an external location.' with a folder icon. The 'Project name' field is filled with 'Utilities'. The 'Use default location' checkbox is checked. The 'Location' field shows the path '/Users/carlosobregon/eclipse-workspace/Utilities' with a 'Browse...' button. The 'JRE' section has three radio buttons: 'Use an execution environment JRE:' (selected), 'Use a project specific JRE:', and 'Use default JRE '17.0.0-oracle' and workspace compiler preferences'. The first option has a dropdown menu showing 'JavaSE-17'. The second option has a dropdown menu showing '16.0.1'. There is a 'Configure JREs...' link. The 'Project layout' section has two radio buttons: 'Use project folder as root for sources and class files' and 'Create separate folders for sources and class files' (selected). There is a 'Configure default...' link. The 'Working sets' section has a checkbox 'Add project to working sets' (unchecked) and a 'New...' button. Below it, the 'Working sets:' field is empty with a 'Select...' button. The 'Module' section has a checkbox 'Create module-info.java file' (checked). At the bottom, there are four buttons: '?', '< Back', 'Next >', 'Cancel', and 'Finish'.

New Java Project

Create a Java Project

Create a Java project in the workspace or in an external location.

Project name:

☒ Use default location

Location: [Browse...](#)

JRE

☒ Use an execution environment JRE: [Configure JREs...](#)

☐ Use a project specific JRE:

☐ Use default JRE '17.0.0-oracle' and workspace compiler preferences

Project layout

☐ Use project folder as root for sources and class files

☒ Create separate folders for sources and class files [Configure default...](#)

Working sets

☐ Add project to working sets [New...](#)

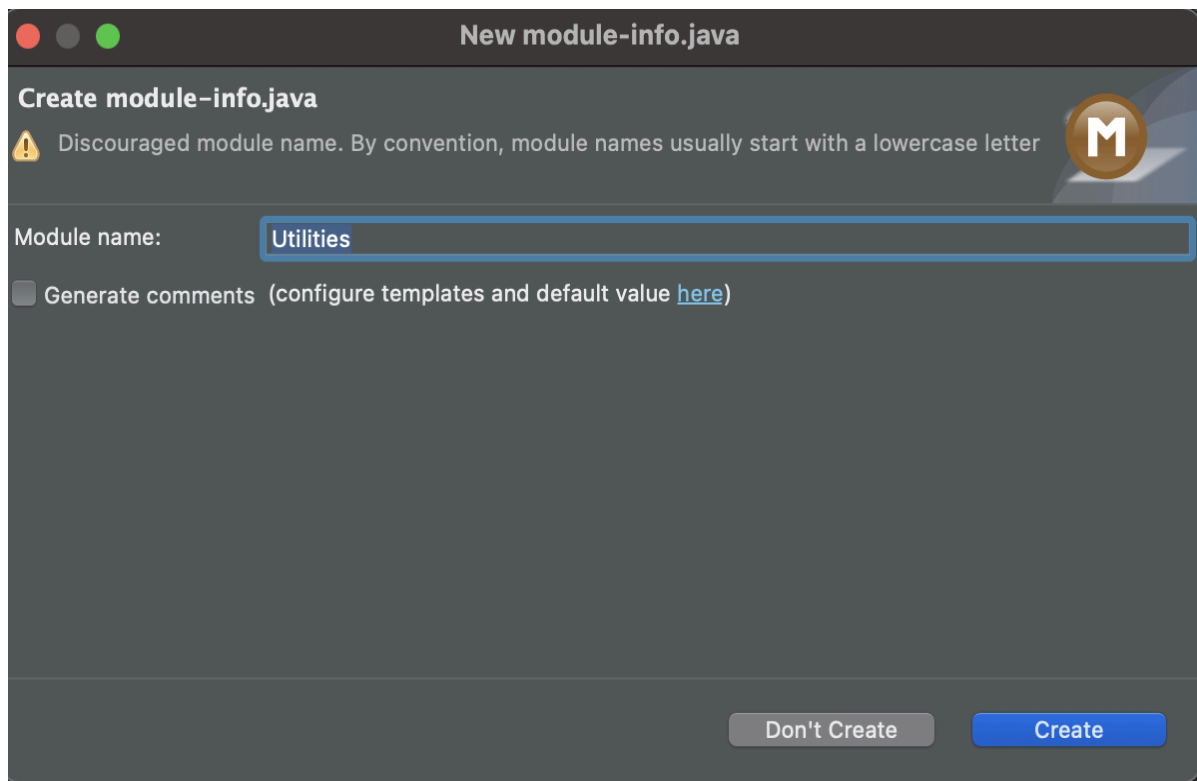
Working sets: [Select...](#)

Module

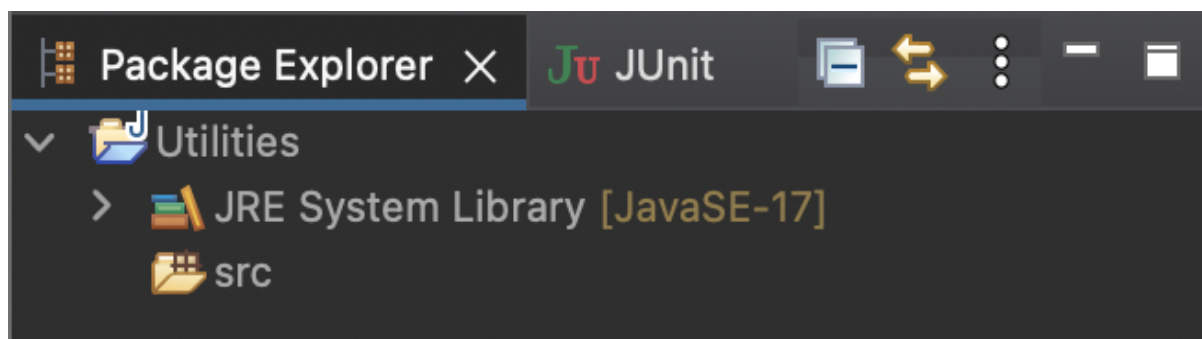
☒ Create module-info.java file

? < Back Next > Cancel Finish

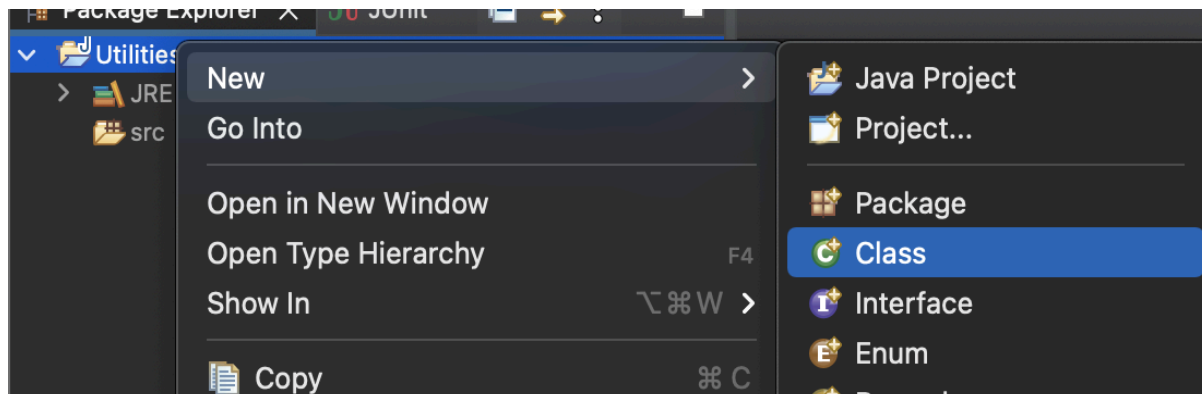
Luego haz clic en Finalizar y se te pedirá que elijas un nombre de un módulo. Los módulos son un tema que está fuera del alcance de este material. Elige: "Don't Create":



Ahora verás tu proyecto en el Explorador de paquetes:



Desde la opción Archivo o haciendo clic derecho sobre el proyecto puedes seleccionar: "New > Class"



En el mensaje de la pantalla, debes elegir el nombre de la clase. Hablaremos de clases en el futuro, pero te recomiendo que la primera clase que crees la llames Main, ya que esto hace obvio cuál es la clase que contiene el punto de entrada de tu programa.

Además, es muy importante proporcionar un valor para el campo "package". La convención es que un paquete tenga al menos 3 identificadores separados por punto. En una empresa los 2 primeros identificadores son el dominio web invertido, p.j. para la empresa Sun sería "com.sun" y para Oracle "com.oracle", etc. Para un proyecto personal puedes usar una convención como: "code.pperez" si tu nombre es Pepito Pérez. El tercero ya sería para representar tu proyecto y los siguientes para organizar tu código.

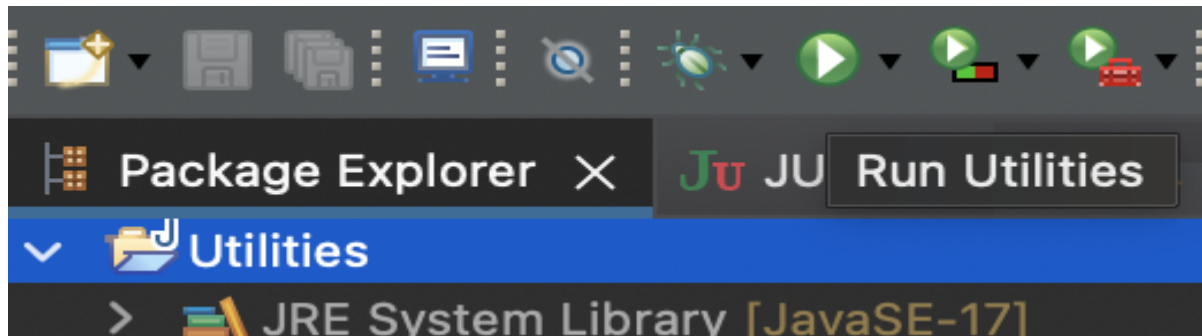
Para la clase Main, y solo para ella, también debes marcar la casilla de verificación "public static void main(String[] args)" en "¿Qué métodos auxiliares te gustaría crear?"

Tu primera clase sería algo como:

```
public class Main {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
    }  
  
}
```

Puede eliminar todo el comentario que inicia con `// TODO`, una vez que estés listo para colocar algo de código.

Si deseas ejecutar tu programa, una opción es usar el botón de reproducción, de fondo verde y un triángulo blanco, después del botón con un insecto:



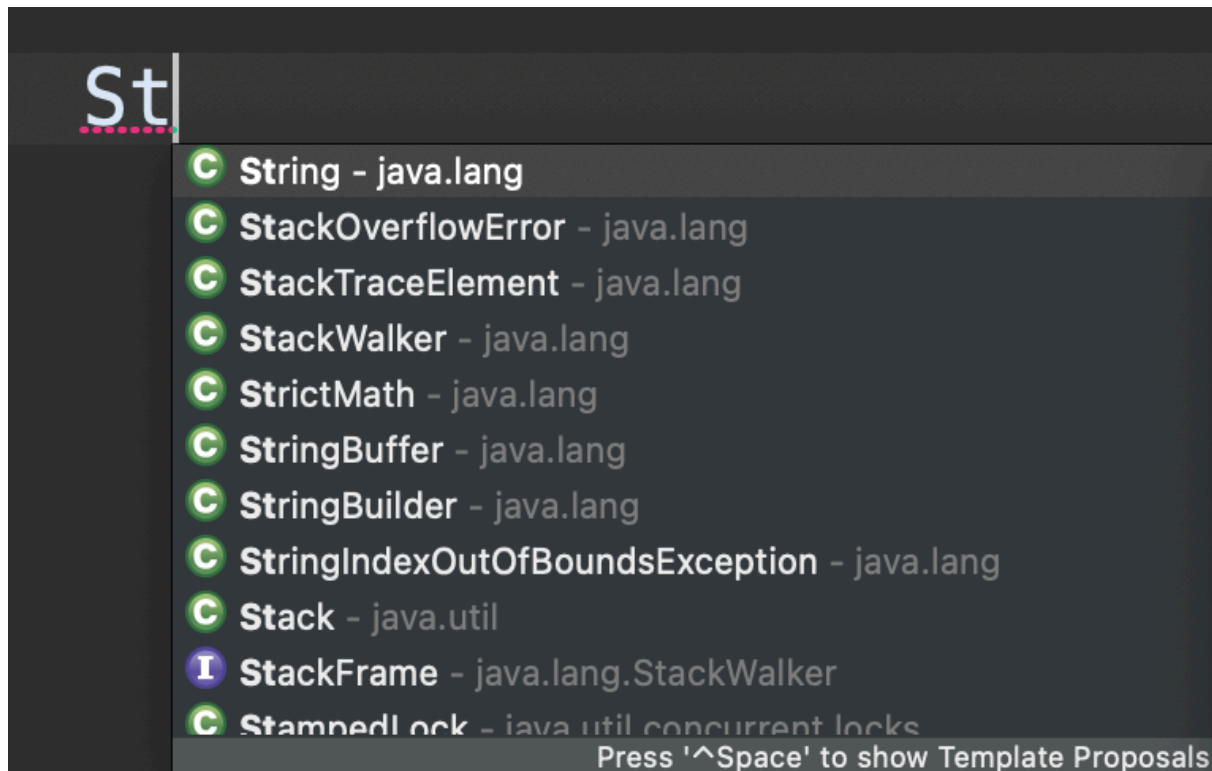
Pongamos algo en el método principal y veamos qué sucede:

```
Main.java x
1 package code.cobregon.utilities;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         System.out.println("Hello World!");
7     }
8
9 }
10
```

```
Console x
<terminated> Main (69) [Java Application] /Users/carlosobregon/.sdkman/candidates/java/17.0.0-oracle/bin/java (Apr 15, 2023, 12:42:32)
Hello World!
```

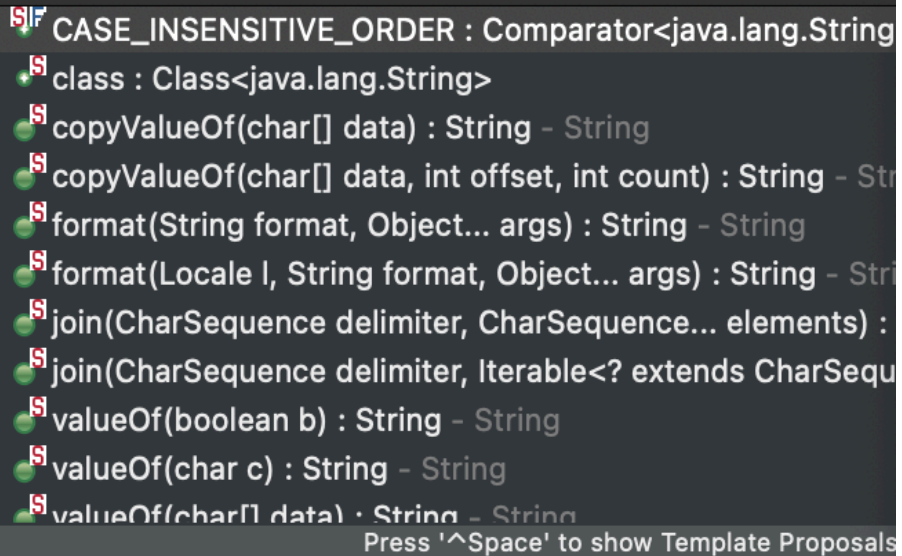
En la parte inferior tienes una simulación del terminal / consola del computador, para que puedas comprobar lo que tu programa está imprimiendo allí.

Una de las características más interesantes de los IDE es la función de autocompletar. En cualquier momento, si escribes solo algunos caracteres de un identificador y presionas CTRL + Barra Espaciadora, o equivalente en IDE distintos de Eclipse, el IDE te dará una lista de posibles identificadores que te puedan interesar:



Además si escribes el nombre de una clase u objeto y presionas "." obtendrás una lista de todos los métodos posibles a los que puedes llamar desde allí:

String.

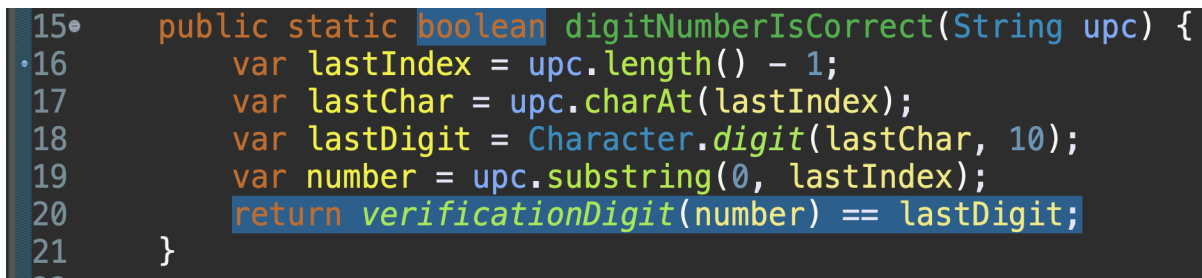


CASE_INSENSITIVE_ORDER : Comparator<java.lang.String>
class : Class<java.lang.String>
copyValueOf(char[] data) : String - String
copyValueOf(char[] data, int offset, int count) : String - String
format(String format, Object... args) : String - String
format(Locale l, String format, Object... args) : String - String
join(CharSequence delimiter, CharSequence... elements) : String
join(CharSequence delimiter, Iterable<? extends CharSequence> elements) : String
valueOf(boolean b) : String - String
valueOf(char c) : String - String
valueOf(char[] data) : String - String
Press '^Space' to show Template Proposals

También es interesante que si mantienes presionada la tecla CTRL en Windows o Command en Mac y haces clic en el nombre de una clase o método, el IDE te llevará hasta donde se definió esa clase / método para que puedas ver su documentación y cómo está implementada.

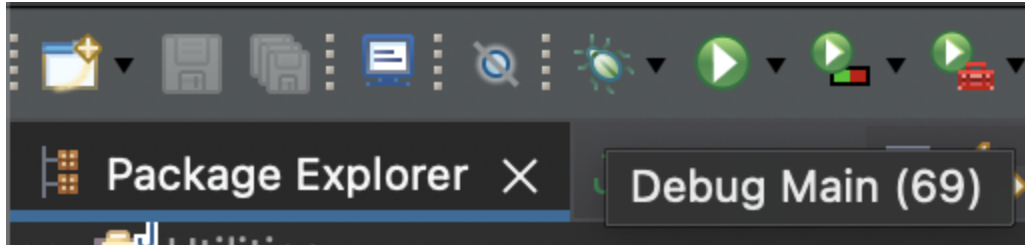
Hay muchas otras funciones interesantes que puedes descubrir por tu cuenta, pero la última que quiero mencionar es el depurador.

En cualquier punto de tu código, puedes establecer un punto de interrupción haciendo doble clic en el número de línea. Verás un punto azul que representa el punto de interrupción. Puedes tener tantos puntos de interrupción como desees.

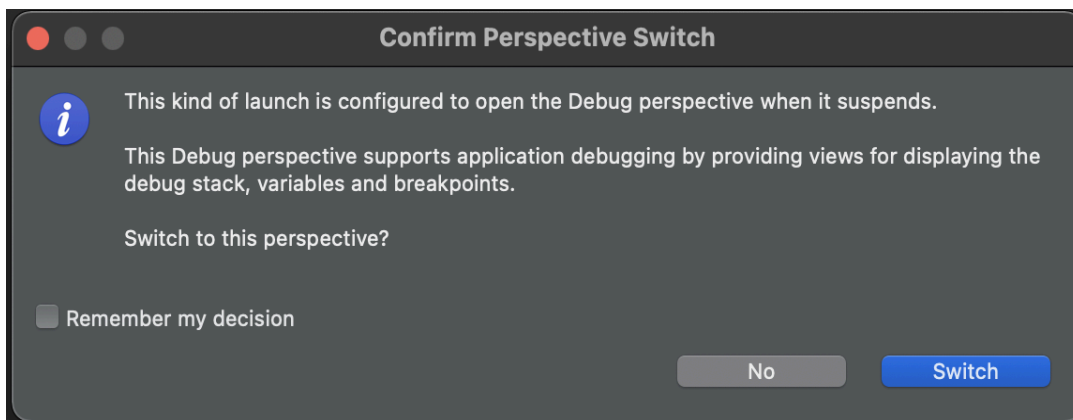


```
15• public static boolean digitNumberIsCorrect(String upc) {  
16     var lastIndex = upc.length() - 1;  
17     var lastChar = upc.charAt(lastIndex);  
18     var lastDigit = Character.digit(lastChar, 10);  
19     var number = upc.substring(0, lastIndex);  
20     return verificationDigit(number) == lastDigit;  
21 }
```

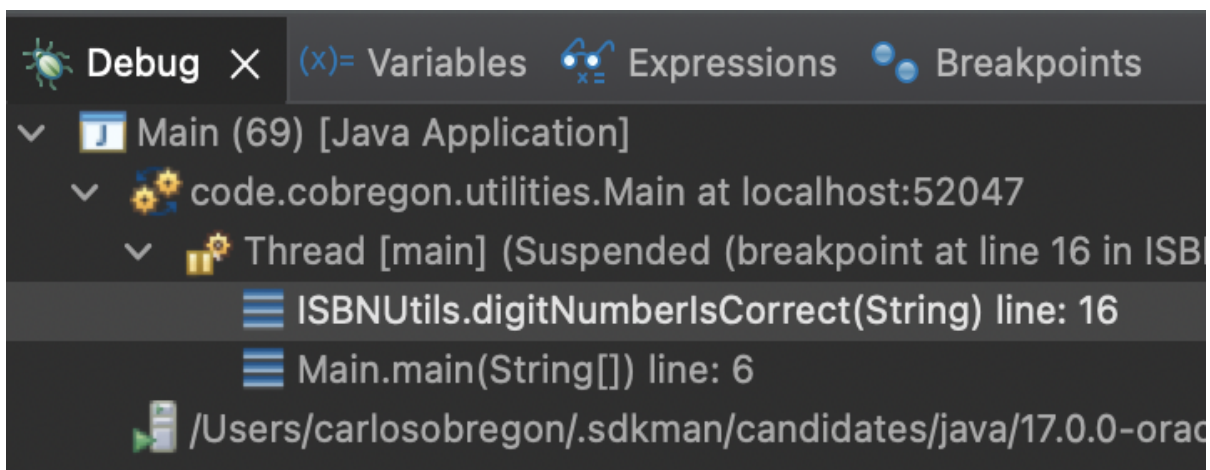
Ahora puedes ejecutar el programa con el depurador haciendo clic en el icono del insecto:



Cuando utilices el depurador, el programa te preguntará si quieres cambiar a una perspectiva diferente. Una perspectiva en Eclipse es una configuración de componentes que pueden brindarte información específica sobre lo que estás haciendo. Cambia siempre de perspectiva al depurar. Presiona "Switch"



Hay un panel que se resaltaré con estas pestañas:



Volveremos a eso después.

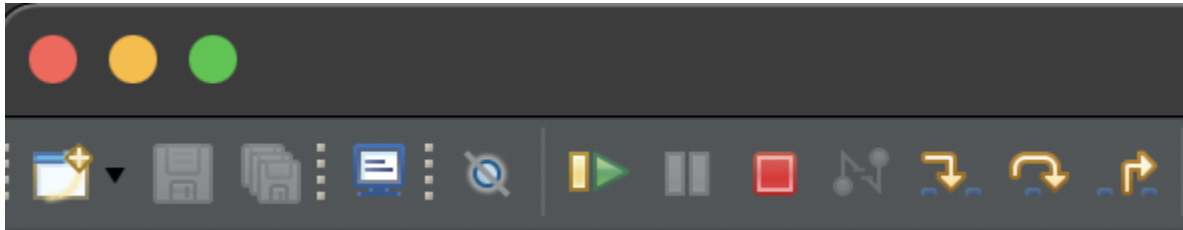
El IDE ejecutará el programa hasta el primer punto de interrupción y se resaltaré la línea del código donde se detuvo la ejecución:


```

15 public static boolean digitNumberIsCorrect(
16     var lastIndex = upc.length() - 1;
17     var lastChar = upc.charAt(lastIndex);
18     var lastDigit = Character.digit(lastChar, 10);
19     var number = upc.substring(0, lastIndex);

```

Ahora puedes controlar el flujo de ejecución con estos botones:



El botón con el triángulo verde es el botón de reanudar. Si lo presionas, el programa saltará al siguiente punto de interrupción. Si estableces un punto de interrupción dentro de un ciclo, pasará a la siguiente iteración de este. Si no hay más puntos de interrupción, el programa se ejecutará hasta que finalice.

El cuadrado rojo es el botón de finalizar, al presionarlo finalizará la ejecución del programa. Si tienes un error al ejecutar tu programa, puedes detenerlo en la línea que causa el error y entender el estado justo antes de que se lance el error.

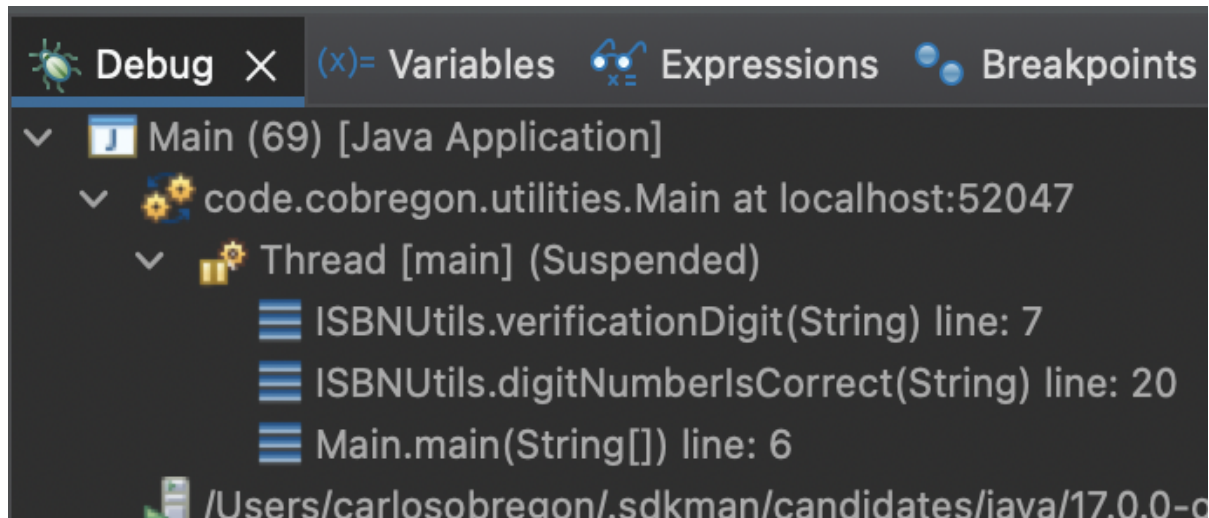
Sin embargo, el flujo principal está controlado por los 3 botones de flecha, de izquierda a derecha: "Step Into", "Step Over" y "Step Return".

"Step Over" ejecuta la siguiente línea de código, si la línea llama a otro método, todo ese método llamado se ejecutará sin interrupción.

"Step Into" se comporta como "Step Over" a menos que se llame a un método, ya que "entrará" en ese método y detendrá la ejecución en él, incluso si el método es parte de una biblioteca o API de Java.

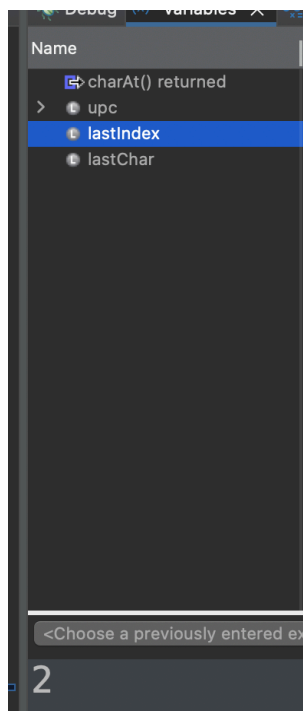
Finalmente, "Step Return" saltará a la sentencia `<return>` del método actual.

Ahora que sabes cómo controlar el flujo de ejecución, veamos qué puedes hacer cuando el flujo está en pausa.



En la pestaña "Debug", puedes ver el stack de ejecución del programa. Aquí puedes ver que el método principal fue el primero en ejecutarse y luego se llama al método `digitNumberIsCorrect()`, que también llama a otro método: `verifcationDigit()`.

La pestaña de variables te mostrará las variables disponibles en el contexto donde se pausa la ejecución: parámetros, variables locales ya definidas y el estado del objeto, si corresponde. Si haces clic en el nombre de uno de ellos, verás su valor:



La pestaña de expresiones te permite agregar una expresión y el IDE te permitirá saber qué valor tendrá en ese punto del programa.

Finalmente, la pestaña de puntos de interrupción le mostrará qué puntos de interrupción se han creado y podrás eliminar alguno.

Usando el depurador puedes entender exactamente cómo se comporta tu programa.

5. Programas en Java

Código 01 - Hola Mundo! Organizado en un proyecto Java

```
01 public class Saludos {
02
03     public static String general() {
04         return para("Mundo");
05     }
06
07     public static String para(String quien) {
08         return String.format("Hola %s!", quien);
09     }
10 }
```

```
01 public class Main {
02
03     public static void main(String[] args) {
04         System.out.println(Saludos.general());
05         System.out.println(Saludos.para("Java"));
06     }
07 }
```

```
Hola Mundo!
Hola Java
```

En Java las construcciones principales son las clases. Todas las funciones que hemos definido deben vivir dentro de una clase. Lo bueno es que el nombre de la clase sirve como nombre de agrupación de sus funciones. Por ejemplo el nombre de la clase Math, sirve como nombre de agrupación para las funciones sqrt(), cosine(), max(), etc.

Esto lo puedes tener en cuenta a la hora de crear tus funciones. En el Código 01, línea 07 la función se llama para() porque para usarla escribirás Saludos.para() que es un nombre muy descriptivo! Las reglas para los identificadores de clase siguen las mismas reglas que para las funciones y variables, pero las clases Java usan CamelCase en mayúsculas en lugar de CamelCase en minúsculas.

También ten en cuenta que las funciones necesitan dos modificadores adicionales que no hemos necesitado usar antes: public static. El modificador <public> se usa para indicar que tu método puede ser usado desde cualquier punto de tu aplicación. El método main siempre debe

ser público. Cuando haces una aplicación muy grande, es útil poder restringir quién puede utilizar ciertas partes de ella. Por ahora usaremos siempre `<public>` en nuestras funciones.

El modificador `<static>` determina, entre otras cosas, como se debe invocar. Un método `<static>` indica que debes anteponer el nombre de la clase al nombre del método separados por punto. Por ejemplo los métodos `sqrt()`, `cosine()`, `max()` son `static` y están declarados en la clase `Math` por lo que debes invocarlos `Math.sqrt()`, `Math.cosine()`, `Math.max()`.

Para poder ejecutar código en Java, necesitarás un método `main`. Como se mencionó, esto debe estar en una clase que puede cualquier nombre, pero es una buena idea darle un nombre que deje claro que es la clase con el método `main`, por lo que `Main` (con `M` mayúscula) parece apropiado.

¿Cuál es el método principal? Es un método con estas propiedades:

- Es público
- Es estático
- No tiene un valor de retorno (de ahí la palabra reservada `void`)
- Se llama `main` (todas las letras en minúsculas)
- Tiene solo un parámetro que es de tipo `String[]`, puede tener cualquier nombre, pero generalmente se llama `args`.

Este es el punto de entrada para un programa Java. Sin uno, su código no será ejecutable por una computadora. Cuando el programa se ejecuta, ese es el primer método que se va a ejecutar.

Como se mencionó, el método `main` puede ser definido dentro de cualquier clase, incluso podríamos ponerlo en la clase `Saludos`, pero sería una mala idea. En general es bueno tener clases que hagan una cosa específica: las hace más fáciles de entender y reutilizar. Si la clase `Saludos` tiene métodos para generar saludos esa ya es una responsabilidad. No tiene sentido agregar además la responsabilidad de tener el punto de entrada del programa. Entonces el método `main()` debería estar en una clase propia.

Para imprimir datos en la salida estándar, que por defecto es la terminal de tu computadora, puede usar el método `println()` del objeto `System.out`, que recibe un `String` como parámetro.

Paquetes

Código 02 - Saludos pero con paquetes

```
01 package code.jugs.saludos;
02
03 public class Saludos {
04
05     public static String general() {
06         return para("Mundo");
07     }
08
09     public static String para(String quien) {
10         return String.format("Hola %s!", quien);
11     }
12 }
```

```
01 package code.jugs.saludos;
02
03 public class Main {
04
05     public static void main(String[] args) {
06         System.out.println(Saludos.general());
07         System.out.println(Saludos.para("Java"));
08     }
09 }
```

Imagínate cuántos proyectos de software tienen una clase Usuario. Podría darse el caso de que estés desarrollando un sistema que utilice una biblioteca externa ¿qué pasa si tu código y el de ellos tienen una clase con el mismo nombre? Cuando te refieres a esa clase, ¿cómo sabría el compilador qué código usar? Bueno, resulta que como humanos tenemos apellidos, mientras que las clases en Java tienen paquetes. Cada clase debería tener un paquete, y cada paquete debería constar de al menos 3 partes separadas por puntos: la costumbre es que las dos primeras sean el dominio inverso de su empresa, por ejemplo. com.java, y el siguiente un identificador del proyecto al que pertenece la clase. Si no es una empresa, puedes utilizar cualquier cosa que consideres sea única.

Solo puede haber un paquete por clase, puede ser tan largo como quieras y siempre tendrá que ser la primera línea de una clase, que no sea un comentario.

Importaciones

Codigo 03 - Import

```
01 package code.jugs.bisiesto.calculos;
02
03 public class Aritmetica {
04
05     public static boolean esMúltiplo(int a, int b) {
06         return (a % b) == 0;
07     }
08
09     public static boolean esPar(int n) {
10         return esMúltiplo(n, 2);
11     }
12
13     public static boolean esImpar(int n) {
14         return !esPar(n);
15     }
16 }
```

```
01 package code.jugs.bisiestos.utils;
02
03 import code.jugs.bisiestos.calculos.Aritmetica;
04
05 public class AñoUtils {
06     public static boolean esBisiesto(int año) {
07         return Aritmetica.esMúltiplo(año, 4)
08             && (Aritmetica.esMúltiplo(año, 400) || !Aritmetica.esMúltiplo(año, 100));
09     }
10 }
```

```
01 package code.jugs.bisiestos;
02
03 import code.jugs.bisiestos.utils.AñoUtils;
04
05 public class Main {
06
07     public static void main(String[] args) {
08         System.out.println(AñoUtils.esBisiesto(2024));
09         System.out.println(AñoUtils.esBisiesto(2000));
10         System.out.println(AñoUtils.esBisiesto(1900));
11     }
12 }
```

```
true  
true  
false
```

Si en su código usa una clase YearUtils, ¿cómo sabe la JVM dónde está esa clase? Bueno, resulta que no es así, por lo que debes proporcionar el nombre completo, que es solo el paquete + nombre de clase para que la JVM lo sepa. Usted proporciona esta información con declaraciones de importación. Las declaraciones de importación deben aparecer después de la declaración del paquete pero antes de la declaración de clase. Puede tener tantas importaciones como necesite, pero si usa otras clases en el mismo paquete, la declaración de importación no es necesaria. También hay algunos paquetes que no requieren declaraciones de importación porque son esenciales, como el paquete java.util de donde provienen clases como Object, String y otras.

Importaciones Static

Código 04 - Mejorando la legibilidad con import static

```
01 package code.jugs.bisiestos.utils;  
02  
03 import static code.jugs.bisiestos.calculos.Aritmetica.esMultiplo;  
04  
05 public class AñoUtils {  
06     public static boolean esBisiesto(int año) {  
07         return esMultiplo(año, 4) && (esMultiplo(año, 400) || !esMultiplo(año, 100));  
08     }  
09 }
```

Por temas de legibilidad, podríamos interesarnos en llamar a las funciones en otra clase, solo por su nombre. Para tal fin podemos usar un import static. Un import static es como un import, pero con el modificador static después de import, y lo que sigue es el paquete de la clase, el nombre de la clase y finalmente el nombre de la función, separados por puntos.

Si quisiera hacer lo mismo con otra función de la misma clase, debo agregar un nuevo import static.

¿Cuándo conviene usar un import static y no un simple import? Es un tema exclusivamente de legibilidad. No hay ninguna funcionalidad impactada por usar uno u otro.

Constantes de Clase

Código 05 - Constantes de Clase

```
01 package code.jugs.basica.fisica;
02
03 public class FísicaPlanetaria {
04
05     private static final double G = 6.67430e-11;
06     private static final double GRAVEDAD_SUPERFICIAL_TIERRA = gravedadSuperficial(5.972e24, 6.378e6);
07     private static final double GRAVEDAD_SUPERFICIAL_MARTE = gravedadSuperficial(6.421e23, 3.3972e6);
08
09     public static double gravedadSuperficial(double masa, double radio) {
10         return G / Math.pow(radio, 2.0) * masa;
11     }
12
13     public static double pesoEnMarte(double peso) {
14         return GRAVEDAD_SUPERFICIAL_MARTE / GRAVEDAD_SUPERFICIAL_TIERRA * peso;
15     }
16
17     public static double energíaGravitatoria(double masa1, double masa2, double distancia) {
18         return -((G * masa1 * masa2) / distancia);
19     }
20 }
```

En las líneas 5 a 7 creamos constantes de clase, indicados por los modificadores `static` y `final`. El modificador `private` simplemente indica que la constante solo va a ser usada dentro de la misma clase `FísicaPlanetaria`, en contraposición al otro modificador de visibilidad que hemos visto: `public`. Más allá de esos modificadores, una constante de clase se declara como una variable cualquiera, utilizando su tipo de dato en vez de `<var>`.

Al igual que las variables, las constantes de clase se crean para darle un nombre a un valor. La diferencia es que siendo constantes de clase estas se calculan una sola vez. Así que sin importar cuantas veces llamemos la función `pesoEnMarte()`, el cálculo de la gravedad superficial de la tierra y de marte se calcula una sola vez. También podría pasar que múltiples funciones usen esta misma constante pero nuevamente al ser una constante de clase solo se calculará una vez.

Esto permite una optimización en desempeño pero también mejora la legibilidad del código al dejar claro que la variable `G` en `gravedadSuperficial()` y en `energíaGravitatoria()` es realmente la misma.

Especificaciones

```
01 package code.jugs.basica.aritmetica;
02
03 public class Artimética {
04     /**
05      * Verifica si un número es múltiplo de otro
06      *
07      * @param a, un número entero
08      * @param b, un número entero diferente de 0
09      * @return true si b es múltiplo de a, false en caso contrario
10      */
11     public static boolean esMúltiplo(int a, int b) {
12         return (a % b) == 0;
13     }
14
15     /**
16      * Verifica si un número es par.
17      * Esta implementación equivale a verificar que el número
18      * sea divisible por 2. Bajo esta implementación, 0 es par.
19      *
20      * @param n, un número entero
21      * @return true if a is a multiple of b, false otherwise
22      */
23     public static boolean esPar(int n) {
24         return esMúltiplo(n, 2);
25     }
26 }
```

A veces tenemos errores en nuestros programas porque hacemos suposiciones sobre las funciones que usamos en nuestro código. Es perfectamente normal que su función tenga condiciones previas: condiciones que los parámetros deben cumplir para que la función funcione correctamente, p. `isLeap()` espera un año positivo. También es una muy buena idea que su función especifique sus condiciones posteriores: qué puede esperar del valor de retorno, p. ¿es positivo? ¿Está siempre entre un rango, etc. Las condiciones previas y posteriores de un método se conocen colectivamente como especificación de una función?

Java proporciona Javadocs para escribir especificaciones. Los Javadocs comienzan con `/**` y terminan con `*/`. Por motivos estéticos, es común agregar un `*` extra en cada línea. Primero, proporcione una breve descripción de lo que hace el método. Si elige un buen nombre, la descripción simple no es tan importante, pero la descripción es un lugar donde puede brindar más detalles sobre el desempeño del método, detalles sobre su implementación u otra información útil. Además de la descripción, debe tener un `@param` y luego el nombre del parámetro y luego explicar sus condiciones previas, si las hubiera. Finalmente tenga un `@return` para especificar las condiciones posteriores.

Existe una herramienta oficial también llamada Javadocs que produce documentación escrita en HTML, de hecho, la documentación oficial que usted ha escrito se produjo con esa herramienta. También se pueden escribir Javadocs para clases, lo cual también es una buena idea

Nota sobre el material

El material seguirá utilizando la convención usada hasta ahora de mostrar los ejemplos de código como funciones sin modificadores `public` y `static` y sin ponerlas como parte de una clase, esto con el fin de ahorrar espacio y hacer el código más fácil de leer.

Si vas a copiar ese código en Eclipse debes recordar crear una clase nueva o reutilizar una clase existente. Si vas a crear una nueva entonces debes escoger también un nombre de paquete.

Los ejemplos de uso también se seguirán mostrando como antes, para probarlos debes ir a la clase `Main` de tu proyecto y usar `System.out.println()` y utilizar el llamado de ejemplo como argumento, y puedes ver en la salida estándar que obtienes el mismo valor indicado.

Ejercicios

E5.1

Rellena el hueco con una de las opciones numeradas, de modo que si ejecutáramos el programa obtuviéramos el resultado indicado

```
package code.jugs.basica;

public class Main {
    public static void     (String[] args) {
        System.out.println("¡Hola Mundo!");
    }
}
```

```
¡Hola Mundo!
```

1. `Main`
2. `main`
3. `init`
4. `saludos`

E5.2

Rellena el hueco con una de las opciones numeradas, de modo que no haya errores de compilación y se produzca el resultado indicado al ejecutar el programa:

```
package code.jugs.basica.aritmetica;

public class Aritmética {

    public static double hipotenusa(double a, double b) {
        return Math.sqrt((a * a) + (b * b));
    }
}
```

```
package code.jugs.basica;

import ____;

public class Main {
    public static void main(String[] args) {
        System.out.println(Aritmética.hipotenusa(3.0, 4.0));
    }
}
```

5.0

1. code.jugs.basica.aritmetica.Aritmética
2. code.jugs.basica.aritmetica
3. Aritmética
4. hipotenusa

E5.3

Rellena el hueco con una de las opciones numeradas, de modo que no haya errores de compilación:

```
package code.jugs.basica.aritmetica;

public class Aritmética {

    public static boolean esMúltiplo(int a, int b) {
        return (a % b) == 0;
    }
}
```

```
package code.jugs.basica.tiempo;

import code.jugs.basica.aritmetica.Aritmética.esMúltiplo;

public class AñoUtils {
    public static boolean esAñoBisiesto(int año) {
        return esMúltiplo(año, 4)
            && (esMúltiplo(año, 400) || !esMúltiplo(año, 100));
    }
}
```

1. public
2. boolean
3. static
4. package

Pruebas Unitarias

¿Cómo podemos garantizar que un método funcione correctamente? Podemos usar el método `main` y algunos `System.out.println()` para hacer algunas pruebas. Y cuando estemos satisfechos, los borramos y esperamos que luego sigan funcionando correctamente.

¿Por qué dejarían de funcionar? Las funciones típicamente sufren cambios con el tiempo. Quizás porque deseamos ampliarlas para hacer más cosas o quizás porque descubrimos que hubo un error que no habíamos encontrado. Pero cuando hacemos un pequeño cambio a veces rompemos cosas que antes funcionaban.

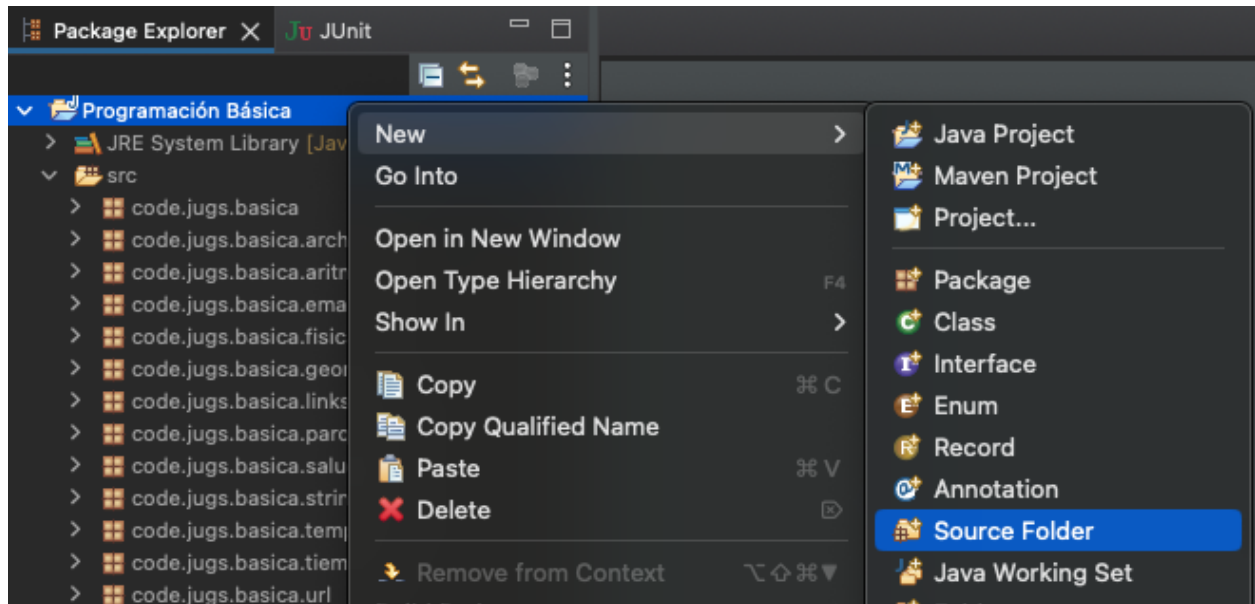
Es importante poder tener una manera de hacer pruebas que no tengamos que borrar y que podamos ejecutarlas fácilmente en el futuro para garantizar que todo siga funcionando. Para eso son las pruebas unitarias.

Una prueba unitaria es básicamente un método que verifica que para un input específico, el output es el que esperaríamos. No hay una manera de escribir pruebas unitarias en el API de Java, debemos depender de un framework. El framework de facto para Java se llama JUnit.

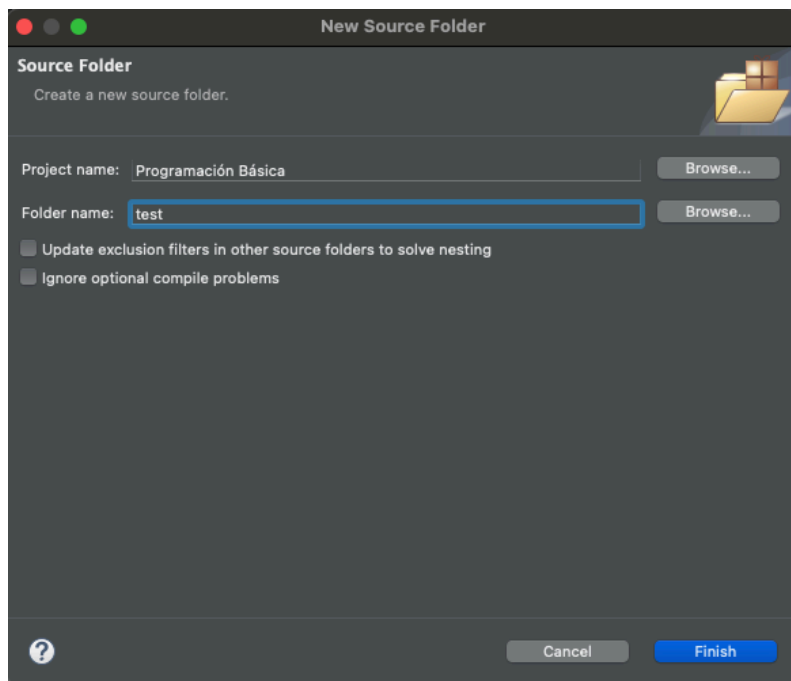
Una de las ventajas de un IDE es que podemos correr pruebas unitarias en el mismo lugar en que escribes tu código.

Las pruebas unitarias son código que no necesitamos cuando está en producción, solo cuando estamos en la etapa de desarrollo. Por lo tanto necesitamos una manera de escribir las pruebas e interactúen con el código que queremos probar pero luego lo podamos ignorar cuando empaquetamos el código para moverlo a producción.

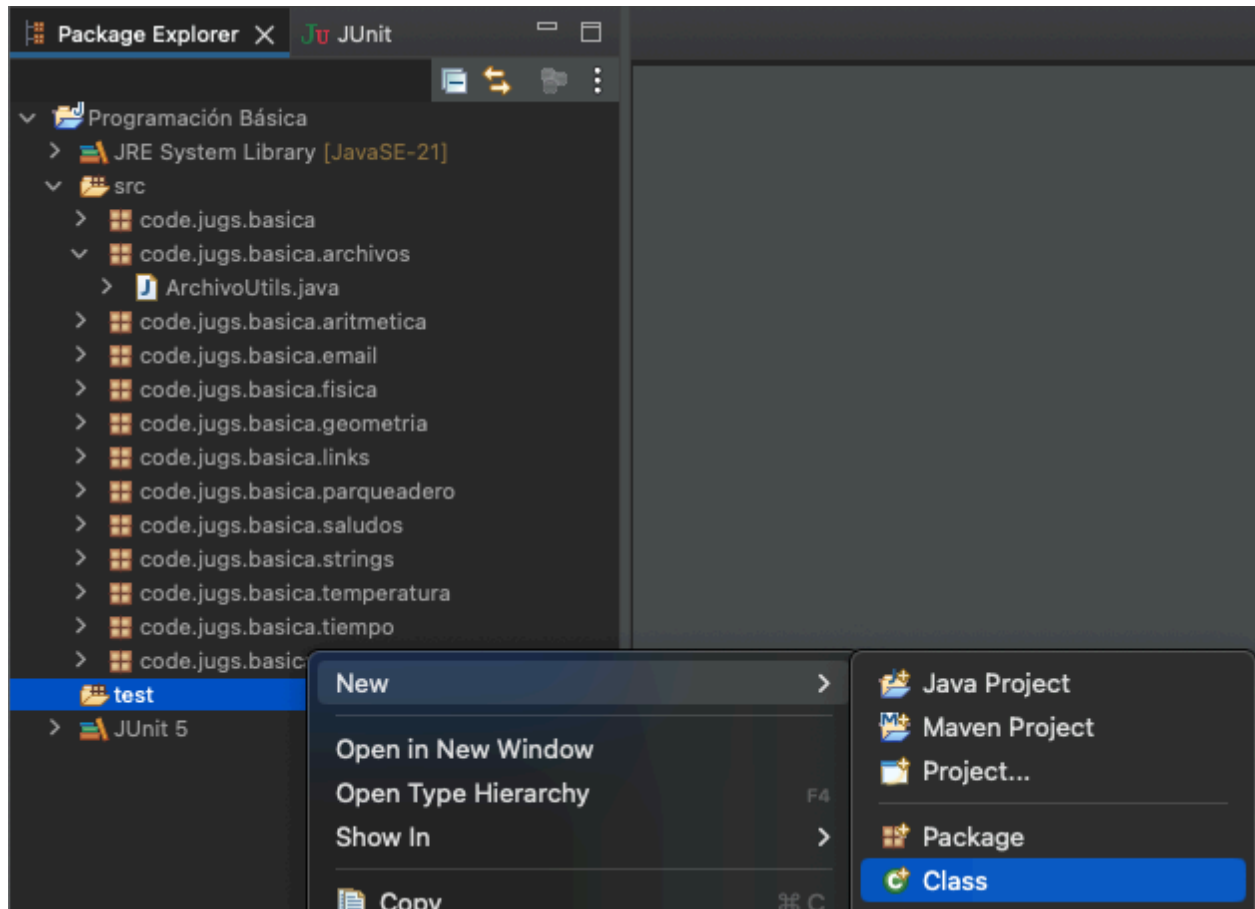
Para eso necesitamos crear un nuevo directorio fuente. En Eclipse lo podemos hacer haciendo click derecho en el directorio del proyecto y luego `New > Source Folder`.



El estándar es que este directorio se llame "test"



Para cada archivo que creamos necesitamos tener otra clase con el mismo nombre pero termine con "Test". De esta manera si tengo una clase ArchivoUtils y quiero ver sus pruebas unitarias, solo tengo que buscar una clase que se llame ArchivoUtilsTest. Algo que también es importante es que esta clase esté en el mismo paquete de la otra clase.



Ahora agrega un método cualquiera, por ejemplo:

```
package code.jugs.basica.archivos;

public class ArchivoUtilTest {

    void foo() {

    }

}
```

Y ahora agrega la anotación "@Test" y haz click en el ícono con la X roja y escoge la opción de "Add JUnit 5 library to the build path".


```
package code.cobregon.basica.geometria;

public class GeometriaTest {
```

@Test

- Add JUnit 4 library to the build path
- Add JUnit 5 library to the build path
- Create annotation 'Test'
- Rename in file (§2 R)
- Fix project setup...

Adds the JUnit 5 library to the build path.

Esto solo lo tienes que hacer la primera vez. Las siguientes veces que puedes autocompletar "@Test" y Eclipse agregará el import necesario.

Ahora ya puedes escribir tu primera prueba unitaria:

```
package code.jugs.basica.archivos;

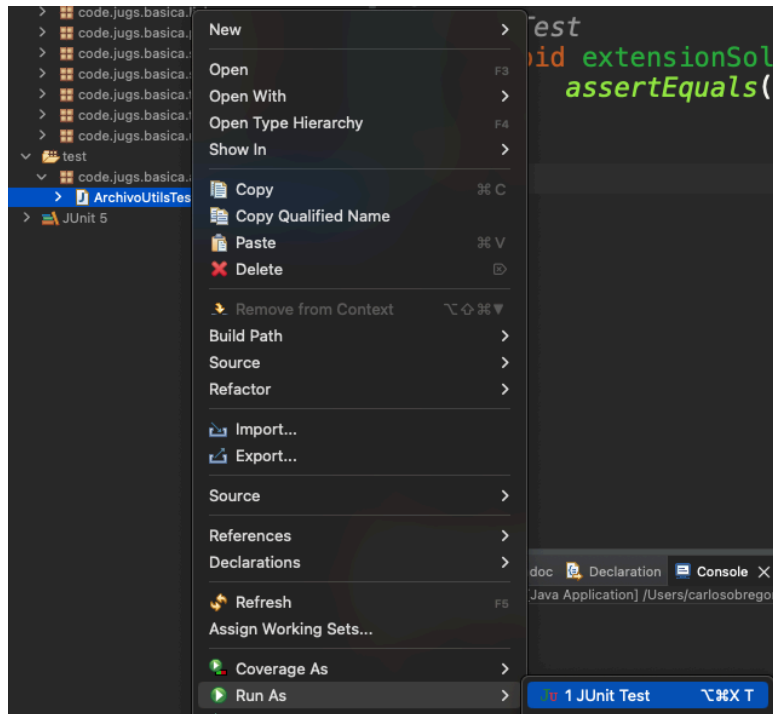
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

public class ArchivoUtilsTest {

    @Test
    void extensionSoloArchivo() {
        assertEquals("sh", ArchivoUtils.extension("debug.sh"));
    }
}
```

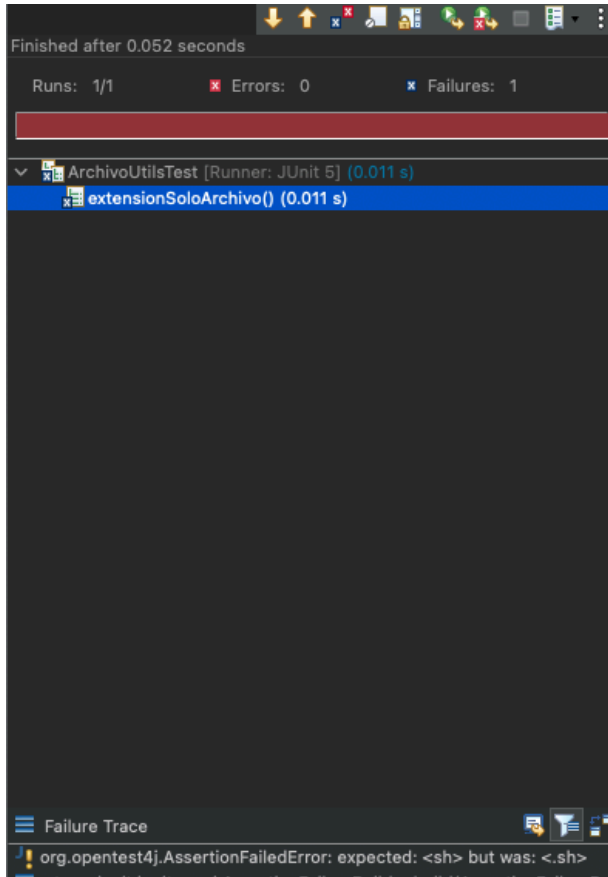
El nombre del método lo podemos cambiar de 'foo' a algo que mejor describa lo que vamos a probar. Y el corazón de la prueba unitaria es el método assertEquals() que como puedes ver, por el import, es parte de JUnit. Este método toma 2 parámetros, el valor de retorno que esperas y luego el llamado al método que quieres probar.

Esta función verifica que ambos valores sean iguales y de modo contrario lanza un error. Para ejecutar una prueba unitaria, haz click derecho sobre el archivo en el explorador del proyecto y escoge Run As > JUnit Test



La barra en color verde significa que no hubo ningún error, es decir que JUnit ejecutó todos los métodos marcados como "@Test" e hizo todos los llamados a nuestras funciones con los argumentos dados y constató que el valor era el que nosotros esperábamos.

Si el método no está haciendo lo que nosotros esperamos veremos un error como este:



En "Failure Trace" podemos ver el mensaje de error: expected <sh> but was: <.sh> así que estamos devolviendo un "." adicional que no deberíamos estar devolviendo. Esto nos ayuda a volver a la implementación de nuestro código y hacer los ajustes necesarios.

Una vez que nuestras pruebas unitarias estén en verde, no tenemos que eliminarlas. Las podemos dejar de lado, con la tranquilidad de poder volver a ejecutarlas en cualquier momento.

Un último comentario, aunque uno podría tener más de un asserts en un método, es mejor no hacerlo inicialmente. Cada assert debe tener su propio método.

Por último además de assertEquals, existe assertTrue y assertFalse para revisar que el valor de retorno de un método es true y false respectivamente.

Nota sobre el material

Los ejercicios de ahora en adelante usar pruebas unitarias para mostrarte cuál es el comportamiento esperado de los métodos. Sin embargo el material mostrará solamente un assertEquals como este:

```
assertEquals(15, sumRangeNumbers(1, 5));
```

Es responsabilidad tuya escribir la clase Test apropiada, y de crear un método "@Test" para contener el assert dado.

6. Ciclos

Código 1 - Ciclo while

```
01  int sumaDígitos(int n) {  
02      var suma = 0;  
03      while (n > 0) {  
04          var últimoDígito = n % 10;  
05          suma = suma + últimoDígito;  
06          n = n / 10;  
07      }  
08  
09      return suma;  
10  }
```

> sumaDígitos(123)	6
--------------------	---

¡El Código 1 muestra el uso de un ciclo! Antes de los ciclos, el flujo de ejecución de un método era: se ejecuta la primera instrucción, luego la segunda, luego la tercera, y así sucesivamente. Los ciclos cambian eso y te permiten repetir un conjunto de declaraciones un número controlado de veces. Hay varias formas de ciclos, sumaDígitos() usa un ciclo while. En un ciclo while, se especifica la condición que debe ser verdadera para que se ejecuten las declaraciones que estén contenidas en su bloque.

La línea 03 es el comienzo del ciclo, indicado por la palabra clave while y entre paréntesis la **condición** del ciclo: que el parámetro <n> debe ser mayor que 0. Las líneas 04 a 06 son las declaraciones que forman parte del ciclo, dado que están dentro del bloque del ciclo, y se ejecutarán hasta que la condición sea falsa. La parte más importante de ese ciclo es la línea 06 en la que hay un **ajuste** a la variable utilizada en la condición, sin él el ciclo se ejecutará para siempre.

La línea 04 utiliza la operación <n % 10> como una forma de obtener el último dígito de un valor entero. La línea 06 utiliza la operación <n / 10> como una forma de obtener un número resultante de “eliminar” el último dígito de un número entero. En el pasado hicimos ejercicios para demostrar este comportamiento. El truco no funcionaría si <n> fuera un doble o si escribieras <n / 10.0>

Para entender lo que sucede en el código, tengamos una tabla con el valor durante cada iteración para sumaDígitos(123):

# Iteración	n	n % 10	n / 10	suma
0	123	n/a	n/a	0
1	123	3	12	3
2	12	2	1	5
3	1	1	0	6
4	0	n/a	n/a	6



Cuando utilices un ciclo while, mantén la inicialización de la variable muy cerca de la condición del ciclo, con suerte que sea la expresión justo antes, o que sea una variable que sea un parámetro. Mantén el ajuste a la variable cerca de la última parte del bloque del ciclo, de esa manera siempre será fácil encontrarlo.

Operadores compuestos

Código 2 - sumaDígitos() usando operadores compuestos

```

01  int sumaDígitos(int n) {
02      var suma = 0;
03      while (n > 0) {
04          var últimoDígito = n % 10;
05          suma += últimoDígito; // sum = sum + lastDigit
06          n /= 10; // n = n / 10
07      }
08
09      return suma;
10  }
```

El código 2 utiliza operadores compuestos como atajo para asignaciones cuando se actualiza el valor de una variable. Este tipo de operación es muy común en los ciclos, por lo que tiene sentido presentarlas ahora. Puedes utilizar un operador compuesto para las siguientes operaciones que hemos visto: +, -, /, * y %.

Entre estos, hay dos que son muy comunes: ajuste sumando 1 o restando 1. Pero vienen en dos formas:

Operadores Compuestos	Post	Pre
n += 1	n++	++n
n -= 1	n--	--n

La diferencia entre un post-incremento o post-decremento y un preincremento o pre-decremento solo es importante cuando los usas en la misma línea con otros operadores, incluidas las asignaciones. Este uso fue muy popular cuando la economía de espacio en la pantalla hizo atractivo meter tanto código en la menor cantidad de caracteres posible. Ahora que esto ya no es tan importante como antes, no tiene sentido hacer que el código sea más difícil de leer sólo para ahorrar algo de espacio en la pantalla.



Usar operadores post-incremento/decremento o pre-incremento/decremento de esta manera:

```
var soyCríptico = ++n;
var muyCríptico = n--;
```

¡Hace que el código sea mucho más difícil de entender sin ninguna ventaja!

Ciclos For

Código 3 - sumaRangoNúmeros() con ciclo for

```
01 int sumaRangoNúmeros(int límite) {
02     var suma = 0;
03     for (var actual = 1; actual <= límite; ++actual) {
04         suma += actual;
05     }
06
07     return suma;
08 }
```

> sumaRangoNúmeros(5) // 1 + 2 + 3 + 4 + 5	15
> sumaRangoNúmeros(3)	6

La construcción más popular es el ciclo for. En el ciclo for tienes las 3 partes de un ciclo: inicialización, condición y ajuste, en una misma línea de código. En el Código 3, sumaRangoNúmeros suma todos los números enteros entre 1 y el límite dado. La línea 02 crea una variable <suma> para acumular la suma de los números. La línea 03 luego tiene la

construcción for: inicializa una variable <actual> para realizar un seguimiento del número actual que se está considerando, luego la condición es que <actual> sea menor o igual a <límite> y finalmente el ajuste es incrementar la corriente en 1.

No te dejes engañar por la inicialización/condición/ajuste que está en la misma línea: la inicialización sólo ocurre una vez antes de que se ejecute el bloque del ciclo. La condición se verifica una vez por iteración, antes de ejecutar el bloque del ciclo. El ajuste siempre al terminar una iteración, antes de verificar la condición.

Un ciclo for no se tiene ninguna parte obligatoria, nada impide que escribamos sumaDígitos() así:

Código 4 - ciclos for sin inicialización

```
01  int sumOfDigits(int n) {  
02      var sum = 0;  
03      for (; n > 0; n /= 10) {  
04          var lastDigit = n % 10;  
05          sum += lastDigit;  
06      }  
07  
08      return sum;  
09  }
```

Pero, ¡seguramente es una mala idea! El Código 4 parece ser una degradación del Código 2 en términos de legibilidad.



Utiliza espacios en blanco apropiados para hacer que los loops sean más legibles. Poner un espacio después de las palabras <for> y <while> es una pequeña pista de que es un ciclo y no una función. En un ciclo for, coloque un espacio después del punto y coma y recuerda colocar un espacio antes y después de un operador.

Strings

Código 5 - los ciclos son los mejores amigos de los Strings.

```
01 String espaciador(String palabra) {
02     var builder = new StringBuilder();
03     for (var i = 0; i < palabra.length(); ++i) {
04         var letra = palabra.charAt(i);
05         builder.append(letra);
06         builder.append(" ");
07     }
08
09     var espaciado = builder.toString();
10     return espaciado;
11 }
```

> espaciador("Java")	"J a v a "
> espaciador("Programming")	"P r o g r a m m i n g "

Recuerda que un String es una secuencia de caracteres. Cada carácter está asociado con un índice. Los ciclos for son una excelente manera de recorrer una cadena, carácter por carácter. En la línea 04 usamos el método `charAt()` para acceder al carácter actual que se está atravesando. Y después de agregar el caracter a un `StringBuilder`, agregamos un espacio para generar el "espaciado"



Es muy común en todos los lenguajes de programación que la variable utilizada para recorrer los índices de un String se llame `<i>`. Hemos abogado por variables con nombres buenos y significativos, pero en este caso, `<i>` se entiende tan universalmente que es un buen nombre para dicha variable.



¡Usar `<i>` sólo es una buena idea para variables que representan índices! ¡No adquieras el hábito de usar ese nombre para cada variable en un loop incluso cuando no estés recorriendo un rango de índices! ¡Usar `<i>` para todos tus ciclos perjudicaría la legibilidad de tu código!

Ciclos For vs Ciclos While

¿Cuándo usar ciclos for y cuándo usar ciclos while? Lo primero que es importante decir es que no hay ninguna diferencia en funcionalidad. Todo código que utilice un ciclo for se puede

reescribir como un código que use un ciclo while y viceversa, aunque puede requerir algunos cambios menores.

Veamos la función sumaRangoNúmeros reescrita con un ciclo while:

Código 6 - sumaRangoNúmeros con un ciclo while

```
01  int sumaRangoNúmeros(int límite) {  
02      var suma = 0;  
03      var actual = 0;  
04      while (actual <= límite) {  
05          suma += actual;  
06          ++actual;  
07      }  
08      return suma;  
09  }
```

En este caso, la legibilidad se ve afectada negativamente porque las declaraciones que definen el ciclo están dispersas; el control del ciclo está en las líneas 03, 04 y 06 mientras que antes eran todos parte de la definición del ciclo for.

Código 7 - esCapicúa(int)

```
01  boolean esCapicúa(int número) {  
02      var n = Integer.toString(número);  
03  
04      var i = 0;  
05      var j = n.length() - 1;  
06  
07      while (i < j && n.charAt(i) == n.charAt(j)) {  
08          ++i;  
09          --j;  
10      }  
11  
12      return i >= j;  
13  }
```

Un número capicúa es un número que se lee igual de izquierda a derecha que de derecha a izquierda, es el equivalente a un palíndromo para las palabras salvo que es un problema más sencillo porque no tenemos que lidiar con caracteres diferentes de dígitos.

Código 07 tiene una implementación que verifica que un número sea capicúa. La idea general es crear un String con el número, ya que es más fácil de analizar el número como un String que como un int. Inicializamos un índice en el primer carácter y otro en el último carácter. Ahora

hacemos un ciclo que se ejecute mientras los índices no se hayan cruzado y mientras que los caracteres sean los mismos. En caso de que sea cierto, ajustamos los índices, moviendo el índice de la izquierda hacia la derecha y el de la derecha hacia la izquierda.

Cuando el ciclo se termina podemos tener la garantía de que el ciclo terminó porque los índices se cruzaron, en cuyo caso todos los caracteres analizados fueron iguales y el número es capicúa, o terminó porque encontró 2 caracteres que debían ser iguales pero no lo son. Así que al final simplemente verificamos la razón por la cuál terminó.

En este caso el ciclo while ofrece mucha mejor legibilidad dado que las variables <i> e <j> deben ser usadas por fuera del ciclo. Sin embargo se podría reimplementar con un ciclo for solo que necesitaría elementos adicionales que no hemos visto.



Una vez que aprendamos condicionales, encontrarás que los ciclos se pueden escribir de diferentes maneras sin sacrificar funcionalidad. La decisión es siempre cuál ofrece mejor legibilidad.

Ciclos Anidados

Código 8 - Un cuadrado con *

```
01 String cuadrado(int tamaño) {  
02     var builder = new StringBuilder();  
03     for (var hilera = 1; hilera <= tamaño; ++hilera) {  
04         for (var columna = 1; columna <= tamaño; ++columna) {  
05             builder.append("*");  
06         }  
07         builder.append(System.lineSeparator());  
08     }  
09     return builder.toString();  
10 }
```

> cuadrado(2)	** **
> cuadrado(3)	*** *** ***

No hay restricciones sobre lo que puedes hacer dentro de un ciclo. También puedes usar un ciclo. Si lo piensas bien, un ciclo dentro de un ciclo es solo un ciclo interno que se repite varias veces y está controlado por un ciclo externo. El código 8 construye una figura cuadrada usando `"*"`. El loop exterior controla el número de filas del cuadrado, mientras que el ciclo interior controla el aspecto de una hilera. Después de cada hilera insertamos un carácter de salto de línea, una de las maneras más sencillas de hacer este carácter es con `System.lineSeparator()`.

Si bien anidar ciclos es válido, en la gran mayoría de casos es seguramente mejor hacer una función que encapsule el ciclo interno. Eso potencialmente mejora la legibilidad:

Código 9 - `cuadrado()` con 2 funciones

```

01 String cuadrado(int tamaño) {
02     var builder = new StringBuilder();
03     for (var hilera = 1; hilera <= size; ++hilera) {
04         builder.append(hilera(tamaño));
05         builder.append(System.lineSeparator());
06     }
07     return builder.toString();
08 }
09
10 String hilera(int tamaño) {
11     var rowBuilder = new StringBuilder();
12     for (var columna = 1; columna <= tamaño; ++columna) {
13         rowBuilder.append("*");
14     }
15     return rowBuilder.toString();
16 }

```



Java ya tiene una función como `row()`: se llama `repeat(int)` y en el código anterior se podría usar así:

```
builder.append("*".repeat(tamaño));
```

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#repeat\(int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html#repeat(int))

Si tienes ciclos anidados, considera mover parte del código interno a una función. Esto mejora la legibilidad de tu código y si la función no se comporta como esperas, puedes probar el ciclo

interno por separado del ciclo externo y encontrar el problema más rápidamente. No todos los ciclos anidados pueden beneficiarse al extraer el ciclo interno de una función, ¡pero vale la pena intentarlo!

Strings de múltiples líneas

Un tema que no habíamos visto antes porque no teníamos un buen propósito eran los literales para Strings de múltiples líneas. Existe en Java una manera de escribir un String de múltiples líneas sin tener que hacer ninguna operación especial. Miremos un ejemplo:

```
var cuadrado = """
    ***
    ***
    ***
    """;
```

La variable `cuadrado` es un String que tiene 3 saltos de línea pero pudimos representarlo simplemente con un literal. La manera sencilla de escribir Strings de múltiple líneas es:

- Empieza con 3 comillas dobles en una sola línea
- En las siguientes líneas escribe el contenido del String pero alineado verticalmente con el delimitador anterior.
- Cierra con las mismas 3 comillas dobles, también alineadas.

¿Por qué es ahora importante? Si quieres hacer una prueba unitaria para una función `cuadrado()` que vimos anteriormente, la manera más sencilla es con un String de múltiples líneas:

```
assertEquals("""
    ***
    ***
    ***
    """, cuadrado(3));
```

Ciclos (Accidentalmente) Infinitos

Código 10

```
01 int cuantasIteraciones() {  
02     var cuenta = 0;  
03     for (var n = 1; n != 100; n += 2) {  
04         ++cuenta;  
05     }  
06     return cuenta;  
07 }
```

Dado ese código, ¿qué valor crees que se devolvería? Bueno, ¡el hecho es que el código nunca se detendrá! ¿Puedes encontrar la razón?

¿Qué sucede si escribes un ciclo infinito? Si tienes suerte, habrá un error dentro del loop que terminará el programa. De lo contrario, tu programa simplemente ejecutará el código dentro del loop hasta que se detenga por la fuerza la ejecución de su programa.

Vigila tus ciclos para evitar ciclos infinitos.



Recuerda que una función invocada es realmente un valor, así que podrías usarlos dentro de tus ciclos de maneras como estas:

```
while(!usuarioLogeado()) {  
    // código  
}  
  
for (var n = 2; n < maxPrimo; n = siguienteMúltiplo(n, m)) {  
    // código  
}
```

Logs

Código 11 - Logs con System.out

```
01 int sumaDígitos(int n) {  
02     var suma = 0;  
03     while (n > 0) {  
04         var últimoDígito = n % 10;  
05         suma += últimoDígito;  
06         System.out.println(suma);  
07     }
```

07	n = n / 10;
08	}
09	return suma;
10	}

> sumaDigitos(123)	3 5 6 ==> 6
--------------------	----------------------

Cuando estás haciendo software, a veces es importante dejar un registro de lo que estás haciendo en tus funciones para que si luego hay un error ya tengas algunos datos recopilados sobre lo sucedido. Hay herramientas para hacer esto a nivel empresarial que están fuera del alcance de este tutorial, pero hay una herramienta que puede ayudarte a entender tus programas sin tener que usar un debugger: imprimir en la salida estándar de la computadora, dentro de tus funciones.

La línea 06 del Código 11 utiliza `System.out.println()` dentro de un ciclo para verificar cómo cambia la suma a través de las iteraciones. Si bien ese código es sencillo, y quizás no se beneficie mucho del log, en general imprimir a la pantalla dentro de un ciclo puede ayudar a entender lo que hace el código:



Consulta también estos métodos que puedes utilizar para imprimir en la salida estándar:

- [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#print\(java.lang.String\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#print(java.lang.String))
- [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#printff\(java.lang.String,java.lang.Object...\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/PrintStream.html#printff(java.lang.String,java.lang.Object...))

No hay límite en cuántas veces puedes imprimir a la salida estándar, ni dónde puedes invocar una función para hacerlo, así que úsala tanto como quieras. Sólo recuerda que existen mejores herramientas para hacer estos logs, pero para ejercicios simples, `System.out` está bien.

Ejercicios

E5.1

Escribe una función que reciba un `String` y lo reverse.

```
assertEquals("avaJ", reversar("Java"));
assertEquals("nuf si avaJ", reversar("Java is fun"));
```



Existe una manera de hacer este ejercicio con un método de StringBuilder:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuilder.html#reverse\(\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/StringBuilder.html#reverse())

Trata de implementar este ejercicio **sin** usarla.

E5.2

¿Recuerdas los métodos `indexOf(char)` e `indexOf(char, int)` de la clase `String`? Utilízalos para crear una función que cuente cuántas veces esta una subcadena en un `String` dado. Los caracteres de una subcadena no pueden ser usados en otra, mira el último ejemplo: la respuesta es 1 porque después de encontrar el primer "ss", el algoritmo debe buscar la siguiente subcadena después del final de esa subcadena.

```
assertEquals(2, conteo("abracadabra", "abr"));
assertEquals(4, conteo("110101110101100", "01"));
assertEquals(3, conteo("xoxox", "x"));
assertEquals(1, conteo("sssa", "ss"));
```

E5.3

Escribe una función que tome un `String` que represente una oración y cuente cuántas palabras contiene. Puedes asumir que la oración solo constará de caracteres alfanuméricos y 1 espacio separará las palabras que contiene.

```
assertEquals(1, cuentaPalabras("one"));
assertEquals(3, cuentaPalabras("one two three"));
```

E5.4

Escribe una versión sobrecargada de `sumaRangoNúmeros` que tome dos parámetros y retorne la sumatoria de los números entre ese rango incluyendo los extremos. Luego vuelve a implementar la función `sumaRangoNúmeros(int)` para usar esta nueva función.

```
assertEquals(75, sumaRangoNúmeros(10, 15)); // 10 + 11 + 12 + 13 + 14 + 15
assertEquals(15, sumRangeNumbers(1, 5));
```


E5.5

La "dígitos suma" de un número es el número resultante después de aplicar esta operación: suma los dígitos de un número. Si el resultado tiene más de un dígito, es mayor o igual a 10, repite la operación. Y este procedimiento se repite hasta tener un número de un solo dígito. Escribe una función para calcular la dígitos suma de un número entero. Recuerda re utilizar la función sumaDígitos() que vimos en este capítulo.

```
assertEquals(1, dígitosSuma(1234)); // 1 + 2 + 3 + 4 = 10 -> 1 + 0 = 1
assertEquals(6, dígitosSuma(555)); // 5 + 5 + 5 = 15 -> 1 + 5 = 6
```

E5.6

Escribe una función serieArmónica() que dado un número n, calcule la suma de los términos de la serie. $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$, por ejemplo si la función es llamada con n igual a 6, calcule la suma $1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6}$.

```
assertEquals(1.0, serieArmónica(1));
assertEquals(1.5, serieArmónica(2)); // 1 + 1/2
assertEquals(2.0833, serieArmónica(4), 1e-4);
```

E5.7

Escribe una función que reciba un String que representa un Id alfanumérico y retorne el número de ceros con los que empieza el Id.

```
assertEquals(0, cerosIniciales("ABC"));
assertEquals(2, cerosIniciales("00ABC"));
assertEquals(1, cerosIniciales("0ABC00"));
```



E5.8

En numerología el número de la vida se calcula como el número resultante de hallar la dígitos suma (mira el ejercicio E5.5) de tu fecha de nacimiento, interpretada sólo como números. En Java existe una clase para representar fechas de este modo: LocalDate. Escribe una función que reciba una fecha y retorne su número de vida.

```
assertEquals(5, númeroVida(LocalDate.of(2000, 1, 20)));
assertEquals(6, númeroVida(LocalDate.of(1945, 10, 31)));
```



La documentación de LocalDate está aquí:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/time/LocalDate.html>

Investiga qué métodos necesitas para que a partir de la fecha puedas extraer los números que necesitas.



E5.9

Escribe una función que cree un String cuadrado con un tamaño dado, utilizando los caracteres "*" y "." altercados. Las hileras impares empiezan con "*" y las pares con ".".

```
assertEquals("""
    * . *
      * .
    * . *
    """, patronAjedrez(3));
assertEquals("""
    * . * . *
      * . * . *
    * . * . *
      * . * . *
    * . * . *
      * . * . *
    """, patronAjedrez(6));
```



E5.10

Escribe una función que cree un String con el siguiente formato: Para un máximo dado, cree todas las parejas posibles de números positivos en que ninguno de ellos sea mayor a ese máximo. Cada pareja las debe registrar en el formato "(#, #)", cada pareja debe estar separada por un "|" y el String debe empezar y terminar con uno. Mira los ejemplos para más claridad:

```
assertEquals("(1, 1)|(1, 2)|(2, 1)|(2, 2)", coordenadas(2));
assertEquals("(1, 1)|(1, 2)|(1, 3)|(2, 1)|(2, 2)|(2, 3)|(3, 1)|(3, 2)|(3, 3)", coordenadas(3));
```

7. Condicionales

Código 1 - usando if y switch para condicionales

```
01 boolean esVocal(char c) {
02     var mayúscula = Character.toUpperCase(c);
03     return switch(mayúscula) {
04         case 'A', 'E', 'I', 'O', 'U' -> true;
05         default -> false;
06     };
07 }
08
09 int contarVocales(String texto) {
10     var vocales = 0;
11     for (var i = 0; i < texto.length(); ++i) {
12         var actual = texto.charAt(i);
13         if (esVocal(actual)) {
14             ++vocales;
15         }
16     }
17
18     return vocales;
19 }
```

<pre>> esVocal('a') > esVocal('A') > esVocal('z') > contarVocales(";Hola Mundo!")</pre>	<pre>true true false 4</pre>
---	------------------------------



En la mayoría de los lenguajes de programación clásicos, la construcción switch tiene una sintaxis diferente. Java usó esa versión exclusivamente hasta la versión 14. La sintaxis aquí es mucho más legible y menos propensa a errores, por lo que esta es la forma sugerida de usar.

El código 1 muestra las dos construcciones condicionales que tienen la mayoría de los lenguajes de programación: declaraciones switch y declaraciones if. Recuerda que antes hemos visto el operador ternario, que también es una forma de tener condicionales en tu programa.

Con un switch, podemos actuar según el valor de una variable. La sentencia switch provee una lista de valores y el valor que se debe retornar si la variable tiene ese valor. Para indicar un valor dentro de un switch se utiliza una sentencia case, como la de la línea 04. En este caso la

sentencia case tiene una lista separada por comas de varios valores y luego una "flecha" para finalmente especificar el valor asociado. Ese valor puede ser un literal o puede ser un llamado a una función. La línea 04 indica que si la variable dentro del switch tiene el valor 'A', 'E', 'I', 'O', o 'U' entonces retornamos el valor true.

Un switch puede tener tantas sentencias case como sean necesarias, pero también tiene que proveer una sentencia default, como en la línea 05, que indique el valor que se debe retornar si ninguna sentencia case se cumple, esto es importante para que el switch sea "completo", es decir que para todo valor existe un valor para asociar. Para la sentencia default solo necesitamos agregar la flecha y el valor correspondiente.

La línea 03 comienza la declaración de switch con la palabra clave switch, la variable que desea usar luego va entre paréntesis. Como las funciones y los ciclos, las condicionales también especifican un bloque como puede ver en las líneas 03 y 06. Un switch es una manera de transformar un valor en otro, el valor de la expresión switch es como el valor retornado por una función que puede ser atrapado en una variable o retornado. En la 03 vemos como inmediatamente retornamos el valor dado que la sentencia return está antes de la sentencia switch.

Esta es la manera recomendada de usar un switch, envolver un switch en su propia función. Esto cumple con la idea de que una función debe hacer una sola cosa, y le da un nombre al switch lo que mejora la legibilidad del código.

La línea 04 especifica los valores para el primer caso a considerar. Anteriormente pusimos el carácter en mayúsculas para simplificar el código, aunque hubiera sido igualmente válido tener código así:

```
boolean esVocal(char c) {  
    return switch(mayúscula) {  
        case 'A', 'a', 'E', 'e', 'I', 'i', 'O', 'o', 'U', 'u' -> true;  
        default -> false;  
    };  
}
```

Una vez listados los valores a considerar, utilizamos los caracteres "->", llamados coloquialmente "flecha", para indicar el valor que se debe asociar con los valores.



Puede tener cualquier tipo de código después de "->" en el caso de un switch. Sin embargo, tener mucho código o una expresión compleja perjudica la legibilidad. Si debes asociar código complejo después de la flecha es mejor encapsular este código en una nueva función.



Una expresión de switch debe ser exhaustiva: necesita un caso para cada valor posible o un caso predeterminado. Si no es exhaustivo, obtendrá un error de compilación.

La línea 06 termina en punto y coma después del final del bloque del switch. Esta forma nueva de switch en que el valor es atrapado en una variable o usado como parte de una expresión return, necesita terminar en punto y coma como cualquier otra expresión.



Es una buena idea que una función solo haga una cosa. Si su función hace varias cosas, su código se vuelve más difícil de razonar y de probar. Si lo piensa, una expresión de switch hace una cosa: asignar el valor de una variable a otro valor de un tipo posiblemente diferente, esto recibe el nombre en inglés de *map function*. Entonces, en general, es una buena idea que cada expresión switch esté envuelta en una función.

contarVocales() tiene una declaración if en la línea 13. Las declaraciones if toman cualquier predicado cuando este es verdadero, ejecutan el código dentro de su bloque. Antes, teníamos un acumulador y un ciclo para recorrer los caracteres de la variable <texto> de tipo String. Recuerde que charAt() devuelve el caracter en un índice dado y el ciclo, como está escrito, recorre todos los índices válidos de <texto>. Si el carácter actual es una vocal, y solo si eso es cierto, entonces incrementamos el valor de la variable acumuladora <vocales>. Si el predicado es falso, no se ejecuta nada.

Los condicionales son simples, pero hay algunas sutilezas que discutiremos a lo largo de este capítulo

Las muchas formas de switch

El switch más básico es este:

```
switch(expression) {  
  case valor1 -> valor1;  
  case valor2 -> valor2;  
  default -> valorPorDefecto;  
}
```

La variable puede ser cualquier primitiva, String o Enum, que veremos más adelante. El switch necesita un bloque para que quede claro qué código forma parte de él. En una cláusula case, se asocia un posible valor con otro valor. Usamos la palabra clave default para especificar el valor a usar en caso de que ninguna sentencia case sea cierta. Esta sentencia es necesaria y aunque no tiene que ser la última dentro del switch, esto mejora su legibilidad.

Se pueden agrupar valores en una sentencia case separándolos por comas:

```
switch(expression) {  
    case valor1, valor2 -> valorComún;  
    case valor3 -> valorPara3;  
    default -> valorPorDefecto;  
}
```

Las muchas formas de if

La forma más simple de una declaración if es:

```
if (predicado) {  
    código;  
}
```

Esto significa que <código> solo se ejecutará si el predicado es verdadero.

Puedes agregar una declaración else como última parte del bloque if:

```
if (predicado) {  
    código-en-verdadero;  
} else {  
    código-en-falso;  
}
```

Ahora <código-en-verdadero> solo se ejecutará si el predicado es verdadero; de lo contrario, se ejecutará <código-en-falso>. Las declaraciones <else> son siempre opcionales. Nota además que nunca se ejecutará <código-en-verdadero> y <código-en-falso> al tiempo. Se ejecuta uno u el otro.

También son opcionales varias declaraciones else-if que deben venir después del bloque if pero antes del bloque else si está presente. Estas condiciones else-if se probarán sólo si todas las condiciones anteriores son falsas:

```
if (predicado1) {  
    condicion-1-code;  
} else if (predicado2) {  
    condicion-2-code;  
}
```

En este caso, el código de <predicado1> se ejecutará cuando <predicado1> sea verdadero y el código de <predicado2> se ejecuta si <predicado1> es falso y <predicado2> es verdadero. No se ejecutará nada si la <predicado1> y <predicado2> son falsos.

Código 2 usando if/else

```

01 String darFormato(String teléfono, String formato) {
02     var builder = new StringBuilder();
03     var índiceTeléfono = 0;
04     for (var i = 0; i < formato.length(); ++i) {
05         var actual = formato.charAt(i);
06         if (actual != 'X') {
07             builder.append(actual);
08         } else {
09             var dígitoTeléfono = teléfono.charAt(índiceTeléfono);
10             builder.append(dígitoTeléfono);
11             ++índiceTeléfono;
12         }
13     }
14     return builder.toString();
15 }
16
17 String teléfonoConFormato(String teléfono) {
18     return switch(teléfono.length()) {
19         case 10 -> formato(teléfono, "(XXX) XXX-XXXX");
20         case 7 -> formato(teléfono, "XXX-XXXX");
21         default -> teléfono;
22     };
23 }

```

> teléfonoConFormato("1234567890")	(123) 456-7890
> teléfonoConFormato("9001000")	900-1000

El código 2 muestra un par de funciones para dar formato para números de teléfono. El corazón de esto es darFormato() que toma un String que representa un número de teléfono que contiene solo dígitos y un formato que tiene 'X' como marcador de posición para los dígitos. La idea es copiar cada carácter del formato a un StringBuilder hasta llegar a un marcador de posición, en cuyo caso agregamos el siguiente dígito disponible del teléfono.

Para mezclar los caracteres necesitamos 2 índices: uno para los caracteres del formato y otro para los del teléfono. El ciclo recorre los caracteres del formato. Otro índice, teléfonoÍndice, se crea fuera del ciclo y se actualiza solo cuando se agrega el dígito de un teléfono al String que se está creando.

La otra función elige el formato que se utilizará en darFormato() según la longitud del teléfono: hay 2 formatos previstos: uno para números de teléfono de 10 dígitos y otro para teléfonos de 7 dígitos. Si el teléfono no tiene ninguno de los dos, entonces el número de teléfono no está formateado.

Flujos de ejecución con condicionales: una advertencia

¿Cuál es el flujo de ejecución del siguiente fragmento de código?

```
// A
if (1) {
    // B
}
// C
```

Sólo hay dos:

Si 1 es verdadero: A-B-C

Si 1 es falso: A-C

¿Qué pasa con este?

```
// A
if (1) {
    // B
} else if (2) {
    // C
} else {
    // D
}
// E
```

Ahora quedan 3:

Si 1 es verdadero, independientemente de 2: A-B-E

Si 1 es falso y 2 es verdadero: A-C-E

Si 1 y 2 son falsos: A-D-E

¿Qué pasa con este?

```
// A
if (1) {
    // B
} else {
    // C
    if (2) {
        // D
    } else if (3) {
        // E
    }
}
if (4) {
    // F
}
// G
```

¡Es demasiado complicado! Realmente necesitas evitar códigos como el último. Si mantienes tus funciones pequeñas y granulares, es difícil terminar con un código mazacotudo, pero si alguna vez te encuentras con él, ¡detente! Respira hondo e intenta desenredar tu lógica en funciones y luego reescribe la función principal llamando esas funciones. Esto suele ser suficiente para terminar con un código más simple.

Orden de condiciones

Código 3 fizzBuzz, cuidado con el orden de las condiciones

```
01 String fizzbuzz(int n) {
02     if (n % 3 == 0) {
03         return "Fizz";
04     } else if (n % 5 == 0) {
05         return "Buzz";
06     } else if (n % 3 == 0 && n % 5 == 0) { // n % 15 == 0
07         return "FizzBuzz";
08     } else {
09         return Integer.toString(n);
10     }
11 }
```

En fizzbuzz() tomas un int y devuelves un String basado en las siguientes reglas:

- Si el número es múltiplo de 3, devuelve "Fizz"
- Si el número es múltiplo de 5, devuelve "Buzz"
- Si el número es múltiplo de 3 y 5 (que es lo mismo que decir es múltiplo de 15), devuelve "FizzBuzz"

- En todos los demás casos, simplemente devuelva el mismo número pero como un String.

Entonces el Código 3 parece correcto, ¿verdad?

Bueno, hay un problema sutil ya que las condiciones if-else sólo se verifican si todas las condiciones anteriores son falsas, la condición `n % 3 == 0 && n % 5 == 0` nunca se verificará, porque si esa condición es verdadera, `n % 3 == 0` también es verdadero, la cual se verificó unas cuantas líneas anteriores. Por tanto, el orden de las condiciones podría importar. El código correcto para fizzbuzz es:

Código 4 fizzBuzz, cuidado con el orden de las condiciones

```

01 String fizzbuzz(int n) {
02     if (n % 3 == 0 && n % 5 == 0) { // n % 15 == 0
03         return "FizzBuzz";
04     } else if (n % 3 == 0) {
05         return "Fizz";
06     } else if (n % 5 == 0) {
07         return "Buzz";
08     } else {
09         return Integer.toString(n);
10     }
11 }

```

Switch vs If

Todo lo que se puede escribir con una declaración de cambio se puede escribir con un if, pero lo contrario no es cierto, ya que un switch solo puede verificar valores pero no otras expresiones booleanas, como el resultado del operador "<".

<pre> String teléfonoConFormato(String teléfono) { return switch(teléfono.length()) { case 10 -> darFormato(teléfono, "(XXX) XXX-XXXX"); case 7 -> darFormato(phoneNumber, "XXX-XXXX"); default -> phoneNumber; }; } </pre>	<pre> String teléfonoConFormato(String teléfono) { if (teléfono.length() == 10) { return darFormato(teléfono, "(XXX) XXX-XXXX"); } else if (phoneNumber.length() == 7) { return darFormato(teléfono, "XXX-XXXX"); } else { return teléfono; } } </pre>
--	--

Veamos nuestra función `darFormato()` implementada en ambos estilos: Puede ser subjetivo, pero las declaraciones de switch parecen más claras en su intención. Si puedes escribir un condicional con un switch probablemente deberías hacerlo, pero recuerda que hay casos en que solo vamos a poder usar un if.

Regreso anticipado

Un gran uso de una declaración `if` es tener un retorno anticipado en una función. Anteriormente hablamos de una función `<ellipsify>` que agregaba puntos suspensivos a un `String` si el `String` tenía más caracteres que un límite determinado. Por ejemplo, si tenemos un título como "Programar con Java es divertido" pero nuestro diseño web admite menos caracteres, entonces transformaremos el título a "Programar con Java...". Por esa función, tomamos un `String` y una longitud máxima y devolvemos el `String` transformado. Sin embargo, hay un caso muy simple para esa función: si el `String` ya tiene menos caracteres que la longitud máxima, entonces podemos devolver el `String` sin modificarlo antes de tiempo.

```
String ellipsify(String texto, case longitudMáxima) {  
    if (texto.length() <= longitudMáxima) {  
        return texto;  
    }  
  
    // código para manejar el caso más complejo  
}
```



En el código anterior, no es necesario escribir una declaración `else` ya que el código anterior se comporta exactamente como si hubiera una declaración `else`. Piénsalo. Al realizar devoluciones anticipadas, evitar el `else` y tener una línea en blanco adicional hace que el código sea más legible.

Al ser claro y frontal sobre cuál es el valor devuelto en una condición determinada, tu código es mucho más legible. Es posible que puedas tener varios valores anticipados.

Misión secundaria: múltiples retornos en una función

Existe una especie de debate religioso sobre si las funciones deberían tener retornos múltiples o no. Existe un punto de vista purista de que las funciones de programación deberían ser como funciones matemáticas y, por eso, solo deberían tener un retorno. Creo que una mejor respuesta sería: depende.



Kent Beck escribió: "cualquier respuesta decente a una pregunta interesante comienza: 'Depende...'. ¡Esto es especialmente cierto en la programación!

El argumento habitual es que tener una sola devolución hace que sea "más fácil" leer un código porque solo necesitas verificar un lugar para comprender lo que se devuelve. Pero a veces, para evitar devoluciones múltiples, el código necesita reasignar variables, lo que creo que va en contra de "solo hay una verificación para comprender lo que se devuelve", ya que necesitará verificar todos los lugares donde las variables tienen una asignación para comprender todos los valores posibles a ser retornados y en qué condiciones se retornan.

Reescribamos `fizzbuzz()` de una manera que evite múltiples devoluciones y veámoslo lado a lado con nuestro código original:

```
String fizzbuzz(int n) {  
    var s = "";  
    if (n % 3 == 0 && n % 5 == 0) {  
        s = "FizzBuzz";  
    } else if (n % 3 == 0) {  
        s = "Fizz";  
    } else if (n % 5 == 0) {  
        s = "Buzz";  
    } else {  
        s = Integer.toString(n);  
    }  
    return s;  
}
```

```
String fizzbuzz(int n) {  
    if (n % 3 == 0 && n % 5 == 0) {  
        return "FizzBuzz";  
    } else if (n % 3 == 0) {  
        return "Fizz";  
    } else if (n % 5 == 0) {  
        return "Buzz";  
    } else {  
        return Integer.toString(n);  
    }  
}
```

Creo que incluso con múltiples devoluciones, el código de la derecha es mucho más legible que el de la izquierda. Si escriben su código de manera limpia, tener múltiples devoluciones no perjudica la legibilidad. No evites múltiples `return` para seguir las tendencias, comprueba qué es lo mejor para el código.

Hay otra versión de `fizzbuzz()` que creo que es mucho menos legible:

```

01 String fizzbuzz(int n) {
02     var s = "";
03     if (n % 3 != 0 || n % 5 != 0) {
04         s = Integer.toString(n);
05     } else {
06         if (n % 3 == 0) {
07             s += "Fizz";
08         }
09         if (n % 5 == 0) {
10             s += "Buzz";
11         }
    }
}

```



Si escribes un fragmento de código y piensas "¡Hmm, esto es ingenioso!"
¡Probablemente estás haciendo que el código sea muy difícil de leer para otros!

Resumen

Para influir en el flujo de ejecución de una función tienes, fundamentalmente, dos construcciones:

- Ciclos
- Condicionales

Con los condicionales, puede escribir código que solo se ejecute bajo una determinada condición.

Las declaraciones de switch asignan un valor posible con algún código, pero solo se ejecutará el código de una cláusula, la cláusula con el valor de la variable que se utiliza o el código predeterminado si no coincide ninguna cláusula.

Si las declaraciones son más complejas, garantizan que el código de su bloque solo se ejecutará si su condición es verdadera, que puede ser cualquier expresión booleana. Se debe tener mucho cuidado para que el código dentro de una declaración if/else sea fácil de leer.

Las declaraciones If son excelentes para proporcionar retornos tempranos en funciones, lo que mejora la legibilidad y hace que sea muy sencillo comprender cómo se comporta su código en condiciones especiales.



Cuando verificas una condición con un valor booleano, no necesitas compararla con verdadero o falso, ya que ya es un valor booleano. En lugar de:

```
if (condicion == true) {  
}
```

escribe:

```
if (condicion) {  
}
```



Si necesitas devolver un valor booleano comprobando una condición que ya es un predicado, realmente no necesitas el if/else ya que puede devolver el valor directamente.

```
if (condicion) {  
    return true;  
} else {  
    return false;  
}
```

escribe:

```
return condicion;
```

En vez de escribir:

```
if (condicion) {  
    return false;  
} else {  
    return true;  
}
```

escribe:

```
return !condicion;
```



Cuando tengas un if/else, escribe primero la condición con el bloque más corto. Si comienzas a leer un if/else y el bloque es muy largo y finalmente encuentras un else, es posible que necesites desplazarte hacia arriba nuevamente para encontrar las condiciones para comprender cuándo se ejecutará el bloque else. Suele ser más fácil si los cambias.

En vez de:

```
if (condicion) {  
    // 1  
    // 2  
    // 3  
    // 4  
    // 5  
    // 6  
} else {  
    // 7  
}
```

escribe:

```
if (!condicion) {  
    // 7  
} else {  
    // 1  
    // 2  
    // 3  
    // 4  
    // 5  
    // 6  
}
```

Ejercicios

E6.1

En Matemáticas todo número entero tiene al menos 2 divisores: 1 y el propio número. Estos se llaman divisores obvios. Los números primos son números mayores que 1 que no tienen ningún otro divisor salvo los divisores obvios. Recuerda que si $\langle a \rangle$ es múltiplo de $\langle b \rangle$, entonces $\langle b \rangle$ es divisor de $\langle a \rangle$. Escribe una función `esPrimo` para comprobar si un número es primo.

```
assertTrue(esPrimo(1));  
assertTrue(esPrimo(7));  
assertFalse(esPrimo(100));
```

E6.2

Los dígitos de verificación son un mecanismo simple para detectar errores cuando se digita un número. Por ejemplo supongamos que generamos un identificador "123", luego aplicamos una fórmula cuyo resultado es un dígito adicional, supongamos "6", entonces el identificador completo sería "1236", a veces escrito como "123-6". Ahora la siguiente vez que digitemos ese identificador en un sistema, este podría validarlo, separando "123" de su dígito de verificación "6", calcular el número de verificación de "123" y revisar que coincide con el "6" obtenido. Esto significa que si una persona se equivoca al digitar el número, por ejemplo intercambiando 2 dígitos de lugar, el número de verificación no coincidirá y así se sabrá que hubo un error al

digitarlo. Para los números UPC (Universal Product Code) el algoritmo para hallar el dígito de verificación es:

- Separa el último dígito que es el número de verificación.
- Suma los dígitos en posiciones impares (el primer dígito, el tercer dígito, etc) y el resultado multiplícalo por 3.
- Suma los dígitos en posiciones pares (el segundo dígito, el cuarto dígito, etc)
- Suma los resultados de los dos pasos anteriores
- Encuentra el módulo 10 de la suma, llamémoslo M
- Si M es 0, ese es el número de verificación. Si no es 0, entonces el dígito de verificación es 10 - M

Crea una función que reciba un número dado como una cadena y verifique si es un número UPC válido verificando el dígito de verificación como se explica en las reglas anteriores.

```
assertTrue(upcVálido("116"));
assertFalse(upcVálido("111"));
assertTrue(upcVálido("019"));
assertTrue(upcVálido("107"));
assertTrue(upcVálido("240"));
assertTrue(upcVálido("036000241457"));
```



¿Cuándo un número es un número en lugar de un String? Ten en cuenta que las variables numéricas como los ints se pueden usar en operaciones aritméticas como suma, resta, etc. Los números como String requieren menos memoria para representarse y se pueden recorrer dígito a dígito fácilmente. ¿Crees que podría haber casos en los que cualquiera de las dos formas esté bien?



Para convertir un char a un int puedes usar:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#digit\(char,int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#digit(char,int)) usando 10 como segundo argumento.

E6.3

Escribe una función `contraseñaFuerte` que tome un String y devuelva verdadero solo si una contraseña cumple con los siguientes criterios:

- Tener al menos una letra mayúscula
- Tener al menos un número
- Tener al menos un carácter no alfanumérico
- Tener al menos 8 caracteres.


```
assertTrue(contraseñaFuerte("Abc123$_"));
assertFalse(contraseñaFuerte("Abc12345"));
assertFalse(contraseñaFuerte("abc123$_"));
assertFalse(contraseñaFuerte("Abcdef$_"));
assertTrue(contraseñaFuerte("ABC123$_"));
```



La clase Character puede tener métodos que encuentres útil para este ejercicio. Por ejemplo:

[https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isLetterOrDigit\(char\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isLetterOrDigit(char))

E6.4

CamelCase es una forma de escribir identificadores con más de una palabra “separándolos” usando letras mayúsculas. El caso serpiente, por otro lado, usa letras minúsculas pero guiones bajos para separar palabras. Escriba una función camelCaseToSnakeCase que tome una String que sea un identificador válido escrito en CamelCase y la devuelva en Snake Case. Además, escribe una función snakeCaseToCamelCase para la operación inversa.

```
assertEquals("holaMundo", snakeCaseToCamelCase("hola_mundo"));
assertEquals("unoDosTres", snakeCaseToCamelCase("unoDosTres"));
assertEquals("identificador", snakeCaseToCamelCase("identificador"));
assertEquals("último_dígito", camelCaseToSnakeCase("últimoDígito"));
assertEquals("es_vocal", camelCaseToSnakeCase("esVocal"));
```



Para verificar si un carácter está en mayúscula, puedes

usar: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isUpperCase\(char\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#isUpperCase(char)) y para convertir un carácter a

mayúscula puedes

usar: [https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toUpperCase\(int\)](https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Character.html#toUpperCase(int))

Arrays y Arrays Multidimensional

Código 1 - esTriángulo usando arreglos

```
01 boolean esTriángulo(double a, double b, double c) {  
02     var lados = new double[]{a, b, c};  
03     Arrays.sort(lados);  
04     return (lados[0] + lados[1]) > lados[2];  
05 }
```

En el Código 1 implementamos esTriángulo() pero usando un arreglo. En otro capítulo hablamos de cómo tres longitudes sólo pueden formar un triángulo si la mayor de ellas es menor que la suma de las otras dos. Al implementar esa función el problema era cómo encontrar el máximo de tres variables y la dificultad de hacer escalar ese proceso: ¿qué pasa si en lugar de 3 variables, hay 20 o 100? La respuesta son los arreglos.

Un arreglo es una variable que guarda una secuencia de valores homogéneos. Al igual que los Strings, los arreglos también usan índices para hacer referencia a estos valores, el primer índice es 0 y el último índice es la longitud del arreglo menos 1. Para acceder a un elemento de una matriz, se utiliza el operador de llaves cuadradas con el índice al que se desea acceder.

La línea 02 crea un arreglo de variables de tipo double, utilizando los parámetros como valores. En cierto sentido, tomamos los 3 parámetros y los empaquetamos juntos en un arreglo. Al estar en un arreglo es más fácil realizar análisis sobre esa información que con las variables sueltas.

Para inicializar un arreglo hay dos maneras: inicializar conociendo los valores, como hicimos en el Código 01 o crear el arreglo primero y luego dar los valores como en el siguiente código:

Código 2 creación de arreglo con inicialización después

```
01 boolean esTriángulo(double a, double b, double c) {  
02     var lados = new double[3];  
03     lados[0] = a;  
04     lados[1] = b;  
05     lados[2] = c;  
06     Arrays.sort(lados);  
07     return lados[0] + lados[1] > lados[2];  
08 }
```

En el Código 01 el compilador sabe que el arreglo tiene 3 valores, dado que la lista de valores separados por comas tiene 3 valores. En cambio, en el Código 02 el tamaño se hace explícito en la línea 02.

Java viene con varios métodos útiles para usar con arreglos, uno de ellos es `Arrays.sort()` que ordena los elementos dentro del arreglo por su orden natural, para variables de tipo `double` es de menor a mayor. Al correr la función sabemos que la variable menor va a estar en el índice 0 y la mayor en el último índice.



Los arrays en Java no tienen métodos propios como los Strings. Pero hay una clase de utilidad llamada `Arrays` con algunos métodos útiles como `Arrays.sort()`. Consulta la documentación:

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html>

Una vez ordenados, podemos sumar los elementos en los índices 0 y 1, los lados menores, y revisar que sean exclusivamente mayores que el elemento en el índice 2, que es el lado mayor.

Arrays como Parámetros

Código 3 promedio de un arreglo

```
01 double promedio(double[] datos) {
02     var suma = 0.0;
03     for (var i = 0; i < data.length; ++i) {
04         suma += datos[i];
05     }
06
07     var promedio = suma / data.length;
08     return promedio;
09 }
```

```
> promedio(new double[] {1.0, 2.0, 3.0}) 2.0
```

Los arreglos son como cualquier variable así que podemos usarlos como parámetros igualmente. El tipo de dato de un arreglo de elementos `double` es `<double[]>`, el de un `String` es `<String[]>`, el de `int` es `<int[]>`, etc.

El código 03 calcula el promedio de un arreglo de `double`. Primero, recorremos el arreglo para sumar todos sus elementos y luego calculamos el promedio dividiendo la suma por el número total de valores usando la propiedad de longitud.

Los ciclos son los mejores amigos de los arreglos. Al igual que con los `String`, podemos recorrer todos los elementos de un arreglo usando un ciclo que se ve muy parecido al que usábamos

con Strings. La única diferencia entre un ciclo para un String y para un arreglo, es que para conocer la longitud de un arreglo no usamos un método, sino un propiedad que se llama igualmente <length>

Al igual que con Strings podemos usar diferentes tipos de ciclos, según lo que necesitemos.

Código 4 - un ciclo que empieza en el índice 1

01	<code>boolean elementosCrecientes(double[] datos) {</code>
02	<code> for (var i = 1; i < datos.length; ++i) {</code>
03	<code> if (data[i - 1] >= datos[i]) {</code>
04	<code> return false;</code>
05	<code> }</code>
06	<code> }</code>
07	<code> return true;</code>
08	<code>}</code>
09	

El código 4 implementa elementosCrecientes(), una función que verifica si los valores en un arreglo están ordenados de menor a mayor y sin repetidos. Toma una arreglo y verifica que cada elemento sea menor que el siguiente. Dado que la operación depende de la posición de cada elemento, necesitamos utilizar un ciclo tradicional, recorriendo sus índices aunque empezamos en el segundo elemento para en cada iteración comparar el elemento actual con el anterior, el elemento en el índice <i - 1>.

En la línea 03 verificamos la condición opuesta que queremos verificar, es decir que el elemento anterior sea mayor o igual al elemento actual, en caso de que un solo elemento la incumpla esta condición hace innecesario continuar todo el ciclo por lo que podemos tener un return temprano. Si el código llega a la línea 07, significa que no se encontró ningún elemento que rompa nuestra expectativa, por lo que podemos retornar true.



Si lees la documentación de Array, es posible que te hayas topado con `Arrays.copy()` y `Arrays.equals()`. Con ellos puedes implementar `incrementarElements()` como:

```
boolean elementosCrecientes(double[] datos) {  
    var ordenados = Arrays.copyOf(datos, datos.length);  
    Arrays.sort(ordenados);  
    return Arrays.equals(ordenados, datos);  
}
```

¿Cuál parece ser una mejor implementación? Bueno, en términos de complejidad, un tema fuera del alcance de este tutorial, esta versión usando métodos de Array tiene mayor complejidad en el tiempo y el espacio. Eso tampoco significa que sea inferior. Más bien ilustra un punto: la programación suele ser una batalla de compensaciones, es imposible escribir código que no tenga algún tipo de contraposición.

Mutabilidad en los arreglos

Código 5 - modificando el parámetro

```
01 double max(double[] datos) {  
02     Arrays.sort(datos);  
03     return datos[datos.length - 1];  
04 }
```

¿Qué podría estar mal con `max()`? Realicemos un experimento después de implementar la función:

```
> var elementos = new double[] {5.0, 4.0, 3.0};  
> max(elementos);  
> elementos;
```

```
double[3] { 5.0, 4.0, 3.0 }  
5.0  
double[3] { 3.0, 4.0, 5.0 }
```

Como puedes ver después de llamar a `max()`, el orden de los elementos fue modificado pero el nombre de la función `max()` no indica que el orden del arreglo se iba a modificar, parecía que lo único que haría el método era devolver el elemento máximo en el arreglo. Cuando utilices un arreglo como parámetro, recuerda que los cambios que realices relacionados con los índices de sus valores serán visibles fuera de la función. Esto rara vez es una buena idea.



La modificación de una matriz, y en realidad de cualquier objeto mutable, dada como parámetro suele ser una causa de errores lógicos en el código. Una forma sencilla de evitar esto es crear una copia de la matriz.

Parámetros de salida

Código 6 - Parámetros de salida

```
01 void duplicar(double[] data) {  
02     for (var i = 0; i < data.length; ++i) {  
03         data[i] *= 2;  
04     }  
05 }
```

```
> var elements = new double[] {5.0, 4.0, 3.0};  
> doubleIt(elements);  
> elements;
```

```
double[3] { 5.0, 4.0, 3.0 }  
double[3] { 10.0, 8.0, 6.0 }
```

Ya hemos dicho que cuando usas un arreglo como parámetro puedes realizarle cambios dentro de una función. Cuando llamas a una función para cambiar uno de los parámetros, un arreglo o un objeto mutable, generalmente se denomina como *parámetro de salida*. Como no produce un valor, generalmente hace que su función sea <void>, lo que significa que no devolverá ningún valor. Ya has visto una función nula: `Arrays.sort()`

Las funciones que tienen parámetros de salida deberían ser la excepción para las funciones públicas y no la regla. Y ten especial cuidado con funciones que tengan muchos parámetros de salida, definitivamente es mejor retornar un objeto, tema que veremos más adelante. Dicho esto, funciones privadas que sean auxiliares a funciones públicas podrían usar parámetros de salida dado que esas funciones se llaman controladamente por el código que tu escribes.

Arrays como valores de retorno

Código 7 - usando arreglos como valores de retorno

```
01 double[] doubleIt(double[] datos) {  
02     var copiaDatos = Arrays.copyOf(datos, datos.length);  
03     for (var i = 0; i < datos.length; ++i) {  
04         copiaDatos[i] *= 2;  
05     }  
06     return copiaDatos;  
07 }
```

Los arreglos como variables no tienen nada especial, así que no debería extrañarnos que se puedan usarlos como valores de retorno, cómo vemos la línea 06.

Antes habíamos visto que podemos usar arreglos como parámetros de salida. ¿Cuál es la alternativa? Hacer una copia del parámetro y hacer las modificaciones sobre esa copia. En la línea 02 hacemos la copia, y luego en la línea 04 hacemos el duplicado sobre esa copia.

Los valores de retorno no tienen que ser copias de otros parámetros, también pueden ser valores nuevos creados en la función. Veremos una de las razones para crearlos a continuación.

Código 8 - arreglos como una forma de devolver más de un valor

```
01 double discriminanteCuadrática(double a, double b, double c) {
02     return b * b - 4 * a * c;
03 }
04
05 double[] cuadrática(double a, double b, double c) {
06     double discriminante = discriminanteCuadrática(a, b, c);
07     if (discriminante < 0) {
08         return new double[0];
09     } else if (discriminante == 0) {
10         var valor = new double[] { -b / 2 * a };
11         return valor;
12     } else {
13         var raízCuadrada = Math.sqrt(discriminante);
14         var valor1 = (-b + raízCuadrada) / (2 * a);
15         var valor2 = (-b - raízCuadrada) / (2 * a);
16         var valores = new double[] { valor1, valor2 };
17         return valores;
18     }
19 }
```

La ecuación cuadrática tiene 0, 1 o 2 respuestas reales. ¿Cómo podemos representar eso en una función? Una opción es utilizar un arreglo ya que puede tener 0 o más elementos.

¿Qué determina cuántas soluciones tiene? Lo hace un valor llamado el discriminante que se calcula como se muestra en la línea 02. Una vez que lo calculamos en la línea 06, hacemos un condicional para crear los valores que vamos a devolver.

En la línea 07 devolvemos un arreglo vacío, en las líneas 10-11 se crea y devuelve un arreglo de un solo elemento y finalmente en las líneas 12-16 creamos un arreglo de dos valores.

Esta solución de regresar un arreglo para poder devolver 0, 1 o 2 valores, es efectiva pero no es ideal. Nuevamente los objetos pueden ser una mejor alternativa, sin embargo es útil saber que puedes usar arreglos para algo simple.

Valores predeterminados

Si un arreglo es un contenedor, ¿qué sucede cuándo creamos un contenedor con un tamaño pero sin ningún valor aún? Hagamos un experimento:

<pre>> new int[1] > new double[1] > new boolean[1] > new char[1] > new String[1]</pre>	<pre>int[1] { 0 } double[1] { 0.0 } boolean[1] { false } char[1] { '\000' } String[1] { null }</pre>
---	--

En programación no hay una manera simple de representar valores *vacíos* para muchos tipos de datos, pero la mayoría de ellos tienen un *valor cero* que es el que obtienes como valor por defecto en un arreglo. Hay 2 valores cero sobre los que quizás valga la pena ahondar: en efecto existe un carácter Unicode con el valor 0, este pertenece al rango de caracteres que no son imprimibles así que imprimirlo no hace mucho. Por otro lado, para Strings vemos el valor null. Tendremos mucho que decir sobre null en el futuro cercano, pero por ahora, solo recuerda que un arrays de String no tiene por defecto Strings vacíos. Este valor predeterminado también se utiliza para cualquier tipo de arrays de valores no primitivos.

Arrays Multidimensionales

Código 10 - matrices

```
01  int[][] matrizNúmeros(int tamaño) {
02      var matriz = new int[tamaño][tamaño];
03      var n = 1;
04      for (var i = 0; i < matriz.length; ++i) {
05          for (var j = 0; j < matriz.length; ++j) {
06              matriz[i][j] = n;
07              ++n;
08          }
09      }
10
11      return matriz;
12  }
```



```
> matrizNúmeros(3)
```

```
double[3][] { int[3] { 1, 2, 3 },  
              int[3] { 4, 5, 6 },  
              int[3] { 7, 8, 9 } }
```

Los arreglos pueden contener cualquier tipo de valor, incluso pueden contener arreglos. A un arreglo de arreglos se le suele llamar una matriz. La línea 02 declara una matriz cuadrada. Pero puede tener una matriz rectangular e incluso puede tener un arreglo que contenga arreglos de diferentes tamaños. Para hacer referencia a un valor en una matriz sólo tienes que proporcionar dos índices en lugar de uno, como puedes ver en la línea 04. `matrizNúmeros()` pone un número creciente, controlado por la variable `<n>`, en cada celda de izquierda a derecha, arriba hacia abajo.

Copiar una matriz

Hemos visto `Arrays.copyOf()` para hacer una copia de un arreglo. ¿Funciona igual con las matrices? No totalmente. Tenemos que usar la función más de una vez, primero para la estructura general y luego por cada hilera. Por ejemplo una manera de copiar una matriz de `int` sería:

```
int[][] copiar(int[][] inicial) {  
    var copia = Arrays.copyOf(inicial, inicial.length);  
    for (var i = 0; i < copia.length; ++i) {  
        copia[i] = Arrays.copyOf(inicial[i], inicial[i].length);  
    }  
    return copia;  
}
```

Si tuviéramos una matriz de `double`, entonces tendríamos que hacer una función idéntica, solo cambiando el tipo de datos. Esta es una limitación de Java con los valores primitivos, no hay una manera de general funciones que solo cambian en su tipo. Sin embargo es una limitación en la que se está trabajando actualmente.

Iteraciones

Código 11 - Usando el `for` mejorado

```
01 double average(double[] data) {  
02     var sum = 0.0;  
03     for (var n : data) {  
04         sum += n;  
05     }  
06 }
```

```

07     var average = sum / data.length;
08     return average;
09 }

```

Hemos visto que podemos realizar un ciclo usando una variable para obtener todos los índices válidos de un arreglo. Esto es propenso a errores porque podrías cometer un error en el valor inicial de la variable o en la condición del ciclo. Java ofrece una forma de ciclo para iterar sobre colecciones, como los arreglos, que puedes ver en la línea 03, llamada *for mejorado*. En lugar de trabajar con índices, crea una variable, denominada <n> en el código, y Java colocará el elemento que está en la iteración actual en esa variable. La sintaxis <for (var n: datos)> debe entenderse como: para cada elemento, <n>, en el arreglo <datos>. Entonces, en la línea 04, puedes usar <n> para actualizar la variable <suma>.

Veamos el código al lado del funcionalmente equivalente usando índices:

```

double average(double[] data) {
    var sum = 0.0;
    for (var n : data) {
        sum += n;
    }

    var average = sum / data.length;
    return average;
}

```

```

double average(double[] data) {
    var sum = 0.0;
    for (var i = 0; i < data.length; ++i) {
        var n = data[i];
        sum += n;
    }

    var average = sum / data.length;
    return average;
}

```

Esta sintaxis funcionará incluso para matrices. Veamos el código que cuenta el número de veces que 0 está en una matriz, usando ambas sintaxis una al lado de la otra:

```

int contarUnos(int[][] data) {
    var conteo = 0;
    for (var i = 0; i < data.length; ++i) {
        for (var j = 0; j < data[i].length; ++j) {
            if (data[i][j] == 1) {
                ++conteo;
            }
        }
    }
}

```

```

int contarUnos(int[][] data) {
    var conteo = 0;
    for (var fila : data) {
        for (var n : fila) {
            if (n == 1) {
                ++conteo;
            }
        }
    }
}

```

```
}  
    return conteo;  
}
```

```
}  
    return conteo;  
}
```

La única limitación de esta sintaxis es que no puedes usarla para modificar los elementos en los arreglos, solo consultarlos. Además, a veces es necesario recorrer el arreglo, pero el índice es tan importante como el elemento, por lo que deberá utilizar el recorrido con índices. Sin embargo, si es posible, utiliza la versión mejorada, ya que ofrece mejor legibilidad y es menos propensa a errores.

Ejercicios

E7.1

Escribe una función `másLargo()`, que reciba un arreglo de `String` y devuelva el índice del `String` con mayor cantidad de caracteres. En caso de haber más de uno con la misma cantidad de caracteres devuelve el que tenga el menor índice.

```
assertEquals(1, másLargo(new String[] { "abc", "casa", "cc" }));  
assertEquals(0, másLargo(new String[] { "a", "b", "c" }));
```

E7.2

Escribe una función `contiene()` que dado un arreglo de `String` y un `String`, retorne verdadero si ese `String` está en el arreglo y falso en caso contrario.

```
assertEquals(true, contiene(new String[] { "a", "b", "c" }, "b"));  
assertEquals(false, contiene(new String[] { "a", "b", "c" }, "1"));
```

E7.3

Escribe una función `buscar()` que dado un arreglo de `ints`, retorne un `int` que sea el índice del último elemento del arreglo que sea mayor que 0 y menor que 10. Si no hay un elemento que cumpla esta condición retorne -1.

```
assertEquals(2, buscar(new int[] { 5, 10, 3, 100 }));  
assertEquals(-1, buscar(new int[] { 10, 10, 10 }));
```

E7.4

Escribe una función `operar()` reciba un arreglo de `ints` y retorne un nuevo arreglo de `ints` con los mismos elementos del arreglo original después de multiplicarlos por 3 y sumarle 1. Por ejemplo si en el arreglo original está el elemento 2, en el arreglo retornado en esa posición debe aparecer el elemento 7.

```
assertEquals(new int[]{4, 7, 0}, operar(new int[] {1, 2, 0}));
assertEquals(new int[]{10, 13, 16}, operar(new int[] {3, 4, 5}));
```

E7.5

Escribe una función que retorne verdadero si todos los elementos de un arreglo de ints son únicos y falso si hay al menos un elemento repetido.

```
assertEquals(true, únicos(new int[] {5, 1, 3}));
assertEquals(false, únicos(new int[] {-1, 7, -1, 3}));
```

E7.6

Escribe una función `segundoMayor()`, que reciba un arreglo de más de un número entero y devuelva el segundo elemento mayor.



Hay un tema que se sale del alcance de este material y es el de la eficiencia en los algoritmos. Si implementamos una función como `segundoMayor()` usando `Arrays.sort()`, estaremos haciendo trabajo adicional porque no solo estamos hallando el segundo mayor, sino ordenando todos los elementos. Para este ejercicio, trata de hacerlo sin usar `Arrays.sort()`

```
assertEquals(3, segundoMayor(new int[] {5, 1, 3}));
assertEquals(7, segundoMayor(new int[] {-1, 7, 10, 3}));
```

E7.7

Escribe una función `moda()` que dado un arreglo de números enteros retorne el elemento que más se repite. Para esta función, haz una copia del arreglo dado y ordénalo antes de hacer el análisis. En caso de haber empates, devuelve el elemento menor.

```
assertEquals(2, moda(new int[] {3, 2, 4, 2, 1}));
assertEquals(1, moda(new int[] {1, 2, 4, 2, 1}));
```

E7.8

Escribe una función que reciba un arreglo de enteros positivos y retorne un histograma representándolo. El histograma debe ser un String con igual número de filas que elementos en el arreglo. Cada fila solo usa el carácter '*' y este se repite la cantidad de veces que indique el número correspondiente. Por ejemplo, si uso el arreglo [5, 1, 3], significa que la primera fila tendrá 5 caracteres, la segunda fila tendrá 1 y la tercera y última fila tendrá 3.

```

assertEquals("""
    *****
    *
    ***
    """, histograma(new int[] {5, 1, 3}));

assertEquals("""
    ***

    ***
    """, histograma(new int[] {3, 0, 3}));

```

E7.9

Dado un arreglo de números enteros entre 0 y 5, escribe una función que devuelva un nuevo arreglo de exactamente 6 posiciones. Cada posición en el arreglo resultante debe representar la cantidad de veces que aparece el número correspondiente en el arreglo original, es decir el número en el índice 0 representa cuántas veces estaba el número 0 en el arreglo original, el número en el índice 1 representa cuántas veces estaba el número 1 en el arreglo original, etc.

Por ejemplo para el arreglo [3], el valor de retorno sería [0, 0, 0, 1, 0, 0] porque en el arreglo original había cero 0s, cero 1s, cero 2s, un 3, cero 4s y cero 5s, mientras que el arreglo [2, 2] el valor de retorno sería [0, 0, 2, 0, 0, 0].

```

assertArrayEquals(new int[] {0, 0, 0, 1, 0, 0}, stats(new int[] {3}));
assertArrayEquals(new int[] {0, 0, 2, 0, 0, 0}, stats(new int[] {2, 2}));

```

E7.10

Crea una función que dadas las dimensiones de una matriz cuadrada, devuelva una matriz con 0 en todas las celdas, excepto en las celdas de la diagonal principal en donde debe haber un 1. La diagonal principal es una celda que tiene el mismo índice para su fila y columna. Por ejemplo, en una matriz cuadrada m de tamaño 3, entonces hay un 1 en `m[0][0]`, `m[1][1]` y `m[2][2]`.

```

assertArrayEquals(new int[][] {{1, 0, 0},
                                {0, 1, 0},
                                {0, 0, 1}}, m(3));

```

E7.11

Crea una función que dadas las dimensiones de una matriz cuadrada, devuelva una matriz con 0 en todas las celdas, excepto en las celdas de la diagonal contraria a la diagonal principal en donde debe haber un 1.

```

assertArrayEquals(new int[][] {{0, 0, 1},

```

```
{0, 1, 0},  
{1, 0, 0}}, m2(3));
```



E7.12

Escribe una función `ventana()` que reciba un arreglo de Strings y un número entero y devuelva una matriz donde se agrupen los elementos de esta manera:

- El número de hileras es el techo del tamaño del arreglo dividido por el número dado
- El número de columnas es el número dado
- Cada elemento del arreglo original se va almacenando en el primer índice disponible de la primera columna que no esté llena
- Si hay menos elementos que posiciones en la matriz, es correcta que estos elementos sean null

Por ejemplo si el arreglo es {"1", "2", "3", "4", "5", "6", "7", "8", "9", "10"} y el número es 3 entonces la matriz tiene 4 hileras y 3 columnas.

```
{{"1", "2", "3"},  
 {"4", "5", "6"},  
 {"7", "8", "9"},  
 {"10", null, null}}
```

Si el número dado hubiera sido 5 entonces el resultado sería:

```
{{"1", "2", "3", "4", "5"},  
 {"6", "7", "8", "9", "10"}}
```

```
var e = new String[]{"1", "2", "3", "4", "5", "6", "7", "8", "9", "10"};  
assertArrayEquals(new String[][] {{"1", "2", "3"},  
                                     {"4", "5", "6"},  
                                     {"7", "8", "9"},  
                                     {"10", null, null}}, ventana(e, 3);  
assertArrayEquals(new String[][] {{"1", "2", "3", "4", "5"},  
                                     {"6", "7", "8", "9", "10"}}, ventana(e, 5);
```



E7.13

Crea una función que reciba una matriz con solo 2 caracteres ' ' y '*' y retorne una nueva matriz que tenga ' ', '*' y '.' de esta manera:

- una celda de la matriz nueva tiene un '*' si en la matriz inicial había un '*'
- una celda que en la matriz inicial tenía un ' ', se vuelve un '.' en la matriz nueva si esa celda comparte una fila o columna donde había un '*' en la matriz inicial
- una celda en la matriz nueva tiene un '.' en los demás casos

Por ejemplo para una matriz como:

```
{{' ', ' ', ' ', ' '},
 {' ', ' ', '*', ' '},
 {' ', ' ', ' ', ' '}}
```

Debería devolver una matriz como esta:

```
{{' ', ' ', '. ', ' '},
 {'. ', '*', '. '},
 {' ', ' ', '. ', ' '}}
```

En la matriz original hay un solo asterisco en la posición (1, 1). Entonces la matriz que se devuelve, tiene '.' a través de toda la hilera con índice 1, salvo donde había un '*' y en toda la columna con el índice 1, salvo donde había un '*'

```
char[][] inicial = {{' ', ' ', ' ', ' '},
                    {' ', ' ', '*', ' '},
                    {' ', ' ', ' ', ' '}};

char[][] esperado = {{' ', ' ', '. ', ' '},
                    {'. ', '*', '. '},
                    {' ', ' ', '. ', ' '}};

assertArrayEquals(esperado, Arreglos.expandir(inicial));
```

```
char[][] inicial = {'*', ' ', '*'},
                    {' ', ' ', ' '},
                    {'*', ' ', '*'};

char[][] esperado = {'*', '. ', '*'},
                    {'. ', ' ', '. '},
                    {'*', '. ', '*'};

assertArrayEquals(esperado, Arreglos.expandir(inicial));
```

Miscelánea de Ejercicios

M1

Un emprendedor decidió subir el precio de su producto estrella por un porcentaje escogido. Pero entonces se dio cuenta que las personas dejaron de comprarlo en la misma cantidad. Entonces decidió hacer un descuento al precio actualizado por el mismo porcentaje. Sin embargo, al hacerlo el precio del producto no fue el mismo en el que comenzó.

Haz una función llamada `aumentoDescuento` que reciba un precio y un porcentaje, p de modo que $0 < p < 100$, y devuelva el precio resultante después de aplicar un aumento por el porcentaje dado y a ese resultado aplicarle un descuento por ese mismo porcentaje.

```
assertEquals(93.75, aumentoDescuento(100, 25.0), 1e-4);
assertEquals(960.0, aumentoDescuento(1000.0, 20.0), 1e-4);
assertEquals(990.0, aumentoDescuento(1000.0, 10.0), 1e-4);
```

M2

Después del paso de un huracán, queremos un análisis de como se afectaron las cercas que rodean las casas de una población. Para tal efecto alguien hace un diagrama empezando por un poste al azar y escribiendo "#" si el poste no fue afectado y "-" si el poste fue afectado y va a tener que ser reemplazado.

Haz una función que reciba un diagrama válido y devuelva la mayor cantidad contigua de postes que deben ser reemplazados. Dado que las cercas son circulares y el inicio de un diagrama es aleatorio, el primer paso es modificar el String para garantizar que no comienza en un "-".

Por ejemplo si el diagrama es "#--#---#-#" la respuesta es 3 dado que hay 4 postes que no deben ser reparados, el espacio más grande está entre el segundo y el tercer poste y el espacio mide 3 postes.

Por ejemplo si el String inicial es "---###---" el primer paso debe ser transformar ese String a "###-----" moviendo cada caracter "-" inicial de la cadena al final de la cadena.

Para este ejemplo la respuesta sería 6.

```
assertEquals(3, máximoHueco("#--#---#-#"));
assertEquals(5, máximoHueco("-----"));
assertEquals(0, máximoHueco("####"));
assertEquals(6, máximoHueco("###-----"));
```

M3

Un día despreocupado ves una nota en un papel en donde está escrito el número 121, pero por alguna razón en vez de leerlo "ciento veintiuno" lo lees "un uno, un dos, un uno" y ahora escribes ese "número" 111211, ahora puedes hacer ese ejercicio nuevamente: "tres unos, un dos, dos unos" y al escribirlo sería "311221" y te das cuenta que podrías continuar así indefinidamente.

Haz una función que tome un String que contiene solo dígitos, y un número entero y retorne el String resultante después de aplicar el proceso antes descrito, la cantidad de veces indicada por el parámetro.

```
assertEquals("111211", selfDescribing("121", 1));
assertEquals("311221", selfDescribing("121", 2));
assertEquals("13212211", selfDescribing("121", 3));
```

M4

Queremos hacer un software que provea estadísticas de un torneo deportivo. En este torneo los partidos pueden tener 3 resultados para un equipo: ganar, empatar o perder. Cada uno de estos resultados los asociamos con un caracter: 'O', '-' y 'X' respectivamente. Para visualizar los últimos 5 resultados podemos escribir un String así: "OO--X" que significa que sus últimos 2 partidos los gano, pero antes de esos partidos llevaba 2 partidos empatados y antes uno perdido. Los partidos más recientes tienen un índice menor en el String.

Haz una función que dado un String de 5 caracteres con los resultados de un equipo con la tendencia del equipo. La tendencia es el conteo de resultados iguales sin interrupción empezando desde el resultado más reciente. Por ejemplo en el ejemplo anterior la tendencia sería "2 ganados" porque el resultado más reciente es un partido ganado y a partir de ahí la cantidad de partidos ganados sin interrupción es 2.

Para las tendencias usa las palabras: "ganado, empatado, perdido" y en caso de que la tendencia sea mayor que 1, agrega una "s" a la palabra.

```
assertEquals("2 ganados", tendencia("OO--X"));
assertEquals("5 empatados", tendencia("-----"));
assertEquals("1 perdido", tendencia("X-XXX"));
```

Tipos de Datos

Código 1: codigo/jugs/tipos/Punto.java - Una clase que representa el tipo de datos de Punto

```
01 package codigo.jugs.tipos;
02
03 /**
04  * Representa un punto en un plano cartesiano.
05  *
06  * Un punto asocia dos números reales que son las coordenadas del punto.
07  */
08 public class Punto {
09
10     private double x;
11     private double y;
12
13     /**
14      * Construye un punto con el estado dado
15      */
16     public Punto(double x, double y) {
17         this.x = x;
18         this.y = y;
19     }
20
21     /**
22      * @return el valor de X para este punto
23      */
24     public double getX() {
25         return x;
26     }
27
28     /**
29      * @return el valor de Y para este punto
30      */
31     public double getY() {
32         return y;
33     }
34 }
```

codigo/jugs/tipos/geometria/Geometria.java

```
01 package codigo.jugs.tipos.geometria;
02
03 import codigo.jugs.basica.tipos.Punto;
04
05 /**
06  * Esta clase contiene métodos relacionados con tipos de datos
07  * relacionados con la Geometría
08  */
09 public class Geometria {
10
11     /**
12      * Retorna la distancia entre p1 y p2.
13      * La distancia aplica la fórmula pitagórica los atributos
14      * de los puntos dados.
15      *
16      * @param p1 un punto no nulo
17      * @param p2 un punto no nulo
18      * @return la distancia entre p1 y p2 como se explicó anteriormente
19      */
20     public static double distancia(Punto p1, Punto p2) {
21         var cuadradoDeDiferenciasXs = Math.pow(p1.getX() - p2.getX(), 2.0);
22         var cuadradoDeDiferenciasYs = Math.pow(p1.getY() - p2.getY(), 2.0);
23         var distancia = Math.sqrt(cuadradoDeDiferenciasXs + cuadradoDeDiferenciasYs);
24         return distancia;
25     }
26 }
```

codigo/jugs/tipos/geometria/Main.java

```
01 package codigo.jugs.tipos;
02
03 public class Main {
04     public static void main(String[] args) {
05         var p1 = new Point(0.0, 1.0);
06         var p2 = new Point(1.0, 0.0);
07         System.out.println(Geometria.distancia(p1, p2));
08     }
09 }
```

1.4142135623730951

Antes de usar tipos, codificamos la función de distancia con cuatro parámetros dobles. Veamos las diferencias:

Código 2 - Función distancia usando tipos de datos arriba y sin tipos de datos abajo

```

public static double distancia(Punto p1, Punto p2) {
    var squareOfDifferenceOfXs = Math.pow(p1.x() - p2.x(), 2.0);
    var squareOfDifferenceOfYs = Math.pow(p1.y() - p2.y(), 2.0);
    var distance = Math.sqrt(squareOfDifferenceOfXs + squareOfDifferenceOfYs);
    return distance;
}

public static double distancia(double x1, double y1, double x2, double y2) {
    var squareOfDifferenceOfXs = Math.pow(x1 - x2, 2.0);
    var squareOfDifferenceOfYs = Math.pow(y1 - y2, 2.0);
    var distance = Math.sqrt(squareOfDifferenceOfXs + squareOfDifferenceOfYs);
    return distance;
}

```

La versión de abajo tiene algunos problemas: estamos representando un punto como dos valores dobles, pero no es evidente que esos parámetros tengan alguna relación, puede que lo intuyas por el nombre pero nada más. No hay nada dentro del código que deje claro que el primer y el segundo parámetro y el tercero y el cuarto y último están relacionados. Si tienes varios métodos que usen la misma información, la misma variable representando un punto, debemos mover esas dos variables individualmente e intentar usarlas en el orden esperado. Es mucho mejor si podemos darle un nombre a las variables, un Punto, y podemos tratarlas como una única variable que tenga 2 valores mis posibilidad de confundirlos.



Cuando tengas funciones que tomen algunos parámetros que estén relacionados, piensa si puedes crear una clase para encapsularlos. Simplemente darles un nombre es un gran beneficio en la legibilidad del código y más adelante veremos más beneficios. La creación de clases es un concepto muy importante para crear software robusto.

La línea 03 contiene una declaración de clase especificada por la palabra clave *class*, que es como puedes crear tus propios tipos. Un tipo encapsula datos, define propiedades que siempre serán verdaderas para el tipo, llamadas invariantes, y define qué operaciones son válidas en esos tipos. Las clases también tienen visibilidad pero solo tienen 2 y la más común es que sean públicas. Como ocurre con muchos elementos, las clases también tienen bloques, por lo que verás, en las líneas 03 y 20, el inicio y el final de un bloque. Los identificadores de clase siguen las mismas reglas que los que has visto pero la convención es que comienzan con una letra mayúscula. De esa manera, cuando lees código, es muy fácil ver qué es una clase, qué es una función y qué es una variable.

Las clases tienen 3 elementos principales: **atributos** (variables), **constructores** y **métodos** (funciones). Las líneas 05 y 06 declaran dos atributos: un atributo de tipo double denominado x y un atributo de tipo double denominado y. Al igual que los parámetros, los atributos necesitan definir su tipo. Tienen modificadores de visibilidad, pero a diferencia de las funciones, el

modificador más común es el privado. Lo que significa que otro código no podrá ver los atributos directamente.

Al leer la sentencia class puedes ver que un Punto es la encapsulación de dos variables de tipo double, llamadas x e y.

La línea 08 inicia un constructor. Puedes saber que es un constructor porque no tiene un tipo de retorno y su identificador es el mismo que el nombre de la clase, incluyendo las mayúsculas. Al igual que las funciones, los constructores también pueden tomar parámetros. Los constructores se utilizan para inicializar los atributos de una clase. El constructor más común es aquel que toma los mismos números y tipos de parámetros que sus atributos. Luego usa esos parámetros para inicializar los atributos como ves en las líneas 09 y 10. Cada vez que veas la palabra clave this, se refiere al objeto en el que se está operando, por lo que <this.x> es el atributo x del objeto con el que estás interactuando. Los constructores también tienen el concepto de visibilidad y normalmente se utiliza un constructor público para crear objetos de una clase. Puedes tener muchos constructores en una clase y eso suele ser muy bueno porque significa que tienes más flexibilidad en la forma de crear objetos. Ahora está claro que para construir puntos necesitas conocer los valores de <x> e <y>.

Si bien hasta ahora hemos usado solo método públicos, con los atributos vamos a ir en contravía: todos los atributos, por ahora, van a ser privados. Si bien hay atributos que pueden no ser privados, igualmente es cierto que un atributo muy, muy rara vez debe ser público. Al hacer un atributo público este código falla por fuera de la clase Punto:

```
var miPunto = new Punto(-1.0, 3.0);  
var abcisa = miPunto.x; // ERROR  
var ordenada = miPunto.y; // ERROR
```

Si es cierto que la variable <miPunto> de tipo Punto, tiene atributos <x> e <y> no los puede acceder directamente como en el código anterior. ¿Entonces cómo puedo conocer la información que contiene un punto?

No sería de mucha utilidad si podemos crear objetos pero no interactuar con ellos, por lo que puedes mezclar datos y comportamiento en las clases. Las líneas 13 y 17 definen **métodos**. Los métodos son funciones que forman parte de una clase y operan sobre sus atributos, pero en términos prácticos un método es una forma especializada de una función por lo tanto todas las buenas prácticas que hemos visto para funciones, se aplican a métodos.

Para el Código 1 sólo estamos agregando los métodos básicos: **accesores**, métodos destinados a exponer algunos de los atributos de la clase. Así podemos conocer la información que contiene un punto. ¿No es mucho trabajo hacer los atributos privados y usar accesores? Se debe a la encapsulación: si los atributos son privados, puedes en el futuro cambiarlos de alguna manera y reimplementar los accesores y el resto del código funcionará sin ningún

cambio. Puede sonar extraño que pensemos en modificar los atributos de una clase, pero sucede más de lo que uno se imaginaría especialmente en software complejo.



Los métodos y los atributos se pueden escribir en cualquier orden en una clase. Por supuesto, tener cierta organización ayuda a los lectores de la clase a comprenderla. Elija este orden: variables de clase, atributos, constructores y métodos. En cuanto a los métodos, no hay consenso sobre cómo ordenarlos, pero intenta leer tu clase y ver si el orden tiene sentido.

Métodos no accesoros

Código 2: Un tipo de dato que representa una coordenada en una matriz.

```
01 package codigo.jugs.tipos;
02
03 public class Coordenada {
04
05     private int hilera;
06     private int columna;
07
08     public Coordenada(int hilera, int columna) {
09         this.hilera = hilera;
10         this.columna = columna;
11     }
12
13     /**
14      * Comprueba si dos coordenadas son ortogonales, es decir están una al lado de la otra.
15      *
16      * Esta implementación verifica si 2 coordenadas tienen el mismo valor para alguno de
17      * sus atributos y para el otro atributo la diferencia no es más de uno.
18      *
19      * @param otra una coordenada no nulo.
20      * @return verdadero si otra es ortogonal al objeto sobre el que se está operando
21      */
22     public boolean sonOrtogonales(Coordenada otra) {
23         if (hilera == otra.hilera || columna == otra.columna) {
24             return Math.abs(columna - otra.columna) == 1 || Math.abs(hilera - other.hilera) == 1;
25         }
26         return false;
27     }
28 }
```

```

01 package codigo.jugs.tipos;
02
03 public class Main {
04     public static void main(String[] args) {
05         var c1 = new Coordenada(3, 3);
06         var c2 = new Coordenada(3, 2);
07         System.out.println(c1.sonOrtogonales(p2));
08     }
09 }

```

```

true

```

Las clases pueden tener cualquier tipo de método. En el Código 2 definimos una clase `Coordenada` que encapsula una fila y una columna. Es útil calcular cuándo dos coordenadas son ortogonales: ambas coordenadas comparten uno de sus valores y la diferencia del otro valor es exactamente 1. Por ejemplo, imagina una cuadrícula como esta:

(0, 0)	(0, 1)	(0, 2)
(1, 0)	(1, 1)	(1, 2)
(2, 0)	(2, 1)	(2, 2)

Puedes ver que la coordenada (1, 1) tiene 4 vecinos ortogonales: (0, 1), comparten la misma columna y su fila está separada por 1, (1, 2), comparten la misma fila y su columna está separada por 1, (2, 1) y (1, 0) por la misma razón.

Las líneas entre 22 y 27 implementan dicho método. Todos los métodos tienen un modificador de visibilidad y su firma: un valor de retorno, un identificador y un conjunto de parámetros. No hay limitaciones sobre lo que puedes devolver o sobre los parámetros, tienes total libertad. Este método toma una `Coordenada` como parámetro, lo que significa que opera con dos: una `Coordenada` implícita en el código llamada `this` y la otra es el parámetro. Esto quizás se puede ver mejor cuando usamos el método:

```

var unoUno = new Coordenada(1, 1);
var unoDos = new Coordenada(1, 2);

unoUno.sonOrtogonales(unoDos); // sonOrtogonales opera sobre 2 coordenadas:
                                // <unoUno> y <unoDos>. En el código de sonOrtogonales,
                                // la variable sobre la que se llama el método, <unoUno>
                                // se referencia como <this> y la variable que es el
                                // parámetro se referencia con el nombre del parámetro

```

En la línea 23 verificamos si las dos coordenadas comparten la misma fila o la misma columna, y si eso es cierto, en la línea 24 revisamos la siguiente condición para ser ortogonales: que las filas o columnas estén separadas por 1, es decir que el valor absoluto de su diferencia es 1.

Podemos implementar más métodos para poder hacer otras operaciones sobre los tipos de datos, pero también podemos crear funciones (también llamadas métodos estáticos) que utilicen nuestros tipos de datos. exploremos la diferencia.

Estatico vs No- Estatico

Código 3 - Punto de clase con métodos.

```
01 package codigo.jugs.geometria.puntos;
02
03 public class Punto {
04
05     private double x;
06     private double y;
07
08     public Punto(double x, double y) {
09         this.x = x;
10         this.y = y;
11     }
12
13     public double distancia(Punto p) {
14         var cuadradoDeDiferenciasDeXs = Math.pow(this.x - p.x, 2.0);
15         var cuadradoDeDiferenciasDeYs = Math.pow(this.y - p.y, 2.0);
16         var distancia = Math.sqrt(cuadradoDeDiferenciasDeXs +
17 cuadradoDeDiferenciasDeYs);
18         return distancia;
19     }
20 }
```


Código 4 - Usando distancia como un método no estático

```
01 package code.jugs.points;
02
03 public class Main {
04     public static void main(String[] args) {
05         var p1 = new Punto(0.0, 1.0);
06         var p2 = new Punto(1.0, 0.0);
07         System.out.println(p1.distancia(p2));
08     }
09 }
```

Antes de este capítulo, siempre usamos el modificador static en nuestras funciones. Ahora que hablamos de objetos, vamos a escribir más método no estáticos que estáticos en nuestros tipos de datos. Entonces, ¿cuál es la diferencia?

Puedes pensar que los métodos no estáticos siempre son parte de un objeto. Primero necesitas un objeto y luego puedes llamar al método no estático en el objeto. Este método obtendrá acceso al atributo del objeto utilizando la palabra clave <this>.

Presta mucha atención a la línea 05 del código 4. Mira que el método de distancia se llama desde p1. Como se mencionó anteriormente, primero necesitas un objeto, el objeto sobre el que operarás. Ahora presta atención a las líneas 14 y 15 del Código 3 de Punto: en esas líneas accedes a los atributos de los 2 puntos necesarios para calcular la distancia. Para el objeto sobre el que estás operando usas <this>, para el parámetro accedes a ellos de la misma manera que lo has hecho antes.

Veamos ambos códigos juntos, usando métodos no estáticos, arriba, y métodos estáticos, abajo:

Código 5 - Función usando funciones no estáticas vs estáticas

```
public class Main {  
    public static void main(String[] args) {  
        var p1 = new Punto(0.0, 1.0);  
        var p2 = new Punto(1.0, 0.0);  
        p1.distancia(p2); // no-estática  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        var p1 = new Punto(0.0, 1.0);  
        var p2 = new Punto(1.0, 0.0);  
        Geometria.distancia(p1, p2); // estática  
    }  
}
```

¿Entonces cuál es mejor?

Como ocurre con muchas cosas en programación, no es útil pensar en términos de ser "mejor" o "peor". Lo que hay que pensar es en términos de ventajas y desventajas.

Una ventaja de los métodos no-estáticos es que tienen visibilidad de los atributos de un objeto: es común tener atributos que no están expuestos con accesores, por lo tanto, tener visibilidad de todos los atributos podría facilitar la implementación de ciertos métodos. Si ese no es el caso, entonces los métodos no tienen ninguna ventaja sobre los métodos estáticos desde el punto de vista de su implementación.

Pero no solo es importante la implementación: Los métodos no-estáticos usan una sintaxis en la que parece que los objetos se "comunican". En el código de arriba, p1 y p2 inician una conversación: p1 inicia la conversación al invocar su método distancia() y p2 se une a la conversación como parámetro. El resultado de esa conversación es su valor de retorno. Esta es una excelente manera de modelar sistemas complejos. Pero nuevamente, esta es una preferencia personal. Cuando se trabaja en equipos, lo más importante es tener un estilo de codificación coherente.



Existe una gran idea errónea sobre los métodos: algunos argumentan que el hecho de que los métodos accedan al estado de un objeto implica que los objetos serán mutables. La mutabilidad es un tema enorme y habrá un capítulo completo sobre ello. Hasta ese momento, verás una gran cantidad de código que utiliza métodos evitando la mutabilidad, por lo que no es cierto que no puedas tener uno sin el otro.

Igualdad

Código 6 - no uses == para verificar igualdad de objetos

```
01 package codigo.jugs.geometria;
02
03 public class Main {
04     public static void main(String[] args) {
05         var p1 = new Point(0.0, 0.0);
06         var p2 = new Point(0.0, 0.0);
07         System.out.println(p1 == p2);
08     }
09 }
10
```

```
false
```

¿Qué crees que debería pasar si usas “==” con dos objetos como puntos? Teniendo en cuenta cómo se comparan otras variables como los ints, podrías pensar que debería funcionar parecido, pero como ves en el código anterior el comportamiento puede parecer inesperado.

Cuando vimos Strings, dijimos que no debían ser comparados usando == y que deberías usar el método equals(), pero qué pasa con otros tipos de datos que creamos: Resulta que somos nosotros los que tenemos que agregar el método equals():

Código 07 - implementando equals para Punto

```
01 package codigo.jugs.tipos;
02
03 public class Punto {
04
05     private double x;
06     private double y;
07
08     public Punto(double x, double y) {
09         this.x = x;
10         this.y = y;
11     }
12
13     public boolean equals(Object otro) {
14         return otro instanceof Punto p && this.x == p.x && this.y == p.y;
15     }
16
17 }
```

```
01 package codigo.jugs.tipos;
02
03 public class Main {
04     public static void main(String[] args) {
05         var p1 = new Punto(0.0, 0.0);
06         var p2 = new Punto(0.0, 0.0);
07         System.out.println(p1.equals(p2));
08         System.out.println(p1.equals("un String"));
09     }
10 }
```

```
true
false
```

La línea 13 implementa el método equals() para el tipo Punto. El tipo de retorno parece esperado, pero ¿por qué usas Object como parámetro y qué significa? Bueno, Object es un tipo muy importante en Java ya que Object es implícitamente el supertipo de todos los tipos en Java. No profundizaremos mucho en qué es un supertipo, pero recuerda que existe una estrecha relación entre todos los tipos de datos, los que creas y los que se proporcionan en la biblioteca estándar de Java. Dado que hay infinitos

tipos posibles, es conveniente tener un método que puedas utilizar para comparar 2 tipos de datos cualesquiera.

El operador `<instanceof>` verifica que un parámetro es de un cierto tipo de dato, así que lo primero que preguntamos es si el parámetro es realmente un punto, y damos el nombre `<p>` a ese punto para verificar si los atributos `<x>` e `<y>` son iguales. Este es un patrón más o menos sencillo de utilizar, aunque al principio puede parecer engorroso pero te acostumbras rápidamente.

Implementación por defecto de una clase

Código 08 - una clase minimalista

```
01 public class Foo {
02
03 }
```

```
01 public class Main {
02     public static void main(String[] args) {
03         var foo = new Foo();
04         var foo2 = new Foo();
05         System.out.println(foo.toString());
06         System.out.println(foo.equals(foo2));
07     }
08 }
```

```
code.jugs.foo.Foo@251a69d7
false
```

¡Espera! ¿Cómo es posible llamar a `toString()` y a `equals()` en la clase `Foo` si no los has implementado? Bueno, se heredan de la clase `Object`. ¡Pero primero lo primero!

Java, hoy en día, tiene dos tipos principales: tipos primitivos y tipos de referencia. Has visto algunas primitivas: `int`, `boolean`, `char`, etc. Has visto algunos tipos de referencia: `String`, `LocalDate` y la clase `Punto` y `Longitud` en este capítulo. Los tipos de referencia son fáciles de detectar porque es necesario utilizar el operador `<new>` para crearlos.

Algo importante que debes saber es que en Java todas las clases heredan directa o indirectamente de la clase Object. La herencia es un tema fuera del alcance de este material, pero si heredas de una clase los métodos de esa clase. Eso garantiza en Java que todas las clases tienen un conjunto básico de métodos, por lo que otras clases pueden depender de ellos. Ya has visto dos de esos métodos: equals(Object) y toString() incluso si no los escribes tú mismo para tu clase, puedes llamarlos en cualquier clase que crees, aunque lo que retornan puede parece confuso.

Si no escribes tu propio toString(), imprimirá una representación interna de dónde está almacenado el objeto. Esto puede cambiar de una computadora a otra, ¡así que nunca confíes en ello! Implementar toString() es muy sencillo, por ejemplo para Punto:

Código 09 - implementando toString()

```
package codigo.jugs.tipos;

public class Punto {

    private double x;
    private double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public String toString() {
        return String.format("(%f, %f)", x, y);
    }

}
```

```
package codigo.jugs.tipos;

public class Main {
    public static void main(String[] args) {
        var p1 = new Punto(0.0, 1.0);
        System.out.println(p1.toString());
    }
}
```

```
(0.000000, 1.000000)
```

En el código 09 ves una implementación de `toString()`, desde que uses esa firma, habrás implementado `toString()` correctamente. La implementación del método ya dependerá del tipo de dato.

Como hemos dicho anteriormente, `equals()` por sí mismo no hace nada muy interesante, deberás implementarlo como lo hemos visto antes. Solo ten en cuenta que si no lo implementas, igual lo podrás usar aunque recuerda su comportamiento poco intuitivo.

Hay más métodos que heredan tu objeto, aunque esos 2 son los que probablemente uses más por ahora. Así que recuerde que métodos como `toString()` y `equals(Object)` son especiales, ya que se heredan, ¡y luego en Java todas las clases heredan de `Object` directa o indirectamente!

Ejercicios

E8.1

Escribe un tipo de dato Autor, para representar el nombre de una persona.

Constructor
Autor(String nombre, String apellido)
Métodos
String nombre()
String apellido()
String toString()

Asserts

```
var nombre = "John";
var apellido = "Doe";
var johnDoe = new Autor(nombre, apellido);

assertAll(
    () -> assertEquals(nombre, user.nombre()),
    () -> assertEquals(apellido, user.apellido()),
    () -> assertEquals("Doe, John", user.toString())
);
```

```
var nombre = "Jane";
var apellido = "Roe";
var janeRoe = new Autor(nombre, apellido);
assertEquals(nombre, janeRoe.nombre());
assertEquals(apellido, janeRoe.apellido());
assertEquals("Roe, Jane", janeRoe.toString());
```

E8.2

Escribe una clase Longitud.

Constructor
Longitud (double medida)

Métodos
<code>double medida()</code>
Longitud <code>sumar</code> (Longitud otra)
<code>boolean equals</code> (Object obj)

Asserts

<code>var measure = 3.14;</code> <code>assertEquals(measure, new Longitud(measure).measure(), 0.0);</code>
<code>assertEquals(new Longitud(3.0), new Longitud(1.0).suma(new Longitud(2.0)), 0.0);</code>
<code>assertEquals(new Longitud(3.25), new Longitud(0.75).suma(nueva Longitud(2.50)), 0.0);</code>

E8.3

Escribe una clase UPC que encapsule un String que representa un número UPC. El constructor toma un String sin su dígito de verificación y tiene un accesor que devuelve el número con el dígito de verificación. El método para calcular el dígito de verificación lo puedes encontrar en el ejercicio E6.2 que se encuentra en el capítulo “Condicionales”.

Constructor
<code>UPC(String número)</code>
Métodos
String <code>número</code> ()

Asserts

<code>var número = "03600024145";</code> <code>var númeroConVerificación = number + "7";</code> <code>assertEquals(númeroConVerificación, new UPC(número).número());</code>
<code>var número = "12345678901";</code> <code>var númeroConVerificación = number + "2";</code> <code>assertEquals(númeroConVerificación, new UPC(número).número());</code>

E8.4

Escribe una clase `Círculo`. Recuerda usar la clase `Punto` con el método no-estático `distancia` que se encuentra en este capítulo.

Constructor
<code>Círculo(Punto centro, double radio)</code>
Métodos
<code>Punto centro()</code>
<code>double área()</code>
<code>double circunferencia()</code>
<code>boolean seInterceptan(Círculo otro)</code>

Utiliza estas fórmulas para los métodos:

- Área: $\text{Math.PI} * \text{radio} * \text{radio}$
- Circunferencia: $2 * \text{Math.PI} * \text{radio}$
- Intersección: Encuentra la distancia entre los centros de ambos círculos. Los círculos se interceptan si la distancia es igual o menor que la suma de los radios.

Asserts

```
assertEquals(Math.PI, new Círculo(new Punto(0.0, 0.0), 1.0).área(), 1e-3);
assertEquals(12.566, new Círculo(new Punto(3.0, 4.0), 2.0).área(), 1e-3);
```

```
assertEquals(Math.PI * 2.0, new Círculo(new Punto(0.0, 0.0), 1.0).circunferencia(),
1e-3);
assertEquals(12.566, new Círculo(new Punto(0.0, 0.0), 2.0).circunferencia(), 1e-3);
```

```
assertEquals(new Círculo(new Punto(1.0, 2.0), 3.0), (new Círculo(new Punto(1.0, 2.0), 3.0)));
assertNotEquals(new Círculo(new Punto(0.0, 0.0), 1.0), (new Círculo(new Punto(0.0, 1.0), 1.0)));
assertNotEquals(new Círculo(new Punto(0.0, 0.0), 1.0), (new Círculo(new Punto(1.0, 0.0), 1.0)));
assertNotEquals(new Círculo(new Punto(0.0, 0.0), 1.0), (new Círculo(new Punto(0.0, 0.0), 2.0)));
```

```
var ceroCero = new Círculo(new Punto(0.0, 0.0), 1.0);
var unoCero = new Círculo(new Punto(1.0, 0.0), 1.0);
assertTrue(ceroCero.seIntercepta(unoCero));
```

```
var ceroCero = new Círculo(new Punto(0.0, 0.0), 1.0);
var dosDos = new Círculo(new Punto(2.0, 2.0), 1.0);
assertFalse(ceroCero.intersects(dosDos));
```

E8.5

Escribe una clase Fracción que tenga operaciones para sumar, restar, multiplicar y dividir fracciones. Una fracción puede considerarse como la encapsulación de dos números enteros llamados: numerador y denominador.

Un concepto muy importante para las fracciones es la reducción. Cuando operas una fracción, quieres usar la fracción con el numerador y denominador más pequeño que sea equivalente. Para ello, necesitamos tener una manera de encontrar el máximo común divisor, mcd, y el mínimo común múltiplo, mcm. Puedes utilizar estas funciones para calcularlos:

```
public class Aritmética {  
  
    public static int mcd(int a, int b) {  
        return b == 0 ? a : mcd(b, a % b);  
    }  
  
    public static int mcm(int a, int b) {  
        return a / mcd(a, b) * b;  
    }  
}
```

Así se debe implementar la clase Fraction (supongamos que el objeto sobre el que se opera es <this> y el parámetro es <that>):

Constructor:

1. Encuentra el MCD de los parámetros: numerador y denominador. Utiliza Math.abs para asegurarte que sea positivo.
2. Calcula numerador / MCD, llámémoslo <numeradorReducido>
3. Calcula el denominador / MCD, llámémoslo <denominadorReducido>.
4. Si el parámetro denominador es negativo, entonces el this.numerador y this.denominador son numeradorReducido * -1 y denominadorReducido * -1. De lo contrario, this.numerador y this.denominador son iguales a numeradorReducido y denominadorReducido.

Sumar: La fracción resultante es

$$\frac{this.numerador * MCM / this.denominador + that.numerador * MCM / that.denominador}{MCM}$$

Restar: La fracción resultante es

$$\frac{this.numerador * MCM / this.denominador - that.numerador * MCM / that.denominador}{MCM}$$

Otra alternativa es implementar la resta usando la suma:

$$\frac{a}{b} - \frac{c}{d} = \frac{a}{b} + \textit{opposite}\left(\frac{c}{d}\right)$$

Donde:

$$\textit{opposite}\left(\frac{a}{b}\right) = \frac{-a}{b}$$

- Multiplicar: la fracción resultante es

$$\frac{\textit{this.numerator} * \textit{that.numerator}}{\textit{this.denominator} * \textit{that.denominator}}$$

- División: La fracción resultante es

$$\frac{\textit{this.numerator} * \textit{that.denominator}}{\textit{this.denominator} * \textit{that.numerator}}$$

Otra alternativa es definir la división en términos de la multiplicación:

$$\frac{a}{b} \div \frac{c}{d} = \frac{a}{b} * \textit{reciprocal}\left(\frac{c}{d}\right)$$

Donde:

$$\textit{reciprocal}\left(\frac{a}{b}\right) = \frac{b}{a}$$

Constructor
Fracción (int numerador, int denominador)
Métodos
Fracción sumar (Fracción that)
Fracción restar (Fracción that)
Fracción multiplicar (Fracción that)

Fracción <code>dividir</code> (Fracción that)

Asserts

<code>assertEquals(new Fracción(1, 2), new Fracción(2, 4));</code>
--

<code>assertEquals(new Fracción(1, 2), new Fracción(-1, -2));</code> <code>assertEquals(new Fracción(-1, 2), new Fracción(1, -2));</code>
--

<code>assertEquals(new Fracción(1, 1), new Fracción(1, 4).suma(nueva Fracción(3, 4)));</code> <code>assertEquals(new Fracción(7, 12), new Fracción(1, 3).sumar(new Fracción(1, 4)));</code>
--

<code>assertEquals(new Fracción(3, 4), new Fracción(1, 1).restar(new Fracción(1, 4)));</code>

<code>assertEquals(new Fracción(5, 24), new Fracción(5, 6).multiplicar(new Fracción(1, 4)));</code>

<code>assertEquals(new Fracción(1, 1), new Fracción(-1, 2).dividir(new Fracción(-1, 2)));</code>
--

Constructores

Código 01 - Protección de invariantes

```
01 public class Longitud {
02     private double medida;
03
04     public Longitud(double medida) {
05         if (medida <= 0.0) {
06             var mensaje = String.format("Medida negativa: %f", medida);
07             throw new IllegalArgumentException(mensaje);
08         }
09
10         this.medida = medida;
11     }
12
13     public double medida() {
14         return this.medida;
15     }
16 }
```

En el código 01 modelamos una longitud. Las longitudes se usan para medir distancias por lo tanto no tiene sentido que sean negativas. Esta es la invariante para longitudes: su medida debe ser siempre positiva.

En la línea 04 tenemos un constructor para la clase Longitud. En la línea 05 verificamos que los clientes de la clase no intenten crear longitudes de medidas imposibles, si es así lanzamos una excepción en la línea 07, lo que significa que el código dejará de ejecutarse, se mostrará un mensaje de error y se muestra el estado en que se encontraba el programa cuando ocurrió el error. La invariante se puede hacer cumplir desde el constructor. Es imposible que se cree una Longitud que no sea válida.

¡Esto es crucial! Si proteges tus invariantes en tu código, comenzando por el constructor, significa que cuando un método toma un objeto de tu clase, probablemente no haya nada que validar desde el parámetro ya que si pudiste construir el objeto sus invariantes fueron verificados en ese momento. Uno de los grandes problemas del software es que los programadores no validan los parámetros de sus funciones, por lo que centralizar algunas de esas validaciones en los constructores hará que muchos errores sean imposibles.

IllegalArgumentException es solo una de las muchas excepciones que proporciona Java e incluso puedes crear la tuya propia. Una excepción transmite un error que ocurrió durante la

ejecución de su programa. Los errores de programación, como intentar crear una longitud negativa, deberían finalizar ruidosamente la ejecución de su programa para que no olvides que es necesario cambiar tu código. La verdadera solución a un error de programación es evitar llamar al código con el argumento incorrecto.

Además de esa excepción, no hay nadie a quien quiera conocer: `IllegalStateException`. Como su nombre lo dice aquí el problema no está en un parámetro en particular sino en algo en general como que los 3 ángulos de un Triángulo no suman 180. Al final no importa mucho si eliges `IllegalArgumentException` o `IllegalStateException` siempre y cuando proporciones un buen mensaje de error. Incluye en ese mensaje tanta información como puedas, porque eso ayudará al cliente de tu código a saber cuál podría ser la causa del error: p.j. si los 3 ángulos no suman 180, imprime cada valor e imprime la suma.

Las excepciones no se devuelven. En cambio, son arrojadas. Una excepción lanzada finaliza inmediatamente la ejecución del constructor o función y, si no se atrapa, eventualmente finalizará la ejecución de sus programas. El manejo de errores está fuera del alcance de este material, pero por ahora es suficiente que sepas cómo proteger las invariantes de tus clases y que la solución es cambiar el código infractor.

Múltiples Constructores

Código 02 - Múltiplos de constructores

```
01 public class Circulo {
02
03     private Punto centro;
04     private Longitud radio;
05
06     public Circulo(Punto centro, Longitud radio) {
07         this.centro = Objects.requireNonNull(centro);
08         this.radio = Objects.requireNonNull(radio);
09     }
10
11     public Circulo(Punto centro, Punto unPunto) {
12         this(centro, new Longitud(centro.distancia(unPunto)));
13     }
14
15 }
```

Las líneas 06 y 11 tienen constructores para la clase `Círculo` que puedes reconocer porque su identificador es el nombre de la clase y no hay valor de retorno. Con diferentes constructores puedes tener diferentes parámetros para crear objetos, lo cuál es increíble, cuanto más flexible sea tu clase, será más fácil de reutilizar. Pero al introducir nuevos constructores, puedes encontrarte con el problema de olvidar proteger los invariantes en todos ellos, por lo que generalmente es una buena idea que los constructores adicionales se basen en otro, generalmente llamado constructor primario donde se protegen los invariantes.

En Java para basar un constructor en otro puedes usar la expresión `this()`, lo que significa que se llamará al constructor de la clase que coincida con los parámetros. Entonces la línea 12 usa `this()` para llamar al constructor `Círculo(Punto,Longitud)` que tiene las validaciones para proteger las invariantes de la clase. Después de llamar a `this()` eres libre de escribir más código, aunque `Círculo` no lo hace, recuerda que puedes hacerlo.

En las líneas 07 y 08, protegemos las invariantes de la clase `Círculo` asegurándonos de que el centro y el radio no sean null. ¿Qué es null? Bueno, es un gran tema que tendrá su propio capítulo, pero puedes pensar en null como sinónimo de "no presente", por lo que queremos asegurarnos de que cuando se llama al constructor, el cliente del código tiene un punto y una longitud. Nuevamente tendremos mucho más que discutir en el futuro sobre null, pero por ahora basta con decir que puedes usar `Objects.requireNonNull()` para evitar valores null en tus parámetros. Si se usa null, se generará una excepción tal como lo hicimos con `Longitud`.

Fábricas Estáticas

Código 03 - Fábricas Estáticas

```
01 public class Ángulo {
02     private double medida;
03     private boolean centígrados;
04
05     private Angle(double medida, boolean centígrados) {
06         this.medida = medida;
07         this.centígrados = centígrados;
08     }
09
10     public static Ángulo enCentígrados(double medida) {
11         return new Ángulo(medida, true);
12     }
13
14     public static Ángulo enRadianes(double medida) {
15         return new Ángulo(medida, false);
16     }
17
18     public double centígrados() {
19         return this.centígrados
20             ? this.medida : Math.toDegrees(this.medida);
21     }
22
23     public double radianes() {
24         return this.centígrados
25             ? Math.toRadians(this.medida) : this.medida;
26     }
27
28 }
```

La línea 06 tiene un constructor típico para `Ángulo` que toma una medida y un boolean, también llamado un *flag*, para saber si el ángulo esta medido en grados centígrados o en radianes. ¡Sin embargo, el constructor es privado! Hacer que los clientes de la clase recuerden establecer el valor correcto del boolean es propenso a errores. Las líneas 11 y 12 tienen fábricas estáticas para crear ángulos. Una fábrica estática es solo un método que crea objetos y es estático. Dado que las fábricas estáticas son solo métodos, pueden tener nombres, así que podemos tener una fábrica estática llamada

`enCentígrados()` y una llamada `enRadianes()` que hace explícito qué es lo que queremos hacer.

¿Por qué preferirías una fábrica estática a un constructor?

Como se mencionó, las fábricas estáticas tienen nombres, por lo que puedes transmitir información sobre el objeto construido en su nombre. No has aprendido sobre herencia pero en una fábrica estática puedes devolver diferentes tipos de datos. Es posible que también la fábrica estática tenga lógica para devolver objetos almacenados en caché o algo similar.

El único inconveniente es que actualmente en la documentación producida por Javadoc no hay nada que haga que esta fábrica estática se destaque.

No todos los tipos tendrían beneficios para las fábricas estáticas, pero es algo que también puedes tener en cuenta al diseñar tus clases.

Java Beans

Código 04 - Punto como un Java Beans. ¡No hagas esto innecesariamente!

```
01 public class Punto {
02     private double x;
03     private double y;
04
05     public void setX(double x) {
06         this.x = x;
07     }
08
09     public void setY(double y) {
10         this.y = y;
11     }
12
13     public double getX() {
14         return x;
15     }
16
17     public double setY() {
18         return y;
19     }
20 }
```

Cuando comenzó Java, uno de los impulsos clave para usarlo fue su capacidad de tener herramientas que automaticen algunas de sus tareas de programación. Para ayudar a las herramientas a "comprender" el código que nosotros escribimos, se creó un estándar llamado Java Beans. Para que una clase sea considerada un Java Bean, necesitaba tener un constructor sin argumentos (o ningún constructor para que ese constructor sea agregado por el compilador) y para cada atributo <foo> un método setFoo() para darle un valor y un método getFoo() para exponer ese atributo.

Los Java Beans fueron excelentes para esta tarea: dejar que las herramientas hagan parte del trabajo por ti, pero de alguna manera su propósito se confundió y la gente comenzó a escribir todas sus clases como Java Beans. Esto es muy desafortunado

porque los Java Beans no brindan ningún beneficio si no usas herramientas que los necesiten, así que no caigas en la trampa de usar Java Beans innecesariamente.

Sabrás que necesitas un Java Bean cuando en la documentación de una librería te indiquen que necesitas escribir tus clases con la estructura de un Java Bean.

Constructores Con Visibilidad por Defecto

Código 05 - visibilidad por defecto en un constructor

```
01  /**
02   * NúmeroSecreto representa un número aleatorio
03   * de 4 dígitos no repetidos
04   */
05  public class NúmeroSecreto {
06      private String número;
07
08      public NúmeroSecreto() {
09          var random = new Random();
10          var usados = new boolean[10];
11          var builder = new StringBuilder();
12
13          while (builder.length() < 4) {
14              var dígito = random.nextInt(10);
15              for (; usados[dígito]; dígito = (dígito + 1) % 10);
16              usados[dígito] = true;
17              builder.append(dígito);
18          }
19
20          this.número = builder.toString();
21      }
22
23      NúmeroSecreto(String número) {
24          this.número = número;
25      }
26  }
```

La aleatoriedad es a veces algo que necesitamos en nuestros programas. Java proporciona un conjunto sólido de clases para usar la aleatoriedad, incluida la clase `Random`, que está bien para programas simples.

Crear una clase `NúmeroSecreto` para encapsular un número aleatorio de 4 dígitos no repetidos es perfecto. La línea 08 crea un objeto de tipo `Random` y en la línea 13 usamos su método `nextInt()`, que toma un argumento de límite superior por lo que usamos el argumento 10 para generar un número del 0 al 9.

¿Cómo evitamos repeticiones? Creamos un arreglo de booleanos, en la línea 10, para que, por ejemplo, `usados[5]` sea falso si aún no se ha usado 5 en el número. Primero creamos un dígito aleatorio en la línea 14 y luego en la línea 15 usamos un ciclo para buscar un dígito que no hayamos usado. Si la condición no ha sido lograda, prueba con el siguiente número, por ejemplo si escogimos el número 3 y esté está usado, entonces intentamos el 4 y así sucesivamente. El incremento usa el operador módulo para garantizar que una vez lleguemos al número máximo válido, que es 9, el siguiente número que intentamos es 0 y luego 1 y así sucesivamente.

Una vez que ya tenemos un número aleatorio que no ha sido usado antes, lo marcamos como usado en nuestro arreglo de booleanos en la línea 16 y lo agregamos al número que estamos construyendo en la línea 17 usando un `StringBuilder` que construimos en la línea 11.

Este proceso se repite hasta que el número aleatorio en construcción tenga menos de 4 dígitos. Una vez tenga esta longitud, el ciclo se acaba e inicializamos el atributo llamando el método `toString()` de `StringBuilder` en la línea 20.

Pero ahora nos enfrentamos a una pregunta compleja: ¿Cómo podemos escribir pruebas unitarias para una clase así? Pueden haber varias soluciones pero una manera muy cercana a Java es usar un constructor con visibilidad por defecto. Hemos visto constructores públicos y privados antes, si no usa un modificador obtendrá algo que se llama visibilidad por defecto o protegido por paquete como en la línea 22. Los métodos/constructores públicos se pueden usar en cualquier lugar, los métodos/constructores privados solo se pueden usar en la misma clase que están definidos. Los métodos/constructores protegidos por paquete se pueden usar en otras clases que forman parte del mismo paquete y, como recordarás, las pruebas unitarias deben crearse en el mismo paquete de la clase que prueban, precisamente por esta razón.

Entonces, una forma muy sencilla de probar clases que podrían tener este tipo de dependencias es tener un constructor protegido por paquete que evite las dificultades y establezca el estado directamente en lo que necesita. No existen soluciones universales, así que recuerda que tienes más de una herramienta para lograr un resultado y, a veces, ¡la herramienta más simple es la menos utilizada!



Vale la pena leer el API de [Random](#) al menos una vez más.

Side Quest: Builders

Código 06 - Builders

```
01 package code.jugs.address;
02
03 public class DirecciónBuilder {
04     String dirección = "";
05     String apartadoDeCorreos = "";
06     String númeroApartamento = "";
07     String códigoPostal = "";
08     String ciudad = "";
09     String estado = "";
10     String país = "";
11
12     public DirecciónBuilder dirección(String dirección) {
13         this.dirección = dirección;
14         return this;
15     }
16
17     public DirecciónBuilder apartadoDeCorreos(String apartadoDeCorreos){
18         this.apartadoDeCorreos = apartadoDeCorreos;
19         return this;
20     }
21
22     public DirecciónBuilder númeroApartamento(String númeroApartamento){
23         this.númeroApartamento = númeroApartamento;
24         return this;
25     }
26
27     public DirecciónBuilder códigoPostal(String códigoPostal) {
28         this.códigoPostal = códigoPostal;
29         return this;
30     }
31
32     public DirecciónBuilder ciudad(String ciudad) {
33         this.ciudad = ciudad;
34         return this;
35     }
36 }
```

```

37
38     public DirecciónBuilder estado(String estado) {
39         this.estado = estado;
40         return this;
41     }
42
43     public DirecciónBuilder país(String país) {
44         this.país = país;
45         return this;
46     }
47
48     public Dirección build() {
49         return new Address(this);
50     }
51 }

```

```

01     public class Dirección {
02         private String dirección;
03         private String apartadoDeCorreos;
04         private String númeroApartamento;
05         private String códigoPostal;
06         private String ciudad;
07         private String estado;
08         private String país;
09
10         Dirección(DirecciónBuilder builder) {
11             this.dirección = builder.dirección;
12             this.apartadoDeCorreos = builder.apartadoDeCorreos;
13             this.númeroApartamento = builder.númeroApartamento;
14             this.códigoPostal = builder.códigoPostal;
15             this.ciudad = builder.ciudad;
16             this.estado = builder.estado;
17             this.país = builder.país;
18         }
19
20         // más código
21     }

```



```

01 public class Main {
02
03     public static void main(String[] args) {
04         var dirección1 = new DirecciónBuilder()
05             .dirección("123 Main Street")
06             .ciudad("Una Ciudad")
07             .códigoPostal("12345")
08             .país("ABC")
09             .build();
10
11         var dirección2 = new DirecciónBuilder()
12             .dirección("456 Java Avenue")
13             .númeroApartamento("101")
14             .códigoPostal("98765")
15             .ciudad("Oak")
16             .estado("Sun")
17             .país("ABC")
18             .build();
19
20         var dirección3 = new DirecciónBuilder()
21             .apartadoDeCorreos("13579")
22             .city("Foo")
23             .state("Bar")
24             .country("ABC")
25             .build();
26     }
27 }

```

Las direcciones son tipos de datos complejos que tienen diferentes valores y que varían de un país a otro. En este ejemplo, las direcciones tienen una línea de dirección o un número de apartado postal, un número de apartamento opcional, códigos postales que con en algunos países obligatorios y en otro no, una ciudad, un estado, que en algunos países es opcional y un país. Puedes intentar tener un constructor para cada combinación posible, pero eso sería propenso a errores y, como todos los parámetros son Strings sería difícil recordar el orden.

Una posible alternativa es escribir un builder. Has utilizado un builder antes: `StringBuilder` para crear String complejos. Creamos un `StringBuilder` y llamamos el

método `append()` reiteradamente para agregar información al `String` y al final usamos `toString()` para finalizar el proceso de construcción y completar el `String`.

Para usar un `Builder` necesitas principalmente dos clases: la clase `Builder` que usualmente se llama `XXXBuilder` donde `XXX` es el tipo de dato a construir y la clase que deseas construir, en nuestro caso: `DirecciónBuilder`. La clase `Builder` debe tener los mismos atributos que la clase que se está creando, todos con visibilidad por defecto y, opcionalmente, puedes darles un valor predeterminado. En este ejemplo, todos los atributos comienzan como un `String` vacío. Los `Builder` no necesitan un constructor, a cambio lo que necesitas es tener un método con el mismo nombre del atributo, que toma un valor para ese atributo y devuelve el objeto que se está construyendo. Parece extraño tener esa declaración `<return this>`, pero eso solo significa que después de cambiar el estado del objeto, el método lo devuelve para que puedas encadenar múltiples llamadas de estos métodos, similar a cómo puedes usar `append()` en `StringBuilder`. Además de esos métodos, necesitas un método `build()` que finalice el proceso de construcción, llamando al constructor de `Dirección` y pasándole como parámetro, el objeto `Builder` que tiene todos los datos con que lo has configurado.

La clase `Dirección` tiene un constructor protegido por paquete. Eso garantiza que los clientes del código necesitan utilizar la clase `Builder`. El constructor toma el `Builder` y le extrae el estado. En la clase `Main` puedes ver que crear muchos tipos de direcciones es muy sencillo.

En los `Builder` podrías tener validaciones en el método `build()` y también puedes agregar un constructor con parámetros para hacer estos atributos obligatorios.

Ejercicios

E9.1

Expande la clase `Fracción` que hiciste para el ejercicio 8.5 para incluir los siguientes miembros:

Constructor
<pre>public Fraccion(int numerador) Crea una fracción con el numerador dado y el denominador 1</pre>

También asegúrate de validar la invariante de que el denominador no sea 0. En caso de que la invariante se incumpla, lanza una excepción de `IllegalArgumentException`.

Asserts

<pre>assertEquals(new Fracción(2, 1), new Fracción(2));</pre>
<pre>assertThrows(IllegalArgumentException.class, () -> new Fracción(1, 0));</pre>

E9.2

Expande el tipo de dato `NúmeroSecreto` que hicimos en este capítulo. Modifique la clase `SecretNumber` para que el constructor público sea ahora un método de fábrica estático: `create()`. Además, implementa métodos de `picas(String)` y `fijas(String)`. En el juego Picas y Fijas alguien piensa un número secreto y la persona que lo adivina dice intento a lo que la otra persona le contesta cuántas picas y cuántas fijas tiene ese intento: una pica es un dígito que se encuentra en el número secreto pero en otra posición y una fija es aquel que se encuentra en la misma posición.

Por ejemplo si el número secreto es "1234" y el intento es "9374" el 3 es una pica ya que está en el número pero en otra posición, en el intento es el segundo dígito y en el número secreto es el tercero; y el 4 es una fija porque está en la misma posición. El 9 y el 7 no son fijas ni picas.

Constructores
<code>private NúmeroSecreto()</code>
<code>NúmeroSecreto(String)</code>
Métodos
<code>public int picas(String)</code>
<code>public int fijas(String)</code>
Métodos Estáticos
<code>public static NúmeroSecreto crear()</code>

Asserts

```
var _1234 = new NúmeroSecreto("1234");

assertAll(
    () -> assertEquals(1, _1234.picas("9374")),
    () -> assertEquals(1, _1234.fijas("9374")),
    () -> assertEquals(0, _1234.picas("9876")),
    () -> assertEquals(0, _1234.fijas("9876")),
    () -> assertEquals(4, _1234.fijas("1234")),
    () -> assertEquals(4, _1234.picas("4321")),
    () -> assertEquals(0, _1234.fijas("4321"))
);
```

E9.3

Los links son una parte importante en Internet. Aunque los enlaces finalmente se representan en archivos HTML, muchos framework web permiten al programador definir componentes con reglas de negocio, y muchos componentes incluyen links a veces con un texto o a veces como un botón. Un link típicamente se ve así en HTML:

```
<a href="https://jcp.org/java-in-education" target="_blank">Java en la Educación</a>
```

Las partes principales de un enlace son:

- Texto: el texto dentro de las etiquetas <a>, "Java en la Educación" en el ejemplo anterior.
- Href: la URL que se cargará después de que el usuario haga clic en el enlace, "https://jcp.org/java-in-education" en el ejemplo anterior.
- Target: debe ser "_blank" si el enlace debe abrirse en otra pestaña de su navegador, o "" si el enlace debe abrirse en la misma pestaña.

La regla de negocio más común para los enlaces es que los enlaces internos siempre deben abrirse en la misma pestaña, pero los enlaces externos siempre deben abrirse en una pestaña nueva. Distinguir enlaces internos y externos es fácil, los enlaces internos son relativos, comienzan con una barra diagonal "/" y los enlaces externos comienzan con un protocolo, en este ejemplo es "https:", pero otros protocolos son "http", "ftp", etc.

Otra regla de negocio es que queremos crear un Link con solo una URL. En este caso usamos la misma URL como el texto del Link.

Queremos modelar un tipo de dato Link con estas reglas de negocio.

Constructores
<code>private Link(String texto, String url, boolean external)</code>
Métodos
<code>public String texto()</code>
<code>public String url()</code>
<code>public String target()</code>
Métodos Estáticos
<code>public static Link de(String texto, String url)</code>
<code>public static Link de(String url)</code>

Asserts

```
var java = Link.de("Get Java", "https://www.java.com/en/");  
  
assertAll(  
    () -> assertEquals("Get Java", java.texto()),  
    () -> assertEquals("https://www.java.com/en/", java.href())  
);
```

```
var javaUrl = Link.de("https://www.java.com/en/");  
  
assertAll(  
    () -> assertEquals("https://www.java.com/en/", javaUrl.texto()),  
    () -> assertEquals("https://www.java.com/en/", javaUrl.href())  
);
```

```
var internalNews = Link.of("/en/news");  
assertEquals("", internalNews.target());
```

```
var javaNewsLink = Link.of("https://dev.java/news/");  
assertEquals("_blank", javaNewsLink.target());
```

 E9.4

Lee el Side Quest "Builders" y después, escribe un tipo de dato Triángulo basada en la clase Longitud y Ángulo que desarrollamos en este capítulo. Un triángulo tiene 3 lados, generalmente llamados $\langle a \rangle$, $\langle b \rangle$ y $\langle c \rangle$ y 3 ángulos, generalmente llamados $\langle \alpha \rangle$, $\langle \beta \rangle$ y $\langle \gamma \rangle$. Opuesto al ángulo $\langle \alpha \rangle$ está el lado $\langle a \rangle$, opuesto al ángulo $\langle \beta \rangle$ está el lado $\langle b \rangle$ y opuesto al ángulo $\langle \gamma \rangle$ está el lado $\langle c \rangle$.

Es posible construir triángulos incluso si no conoces estas seis medidas. Para este ejercicio asume que debes tener al menos 3 condiciones básicas: conocer los 3 lados, conocer 2 lados y el ángulo entre ellos o dos ángulos y el lado que los conecta. Además de estas medidas, es posible que ya conozcas alguna de las otras 3 medidas, por lo que podrías proporcionarlas. Todos los valores desconocidos restantes se calcularán antes de construir el Triángulo.

Dado que tenemos varias formas de crear objetos de tipo Triángulo, aprovecharemos el patrón Builder que vimos antes. Para encontrar algunos de los valores desconocidos, utiliza estas funciones:

```

01 public class Geometria {
02
03     public static Ángulo tercerÁngulo(Ángulo alfa, Ángulo beta) {
04         return Ángulo.enCentígrados(180.0 - (alfa.centígrados() + beta.centígrados()));
05     }
06
07     public static boolean esTriángulo(Longitud a, Longitud b, Longitud c) {
08         var max = Math.max(a.medida(), Math.max(b.medida(), c.medida()));
09         var sumaÁngulosMenores = (a.medida() + b.medida() + c.medida()) - max;
10         return sumaÁngulosMenores > max;
11     }
12
13     public static boolean esTriángulo(Ángulo alfa, Ángulo beta, Ángulo gamma) {
14         var suma = alfa.centígrados() + beta.centígrados() + gamma.centígrados();
15         return Math.abs(suma - 180.0) < 1e-3;
16     }
17
18     public static Longitud leyCosenos(Longitud a, Longitud b, Ángulo gamma) {
19         var cuadradoDeA = Math.pow(a.medida(), 2.0);
20         var cuadradoDeB = Math.pow(b.medida(), 2.0);
21         var tercerTérmino = 2 * a.medida() * b.medida() * Math.cos(gamma.radianes());
22         var c = Math.sqrt(cuadradoDeA + cuadradoDeB - tercerTérmino);
23
24         return new Longitud(c);
25     }
26
27     public static Angle leyCosenos(Longitud a, Longitud b, Longitud c) {
28         var cuadradoDeA = Math.pow(a.medida(), 2.0);
29         var cuadradoDeB = Math.pow(b.medida(), 2.0);
30         var cuadradoDeC = Math.pow(c.medida(), 2.0);
31
32         var numerador = cuadradoDeA + cuadradoDeB - cuadradoDeC;
33         var denominador = 2 * a.medida() * b.medida();
34         var coseno = numerador / denominador;
35
36         var gamma = Math.toDegrees(Math.acos(coseno));
37         var ángulo = Ángulo.enCentígrados(gamma);
38         return ángulo;
39     }
40
41     public static Ángulo leySenos(Longitud a, Longitud b, Ángulo beta) {
42         var senoBeta = Math.sin(beta.radianes());
43         var razón = senoBeta / b.medida();
44         var senoAlfa = razón * a.medida();
45
46         var alfa = Math.asin(senoAlfa);
47         var ángulo = Angle.ofRadians(alfa);
48         return ángulo;
49     }
50
51     public static Longitud leySenos(Longitud a, Ángulo alfa, Ángulo beta) {
52         var senoAlfa = Math.sin(alfa.radianes());

```

53	<code>var senoBeta = Math.sin(beta.radians());</code>
54	
55	<code>var razón = a.medida() / senoAlfa;</code>
56	<code>var b = razón * senoBeta;</code>
57	<code>return new Longitud(b);</code>
58	<code>}</code>
59	<code>}</code>

Puedes idear tu propia estrategia, pero ésta es sencilla:

- Como parte del constructor `Triángulo(Longitud, Longitud, Longitud)`
 - Llama a `Geometria.leyCosenos(Length, length, length)` dos veces para obtener los valores de 2 ángulos.
 - Llama a `Geometria.tercerÁngulo(Ángulo, Ángulo)` para obtener el último ángulo y completar las 6 medidas.
- Como parte del constructor `Triángulo(Longitud, Ángulo, Longitud)`
 - Encuentra la longitud desconocida usando `Geometria.leyCosenos(Longitud, Longitud, Ángulo)`;
 - Encuentra uno de los ángulos que falta usando `Geometria.leySenos(Longitud, Longitud, Ángulo)` usando el ángulo dado como parámetro y el lado que encontraste en el punto anterior.
 - Encuentra el tercer ángulo usando `Geometria.tercerÁngulo(Ángulo, Ángulo)`
- Como parte del constructor `Triángulo(Ángulo, Longitud, Ángulo)`
 - Encuentra el tercer ángulo usando `Geometria.tercerÁngulo(Ángulo, Ángulo)`
 - Encuentra las longitudes que faltan usando `Geometria.leySenos(Longitud, Ángulo, Ángulo)` usando la longitud dada como parámetro y el ángulo encontrado en el punto anterior.

Si el usuario quiere reescribir estos valores puede usar los métodos que retornan un builder para ese fin.

En el constructor de `Triángulo` valida las 2 invariantes: los 3 lados deben ser válidos usando `Geometria.esTriángulo(Longitud, Longitud, Longitud)` si no, arroja una excepción `IllegalStateException` con un mensaje mostrando las longitudes de los 3 lados. Luego verifica la invariante para los ángulos usando `Geometria.esTriángulo(Ángulo, Ángulo, Ángulo)`

Adicionalmente debes agregar en los código que ya hemos, métodos para verificar si 2 ángulos son equivalentes y 2 longitudes son equivalentes:

```
public class Ángulo {  
    // más código  
  
    public boolean equivalente(Ángulo otro) {  
        return Math.abs(centígrados() - otro.centígrados()) < 1e-3;  
    }  
}
```

```
public class Longitud {  
    // más código  
  
    public boolean equivalent(Longitud otra) {  
        return Math.abs(medida - otra.medida) < 1e-3;  
    }  
}
```

TriánguloBuilder

Constructores

```
public TriánguloBuilder(Longitud a, Longitud b, Longitud c)
```

```
public TriánguloBuilder(Longitud a, Ángulo gamma, Longitud b)
```

```
public TriánguloBuilder(Ángulo alfa, Longitud c, Ángulo beta)
```

Métodos

```
public TriánguloBuilder a(Longitud a)  
Da un valor al atributo <a>
```

```
public TriánguloBuilder b(Longitud b)  
Da un valor al atributo <b>
```

```
public TriánguloBuilder c(Longitud c)  
Da un valor al atributo <c>
```

```
public TriánguloBuilder alfa(Ángulo alfa)
```

Da un valor al atributo <alfa>
<code>public TriánguloBuilder beta(Ángulo beta)</code> Da un valor al atributo <beta>
<code>public TriánguloBuilder gamma(Ángulo gamma)</code> Da un valor al atributo <gamma>
<code>public Triángulo build()</code> Crea un objeto de tipo Triángulo con los valores dados.

TriánguloBuilder

Constructor
<code>TriánguloBuilder(TriánguloBuilder builder)</code>

Métodos
<code>public Longitud a()</code> Accesor para el atributo <a>
<code>public Longitud b()</code> Accesor para el atributo
<code>public Longitud c()</code> Accesor para el atributo <c>
<code>public Ángulo alfa()</code> Accesor para el atributo <alfa>
<code>public Ángulo beta()</code> Accesor para el atributo <beta>
<code>public Ángulo gamma()</code> Accesor para el atributo <gamma>
<code>public boolean equivalente(Triángulo otro)</code> Verifica si 2 triángulos son equivalentes. Esto es cierto si sus correspondientes ángulos y correspondientes lados son equivalentes.



Si ves la API de TriánguloBuilder y Triángulo, ¿crees que la mejor manera de implementarla es tener 6 atributos? ¿O tiene sentido tener una arreglo de 3 longitudes y un arreglo de 3 ángulos?

Asserts

```
var longitudUno = new Longitud(1.0);
var ángulo60 = Ángulo.deCentígrados(60);
var triánguloEquilatero = new TriánguloBuilder(longitudUno, ángulo60, longitudUno).build();

assertTrue(triánguloEquilatero.a().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.b().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.c().equivalent(new Longitud(1.0)));
assertTrue(triánguloEquilatero.alfa().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.beta().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.gamma().equivalente(Ángulo.enCentígrados(60.0)));

var longitudUno = new Longitud(1.0);
var triánguloEquilatero = new TriánguloBuilder(longitudUno, longitudUno,
longitudUno).build();

assertTrue(triánguloEquilatero.a().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.b().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.c().equivalent(new Longitud(1.0)));
assertTrue(triánguloEquilatero.alfa().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.beta().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.gamma().equivalente(Ángulo.enCentígrados(60.0)));

var longitudUno = new Longitud(1.0);
var ángulo60 = Ángulo.deCentígrados(60);
var triánguloEquilatero = new TriánguloBuilder(ángulo60, longitudUno, ángulo60).build();

assertTrue(triánguloEquilatero.a().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.b().equivalente(new Longitud(1.0)));
assertTrue(triánguloEquilatero.c().equivalent(new Longitud(1.0)));
assertTrue(triánguloEquilatero.alfa().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.beta().equivalente(Ángulo.enCentígrados(60.0)));
assertTrue(triánguloEquilatero.gamma().equivalente(Ángulo.enCentígrados(60.0)));
```



Algunos de los métodos de la clase Geometría parecen métodos de fábrica estáticos, ¿no es así? Incluso se puede decir que tal vez deberían ser constructores de Longitud y Ángulo. La programación que se basa en clases con métodos se denomina programación orientada a objetos. La programación que se basa más en funciones que utilizan objetos se denomina programación funcional. Ninguna es superior a la otra y, de hecho, se puede decir que se complementan. ¿Cuándo crees que es más recomendable un método de fábrica estático que un método estático? Recuerda, para estas preguntas, ¡no hay una respuesta incorrecta!

Referencias y Null

Java tiene, por ahora, dos tipos de variables: **primitivas**, como boolean, char, int, etc.; y tipos de referencia, String, LocalDate y todas las clases que escribas. La mayor diferencia es que para los **tipos de referencia** no manejas objetos directamente sino que lo haces a través de una referencia. En un código como este:

```
String foo = "Hello"; // not a String, but actually a reference to a String
LocalTime ahora = LocalTime.now(); // a reference to a LocalTime
```

La variable <foo> no es estrictamente un String y <ahora> no es estrictamente un LocalTime, ¡ambas son referencias a esos valores! Veremos cómo afecta esto a tus programas.

Pasar por valor

Código 1 - duplicar 1.0

```
01 public class Main {
02     public static void main(String[] args) {
03         var n = 1;
04         duplicar(n);
05         System.out.println(n); // 1
06     }
07
08     public static void duplicar(int n) {
09         n *= 2;
10     }
11 }
```

¿Qué imprime el Código 01? ¡Puede que te sorprenda saber que imprime 1! ¿Qué pasó? Bueno, sucede que Java, como muchos otros lenguajes, al llamar a una función, creará una **copia** del argumento y la utilizará en la ejecución del método. Esto se llama **pasar por valor**. La línea 03 crea una variable <n> que solo es visible dentro de main(). La línea 08 establece que el parámetro a utilizar también se llama <n> pero no tiene relación con el otro, ya que este solo sería visible dentro de duplicar(). Si ambas variables comparten su nombre es sólo coincidencia. En la línea 09, la variable <n> local de duplicar() se duplica, pero esa variable se destruye cuando la función regresa.

En la Línea 05 lo que estás imprimiendo es la variable `<n>` que es local a `main()` y que siempre ha tenido valor 1.

Diseño de memoria para el código 1 en la línea 09

duplicar	
$n = 2$	
main	
$n = 1$	



¡No te dejes engañar por los nombres! Verás muchas variables en tus programas que tienen el mismo nombre, pero aprende sobre su alcance para que cada vez que veas un nombre puedas reconocer cuál de ellas se está utilizando realmente. Ésa es otra razón para dar nombres expresivos a las variables: para evitar conflictos de nombres, aunque a veces es imposible evitarlo.



¡Una variable solo es visible en el ámbito en el que fue creada! Si usa esa variable como argumento, se creará una copia.

Estudiemos ahora una nueva versión, ¿qué crees que se imprime? Recuerda que mencionamos que los objetos se manejan por referencia:

```
01 public class Main {  
02     private int n = 1;  
03  
04     public static void main(String[] args) {  
05         var obj = new Main();  
06         duplicar(obj);  
07         System.out.println(obj.n); // 2  
08     }  
09  
10     public static void duplicar(Main obj) {  
11         obj.n *= 2;  
12     }  
13 }
```

¡Ahora el código imprime 2! ¿Cómo ocurrió eso? Bueno, ahora no estamos tratando con tipos de datos primitivos, sino con objetos. La clase Main tiene un atributo, un int llamado, ¿qué más?, <n>. En la Línea 05, se crearon dos cosas diferentes: un objeto de tipo Main y una referencia a él llamada <obj>. Ambos se crean en lugares diferentes: el objeto se llama en un espacio de memoria especial llamado "heap" y la referencia se crea en otro llamado el stack de ejecución.

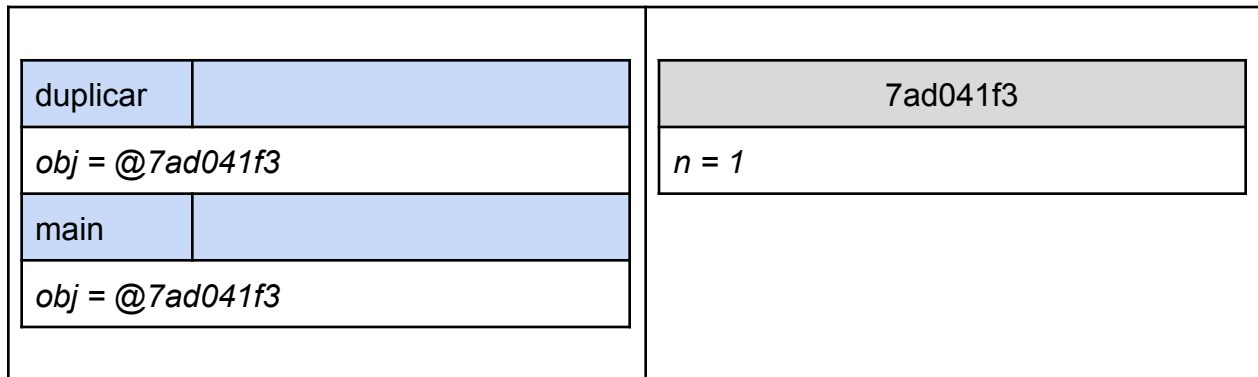
Dado que los objetos pueden ser complejos, cada uno de ellos con varios atributos, y sus atributos potencialmente con otros atributos, sería inconveniente que todo este objeto complejo se copiara al ser usado como parámetro. Pero al tener una grado de indirección, el objeto nunca se copia se mantiene en el heap y solo se copia la referencia.

Un objeto sólo será "destruido", o su memoria será recuperada, cuando todas sus referencias hayan sido destruidas. La línea 06 usa <obj> como argumento y, como se mencionó anteriormente, la línea 10 usa una copia de ese argumento, también denominada <obj>. Al igual que antes, el <obj> de tipo Main solo es visible dentro de él y el <obj> de duplicar() solo es visible en esa función, pero ambos tienen el mismo valor: ¡ambos apuntan al mismo objeto, el creado en la línea 05!

Entonces, en la Línea 11 modificamos el estado del único objeto que se ha creado, ¡pero al que hacemos referencia a través de dos variables: <obj> de main() y el <obj> de duplicar()! Cuando duplicar() regresa, solo hay una referencia, pero el estado se

modificó previamente y podemos ver el cambio al usar el <obj> de main(). Recuerda que **new** es el operador para crear, bueno, nuevos objetos.

Diseño de memoria para el código 2 en la línea 10



Al ejecutar la línea 05 se crea una variable de tipo Main en el heap y en mi máquina se crea en la dirección en memoria 7ad041f3. Adicionalmente se crea una referencia a ese objeto y en él se almacena la dirección del objeto.

Una vez que se ejecuta la línea 11, ya se ha creado en una nueva referencia que tiene el mismo valor que la de main(), la dirección en memoria del objeto de tipo Main.

Código 3 - duplicar 3.0

```

01 public class Main {
02     private n = 1;
03
04     public void main(String[] args) {
05         var a = 1;
06         var b = a;
07         a *= 2;
08         System.out.println(String.format("b == %d", b));
09
10         var obj = new Main();
11         var copy = obj;
12         obj.n *= 2;
13         System.out.println(String.format("n == %d", copy.n));
14     }
15 }

```

```
b == 1
n == 2
```

La primera parte de esto no debería sorprendernos. Línea 05 creamos <a> con valor 1, en Línea 06 creamos como una copia de a, por lo que también tiene el valor de 1. Modificamos <a> duplicándolo y ahora tiene valor 2, pero tiene 1, ya que lo que le sucede a <a> no afecta a .

La segunda parte es complicada si no recuerdas las referencias, pero esto es lo que realmente sucede: en la línea 10 creamos un objeto de tipo Main y obtenemos una referencia llamada <obj>. Luego, en la línea 11, creamos una copia de la referencia del objeto creado en la línea anterior y le asignamos el nombre <copy>. Las referencias son diferentes de las primitivas, por lo que al copiar lo que estamos haciendo se crea otra referencia al mismo objeto. Ahora solo hay un objeto de tipo Main al que se hace referencia 2 veces: ¡con <obj> y <copy>! ¡Cualquier cambio que hagamos en el objeto será visible desde cualquiera de ellos! Observe que modificamos el objeto en la Línea 12 usando la referencia <obj>, pero en la Línea 13 lo imprimimos usando <copy>

Diseño de memoria para el Código 3 en la línea 13. En esta computadora, el objeto principal se creó en la dirección 251a69d7

<table><tr><td>main</td><td></td></tr><tr><td colspan="2"><i>a = 2</i> <i>b = 1</i> <i>obj = @251a69d7</i> <i>copy = @251a69d7</i></td></tr></table>		main		<i>a = 2</i> <i>b = 1</i> <i>obj = @251a69d7</i> <i>copy = @251a69d7</i>		<table><tr><td colspan="2">251a69d7</td></tr><tr><td colspan="2"><i>n = 2</i></td></tr></table>		251a69d7		<i>n = 2</i>	
main											
<i>a = 2</i> <i>b = 1</i> <i>obj = @251a69d7</i> <i>copy = @251a69d7</i>											
251a69d7											
<i>n = 2</i>											

El objetivo de este ejemplo es que recuerdes que tener múltiples referencias a objetos mutables es complicado y es posible que parte del código no se comporte como esperabas si no tomas en cuenta que los objetos son manejados por referencias.



¡No cree copias de una referencia! Con múltiples referencias al mismo objeto, es difícil razonar si se supone que un cambio que estás haciendo con uno de ellos es visible en el contexto de las otras referencias. Si hace uno, piensa en lo que está tratando de lograr y busque una forma más segura de hacerlo.

Entonces, ¿cuál es la forma segura de modificar el estado de un objeto como parte de una funcionalidad? La mejor acción es hacer que tus clases sean inmutables, que es un tema que cubriremos pronto. Pero si se trata de una clase mutable, entonces se puede crear una copia del objeto. Veamos un ejemplo muy sencillo con una clase que ya has visto antes:

```
01 public class Length {
02     private double measure;
03
04     public Length(double measure) {
05         if (measure <= 0.0) {
06             var message = String.format("Measure can't be zero or
07 negative. Current value is %f", measure);
08             throw new IllegalArgumentException(message);
09         }
10         this.measure = measure;
11     }
12
13     double measure() {
14         return measure;
15     }
16 }
```

```
Length length = new Length(1.0);
Length copy = new Length(length.measure());
```

En lugar de <copiar = longitud> creamos un nuevo objeto y pasamos la medida de la primera longitud como parámetro en el constructor. Por supuesto, este ejemplo es específico de Longitud, pero diferentes clases necesitarán enfoques diferentes, pero similares. Cuando trabaje con clases mutables, intente hacer una copia de los objetos antes de manipular su estado. Para tus clases, aprende cómo hacerlos inmutables, ¡que es un tema que cubriremos pronto! Solo haga que una clase sea mutable si tiene

una buena razón para hacerlo, p. cuando un generador de perfiles te muestra que tu programa tiene un problema de rendimiento y de hecho está relacionado con tu clase y la creación de nuevos objetos con el estado actualizado de los mismos. ¡Este tipo de escenarios rara vez se ven en el software cotidiano!

NULL

Dado que tenemos referencias, ¿cuál es el valor de una referencia que actualmente no se refiere a un objeto? Ese valor es <nulo>. Hubo un tiempo en el que era muy común inicializar la referencia a <nulo> y luego asignarles otro valor. Como mencionamos antes, la reasignación de variables es una mala idea, y eso también se aplica a las referencias. Entonces, si la reasignación de variables es una mala idea, ¿qué papel debería tener null en su código?

Durante mucho tiempo, <nulo> fue la única opción para indicar la ausencia de un valor en un método de devolución. Por ejemplo, considere un método `find(String)` en una clase `WordSearch` que busca la palabra pasada como parámetro en la cuadrícula con letras. Cuando se encuentra la palabra, devuelve un rango de coordenadas donde están todas las letras de la palabra. Pero ¿qué pasa si no se encuentra la palabra? Como se mencionó, durante mucho tiempo la única opción posible era devolver nulo para indicar que no se encuentra dicha palabra.

Pero devolver nulo es peligroso. Piensa en cuando lees un método como el mencionado anteriormente:

```
01 public class WordSearch {
02
03     // more code
04     public Range find(String word) {
05         // implementation
06     }
07
08 }
```

¿Cómo saber si el método puede devolver nulo o no? No puedes simplemente mirar la firma del método. Por lo tanto, confía en leer la documentación del método, algo que muchas personas nunca hacen o que podría ser incorrecta, o necesita leer cómo se implementa el código y asegurarse de que esté lo suficientemente limpio como para que sea obvio cuando el método devuelve nulo.

¿Por qué es tan malo que no exista una forma sencilla de saber si un método devuelve nulo? Veamos el error más común que cometen los programadores al codificar:

```
01 String s = null;
02 s.length();
```

```
Exception java.lang.NullPointerException
```

Dado que no hay ningún objeto al que se refiera `s`, si intenta llamar a cualquier método de `String` en él, fallará con una excepción: `NullPointerException`, generalmente abreviada como `NPE`. Puedes pensar que si inicializas `<s>` en nulo, ¿por qué intentarías llamar a un método en él?

¿Qué te parece ahora?

```
01 String s = mysteryMethod();
02 s.length();
```

¿Crees que arrojaría una `NPE`? Bueno, no lo sabes. Entonces, ¿qué haces si no puedes estar seguro? Haces un control defensivo:

```
01 String s = mysteryMethod();
02 if (s != null) {
03     s.length();
04     // more code
05 }
```

Eso no es tan malo, ¿no? Sí, siempre y cuando recuerdes realizar comprobaciones defensivas para todos tus tipos de referencia. Pero en un programa largo es fácil pasar por alto algunos.

Devolver nulo no es una solución ideal, pero como se mencionó, era la única opción disponible. Y es, incluso hoy en día, la solución más sencilla para "señalar la ausencia de un valor", y verás mucho código que la utilizaría, por lo que es bueno que te familiarices con ella. Desafortunadamente, la mejor alternativa para un valor de retorno, `Optional`, está fuera del alcance de este material.

Valores Nulos

```
01 String domain(String url) {
02     var doubleSlash = "//";
03     var startOfDoubleSlash = url.indexOf(doubleSlash);
04     if (startOfDoubleSlash == -1) {
05         return "";
06     }
07
08     var start = startOfDoubleSlash + doubleSlash.length();
09     var nextSlash = url.indexOf("/", start);
10     var end = nextSlash != -1 ? nextSlash : url.length();
11     return url.substring(start, end);
12 }
```

No siempre es necesario devolver un valor nulo u otra alternativa si su tipo tiene un valor obvio que se consideraría "vacío", generalmente llamado valor nulo. En nuestro ejemplo, extraemos el dominio de una URL, pero si tiene un formato incorrecto podemos devolver una cadena vacía para indicar que no había ningún dominio en ella. Otro tipo que ha utilizado que tiene un valor nulo obvio: si devuelve una matriz o una colección, puede devolver una vacía.

El problema con los valores nulos es que no todos los tipos tienen uno. Por ejemplo, si su método devuelve `LocalDateTime`, ¿cuál sería un posible valor nulo? Entonces, a veces necesitas un valor como nulo.

Ejercicios

Escribe un conjunto de clases para encontrar palabras en un rompecabezas de WordSearch. La clase `WordSearch` tiene internamente una cuadrícula, posiblemente una matriz de caracteres. Para mostrar dónde están las palabras se utilizan dos clases: `Coordenadas` y `Rango`. Una `Coordenada` es la encapsulación de una fila y una columna, y un `Rango` es la encapsulación de dos `Coordenadas`, una para la primera letra de una palabra y otra para la letra final de una palabra.

Coordenada

Constructor

```
public Coordinada(int hilera, int columna)
```

Métodos

```
public int hilera()
```

```
public int columna()
```

```
public Coordinada desplazar(int desplazamientoHilera, int desplazamientoColumna)  
Retorna la coordenada resultante de tomar la hilera actual y sumarla  
con desplazamientoHilera y la columna actual y sumarla con  
desplazamientoColumna
```

```
public boolean equals(Object obj)
```

```
public String toString()  
Retorna un String con el formato "(this.hilera, this.columna)"
```

Asserts

```
var unoTres = new Coordinada(1, 3);  
assertEquals(1, unoTres.hilera());  
assertEquals(3, unoTres.columna());
```

```
var unoTres = new Coordinada(1, 3);  
assertEquals("(1, 3)", unoTres.toString());
```

```
var unoTres = new Coordinada(1, 3);  
var dosCuatro = unoTres.desplazar(1, 1);  
assertEquals(2, dosCuatro.hilera());  
assertEquals(4, dosCuatro.columna());
```

Rango

Constructor

```
public Rango(Coordenada inicio, Coordinada fin)
```

Métodos

```
public Coordenada inicio()
```

```
public Coordenada fin()
```

```
public boolean equals(Object obj)
```

```
public String toString()
```

Retorna un String con el formato "this.inicio - this.fin"

Asserts

```
var ceroCeroUnoUno = new Rango(new Coordenada(0, 0), new Coordenada(1, 1));  
assertEquals("(0, 0) - (1, 1)", ceroCeroUnoUno.toString());
```

SopaDeLetras

Constructor

```
public SopaDeLetras(int hileras, int columnas, String[] sopa)
```

Crea una sopa de letras de las dimensiones dadas y la configuración dada en sopa, cada String representa una fila de la cuadrícula. Puedes asumir que todas las cadenas serán del mismo tamaño que el indicado en el parámetro de columnas.

Métodos

```
public Rango buscar(String palabra)
```

Busque la palabra dada en la pizarra. Si se encuentra la palabra, devolverá un Rango, donde la Coordenada inicial es el lugar donde se encuentra la primera letra de la palabra y la Coordenada final es el lugar donde está la última letra. Si no se puede encontrar la palabra, devuelve nulo.

Siéntete libre de agregar métodos adicionales a cualquier clase si lo crees apropiado.

Asserts

Para estas afirmaciones, suponga que tiene esta constante de clase:

```
01 private static final String[] board = {  
02     "VBREEFISHRACHP",  
03     "ANACROCODILEEB",
```

04	"AOSTRICHEGRDA",
05	"IADDHCHEETAHGD",
06	"BHRODRAVENENEG",
07	"EYWDLSAMOLELHE",
08	"ARTPVPRCBOLROR",
09	"RHTOAAHCROWAGH",
10	"CCANNORIAZEBRA",
11	"HANYTAEKNINAWA"
12	};

```
var juego = new SopaDeLetras(10, 14, sopa);
assertEquals(new Rango(new Coordenada(0, 5), new Coordenada(0, 8)), juego.buscar("FISH"));
```

```
var juego = new SopaDeLetras(10, 14, sopa);
var hedgehog = juego.buscar("HEDGEHOG");
assertEquals(new Rango(new Coordenada(0, 12), new Coordenada(7, 12)), hedgehog);
```

```
var juego = new SopaDeLetras(10, 14, sopa);
var bat = juego.buscar("BAT");
assertEquals(new Rango(new Coordinate(0, 1), new Coordenada(2, 3)), bat);
```

Enums

Código 01 Enums para modelar la carta de una baraja

```
01 public enum Valor {  
02     AS, DOS, TRES, CUATRO, CINCO, SEIS, SIETE, OCHO, NUEVE, DIEZ, J, Q, K;  
03 }
```

```
01 public enum Palo {  
02     TRÉBOLES, DIAMANTES, CORAZONES, PICAS;  
03 }
```

```
01 public class Carta {  
02  
03     private Valor valor;  
04     private Palo palo;  
05  
06     public Carta(Valor valor, Palo palo) {  
07         this.valor = valor;  
08         this.palo = palo;  
09     }  
10  
11     public Valor valor() {  
12         return valor;  
13     }  
14  
15     public Palo palo() {  
16         return palo;  
17     }  
18 }
```

```
01 public class Main {  
02  
03     public static void main(String[] args) {  
04         var asCorazones = new Carta(Valor.AS, Palo.CORAZONES);  
05         System.out.printf("%s %s", asCorazones.valor(), asCorazones.palo());  
06     }  
07  
08 }
```


El código 01 utiliza Enums para modelar el Valor y el Palo de una carta de una baraja. Un Enum es una forma especializada de una clase. Para crear un Enum en lugar de utilizar la palabra reservada <class>, utiliza la palabra reservada <enum>. Igual que con una clase necesitas definir un bloque para los Enum.

Hemos visto cómo puedes crear nuevos tipos de datos con clases. Los objetos que creamos a partir de esas clases tienen un estado y normalmente los atributos tienen muchos valores posibles, por lo que se puede decir que el número de posibles estados es muy grande. Pero ¿qué sucede cuando queremos modelar tipos de datos como el valor y el palo de una carta de una baraja, pero también el color de los ojos de una persona, o el tipo de documento de una persona, etc? Para estos tipos de datos donde el conjunto de valores es fácil de enumerar, lo recomendable es usar un Enum.

Las líneas 02 de Valor y 02 de Palo enumeran los posibles valores de su respectivo tipo de dato. Puedes elegir cualquier nombre siempre que se ajuste a las reglas para identificadores. Ni siquiera es necesario tener todos los identificadores en mayúsculas, pero es costumbre hacerlo ya que estás definiendo una constante.

Ahora con estos Enums puedes crear cartas sabiendo que el Valor y el Palo son valores válidos. La línea 04 de Main crea una carta con Valor.AS y Palo.CORAZONES. A diferencia de las clases, no creas valores de Enums usando el operador new, de los valores de un Enum no los creas simplemente los referencias de la misma manera que cualquier constante de clase.

Pero ahora puedes decir "Bueno, las enumeraciones parecen muy limitadas, ¿no?" Veamos más cosas que puedes hacer con ellas.

Metodos y Atributos

Código 02 - Valor con métodos

```
01 public enum Valor {
02     AS, DOS, TRES, CUATRO, CINCO, SEIS, SIETE, OCHO, DIEZ, J, Q, K;
03
04     public int puntaje() {
05         return ordinal() + 1;
06     }
07
08     public String toString() {
09         return switch(this) {
10             case DOS, TRES, CUATRO, CINCO, SEIS, SIETE, OCHO, NUEVE ->
11             Integer.toString(value());
12             case AS -> "A";
13             case DIEZ -> "T";
14             case J, Q, K -> super.toString();
15         };
16     }
17 }
```

Código 02 - Palo con atributos

```
01 public enum Palo {
02     TRÉBOLES("♣"), DIAMANTES("♦"), CORAZONES("♥"), PICAS("♠");
03
04     private String símbolo;
05
06     private Suit(String símbolo) {
07         this.símbolo = símbolo;
08     }
09
10     public String toString() {
11         return símbolo;
12     }
13 }
```

La línea 04 de Palo define un atributo. Los Enum son también clases por lo que pueden tener atributos y métodos como cualquier otra clase. Sin embargo el orden de los elementos es importante, sólo puedes definir atributos después de la lista de valores (línea 02 en la misma clase). La línea 06 tiene un constructor. Los constructores en Enums siempre son privados porque no puedes llamarlos directamente. Lo que debes hacer es especificar los valores que usará cada constante cuando se llame al constructor. Ese constructor lo llama el ambiente de ejecución mucho antes de que tu programa comience a ejecutarse. Entonces, en la línea 02 verás que tienes una especie de invocación donde cada Palo define el valor del atributo.



El conjunto de constantes siempre debe ser el primer elemento de una Enum, de lo contrario, obtendrás un error de compilación.



No te dejes engañar por un constructor Enum que no tenga el modificador privado. Incluso si no lo pones, el compilador tratará al constructor como privado. En tus programas usarás las constantes para referir los valores de Enum, por lo que nunca tendrás la necesidad de llamar al constructor.



Si defines un constructor con parámetros, pero no proporcionas un valor junto a cada constante de la lista, obtendrás un error de compilación. También debes proporcionar el número exacto de parámetros y del tipo correcto.

Puedes definir cualquier cantidad de métodos y de cualquier tipo en un Enum. Las líneas 04 a 06 de Valor definen un método puntaje() ya que los valores en una carta generalmente están asociados con un valor numérico. Todos los tipos de Enum tienen algunos métodos ya implementados, uno de ellos es ordinal() que devuelve la posición actual de la constante en la lista. En este caso, dado que la posición en la lista está relacionada con su valor numérico podemos usar ordinal() para implementar puntaje(), aunque ordinal() devuelve un índice por lo que empieza en 0, razón por la cual debemos sumarle 1.



Ten cuidado al usar ordinal() en tus implementaciones ya que su valor de retorno cambiará si se reorganiza la lista de constantes. En el caso del valor de una carta, esto es una ventaja pero en otros Enum podría no serlo.

Las líneas 08-13 definimos el método toString(). Uno de los grandes beneficios de Enums es que se combinan bastante bien con los switch. Una de las cosas buenas es que si alguna vez agregas una nueva constante al Enum, el compilador dará un error de compilación en el switch que usan ese tipo, hasta que agregues un <case> para esa nueva constante.



No escribas un case default en switch que usen Enums. Si necesitas agrupar un valor con muchos casos, utiliza la sintaxis separada por comas para ello. Programar es difícil y ser explícito suele evitar errores sutiles. ¡Esto no se limita a Enums! Solo usa default para valores para los cuales realmente no puedes enumerar todos los valores posibles en el momento de la compilación o es engorroso, p.j variables de tipo int.

Misión Secundaria: sobreescritura de métodos por constante

Código 3 - Palo con método sobreescritos por constante

```
01 public enum Palo {
02     TRÉBOLES {
03         public String toString() {
04             return "♣";
05         }
06     },
07     DIAMANTES {
08         public String toString() {
09             return "♦";
10         }
11     },
12     CORAZONES {
13         public String toString() {
14             return "♥";
15         }
16     },
17     PICAS {
18         public String toString() {
19             return "♠";
20         }
21     };
22 }
```

Otra opción es que puedes proporcionar una implementación específica de un método para una constante. La sintaxis puede ser un poco extraña a la vista, pero mejora si utilizas un buen espacio vertical: para sobrecribir métodos debes agregar un bloque después del nombre de la constante y dentro de ese bloque escribir la implementación específica para esa constante del método.

Si el método no se hereda, debes declararlo fuera de las constantes, como por ejemplo:

Código 4 - sobrescribiendo métodos declarados como abstract

```
01 public enum Suit {
02     TRÉBOLES {
03         public String toString() {
04             return "♣";
05         }
06     },
07     DIAMANTES {
08         public String toString() {
09             return "♦";
10         }
11     },
12     CORAZONES {
13         public String toString() {
14             return "♥";
15         }
16     },
17     PICAS {
18         public String toString() {
19             return "♠";
20         }
21     };
22
23     abstract String toString();
24 }
```

La línea 21 define toString() como un método abstracto. Esto no es necesario para toString() pero para métodos no-heredados, sí deberás hacerlo. El método abstracto es un tema más extenso relacionado con el concepto de herencia, que se sale del alcance de este material.

Ejercicios

E11.1

Escribe una función tipoNúmero() que reciba un número entero y retorne si un número es perfecto, abundante o deficiente.

Para este ejercicio apóyate de estas funciones:

```

public class Aritmética {

    public static int sumaAlícuota(int n) {
        var suma = 0;
        for (var a = 1; a <= n / 2; ++a) {
            if (esMúltiplo(n, a)) {
                suma += a;
            }
        }
        return suma;
    }

    public static boolean esMúltiplo(int a, int b) {
        return a % b == 0;
    }
}

```

Un número es:

- Perfecto: si es igual que su suma alícuota
- Imperfecto si es menor que su suma alícuota
- Abundante si es mayor que su suma alícuota

Para esta función, crea un Enum:

TipoNúmero

Constantes de Enum
PERFECTO
DEFICIENTE
ABUNDANTE

Asserts

```

assertAll(
    () -> assertEquals(TipoNúmero.PERFECTO, tipoNúmero(6), "6 es número perfecto"),
    () -> assertEquals(TipoNúmero.PERFECTO, tipoNúmero(28), "28 es número perfecto"),
    () -> assertEquals(TipoNúmero.DEFICIENTE, tipoNúmero(9), "9 es número deficiente"),
    () -> assertEquals(TipoNúmero.DEFICIENTE, tipoNúmero(15), "15 es número deficiente"),
    () -> assertEquals(TipoNúmero.ABUNDANTE, tipoNúmero(12), "12 es número abundante"),
    () -> assertEquals(TipoNúmero.ABUNDANTE, tipoNúmero(18), "18 es número abundante")
);

```

E11.2

En muchos países, hay leyes que establecen que cuando la fecha de algunos días festivos no sea lunes, el día festivo se debe correr al siguiente lunes.

Java tiene una extensa lista de clases relacionadas con el tiempo, que incluye algunos Enums, como: Month y DayOfWeek.

LocalDate es un tipo de dato para representar una fecha sin tener en cuenta una zona horaria. Esta clase tiene un método `getDayOfWeek()` que retorna una variable de tipo `DayOfWeek`.

`DayOfWeek` por su lado tiene un método llamado `getValue()` que retorna un número entre 1 y 7, siendo 1 cuando el día de la semana es lunes y así sucesivamente hasta 7 cuando el día de la semana es domingo.

Escribe una función que se llame `siguienteLunes(LocalDate)` que retorne un `LocalDate` que representa el siguiente lunes del parámetro dado. Sigue este algoritmo:

- Si el día de la semana de la fecha ya es lunes, retorna el parámetro
- Sino, calcula la diferencia entre `getValue()` del domingo y el `getValue()` de la fecha dada. A este valor súmale 1, llámémoslo `díasHastaLunes`.
- Retorna el resultado de llamar `plusDays(int)` del parámetro dado y usa la variable `díasHastaLunes` como argumento.

Asserts

```
assertAll(  
    () -> assertEquals(LocalDate.of(2024, 1, 1), siguienteLunes(LocalDate.of(2024, 1, 1))),  
    () -> assertEquals(LocalDate.of(2024, 1, 8), siguienteLunes(LocalDate.of(2024, 1, 2))),  
    () -> assertEquals(LocalDate.of(2024, 1, 8), siguienteLunes(LocalDate.of(2024, 1, 5)))  
);
```

E11.3

Escribe un Enum `Dirección` para representar direcciones dentro de una matriz, con ocho posibles valores: por ejemplo, la constante `NORTE` representa una dirección que significa subir una fila pero mantener la misma columna. Escoge cómo implementarías los métodos: con atributos / constructores, con switch o sobrescribiendo los métodos para cada constante.

Dirección

Constantes de Enum
NORTE
NOROESTE
OESTE
SUROESTE
SUR
SURESTE
ESTE
NORESTE

Métodos
<pre>public int desplazamientoFila() devuelve el desplazamiento de la fila para esta dirección. Debería devolver -1 para NORTE, NOROESTE y NORESTE; 0 para OESTE y ESTE; 1 para SUR, SUROESTE y SURESTE.</pre>
<pre>public int desplazamientoColumna() devuelve el desplazamiento de la fila para esta dirección. Debería regresar -1 para ESTE, NORESTE, SURESTE; 0 para SUR y NORTE; 1 para OESTE , NOROESTE y SUROESTE.</pre>

Con este enum puedes verificar vecinos de una casilla en una matriz con este código como modelo:


```

var x = ...
var y = ...
for (var dirección : Dirección.values()) {
    var nx = x + direccion.desplazamientoFila();
    var ny = y + direccion.desplazamientoColumna();
    // más código
}

```

E11.2

[El juego de la vida de Conway](#) es un juego sin jugadores, donde un mundo tiene un estado inicial y puedes ver cómo evoluciona. El mundo es una cuadrícula con células que pueden estar en dos estados: vivas y muertas. Después de cada generación algunas células cambian de estado mientras que otras no con estas simples reglas:

- Las células muertas cobran vida si tienen exactamente 3 vecinos vivos.
- Las células vivas mueren si tienen menos de 2 o más de 3 vecinos vivos.
- En todos los demás casos las células mantienen su estado.

Para este ejercicio desarrollarás un mundo que es finito. Escoge cómo implementarías los métodos: con atributos / constructores, con switch o sobrescribiendo los métodos para cada constante.

Célula

Constantes de Enum

VIVA

MUERTA

Métodos

public Célula siguienteGeneración(**int** vecinosVivos)
Devuelve el estado de la célula para la próxima generación, según las reglas del Juego de la Vida de Conway.

public String toString()

retorna “#” para VIVA y “.” para MUERTA

Asserts

<code>assertEquals(Célula.MUERTA, Célula.VIVA.siguienteGeneración(1));</code>
<code>assertEquals(Célula.MUERTA, Célula.VIVA.siguienteGeneración(4));</code>
<code>assertEquals(Célula.VIVA, Célula.VIVA.siguienteGeneración(2));</code>
<code>assertEquals(Célula.VIVA, Célula.VIVA.siguienteGeneración(3));</code>
<code>assertEquals(Célula.MUERTA, Célula.DEAD.siguienteGeneración(2));</code>
<code>assertEquals(Célula.VIVA, Célula.DEAD.siguienteGeneración(3));</code>
<code>assertEquals(Célula.MUERTA, Célula.DEAD.siguienteGeneración(4));</code>

Para la siguiente clase, reutiliza la clase Coordinada que escribiste anteriormente.

Mundo

Constructores
<code>public Mundo(int filas, int columnas, Coordena[] célulasVivas)</code> construye un mundo finito con las dimensiones dadas con todas sus celular inicialmente en estado MUERTA salvo en las coordenadas indicadas en célulasVivas
Métodos
<code>public Mundo siguienteGeneración()</code> crea un nuevo mundo con las células en su estado actualizado después de llamar siguienteGeneración(int) en ellas.
<code>public String toString()</code> devuelve un String de múltiples líneas con "." representando una célula muerta y "#" representando una célula viva.

Asserts

<code>// 01234</code> <code>// 0.....</code>

```

// 1..#..
// 2..#..
// 3..#..
// 4.....
var inicial = new Mundo(new Coordenada[] {
    new Coordenada(5, 5),
    new Coordenada(1, 2),
    new Coordenada(2, 2),
    new Coordenada(3, 2),
});

// 01234
// 0.....
// 1.....
// 2.###.
// 3.....
// 4.....

assertEquals(new Mundo(new Coordenada[] {
    new Coordenada(5, 5),
    new Coordenada(2, 1),
    new Coordenada(2, 2),
    new Coordenada(2, 3),
}), inicial.siguienteGeneración());

```

```

// 012345
// 0.....
// 1.....
// 2..###.
// 3.###..
// 4.....
// 5.....
var inicial = new Mundo(new Coordenada[] {
    new Coordenada(6, 6),
    new Coordenada(2, 2),
    new Coordenada(2, 3),
    new Coordenada(2, 4),
    new Coordenada(3, 1),
    new Coordenada(3, 2),
    new Coordenada(3, 3),
});

// 012345
// 0.....
// 1...#..
// 2.#...#.

```

```
// 3.#..#.
// 4..#...
// 5.....

;

assertEquals(new Mundo(new Coordenada[] {
    new Coordenada(6, 6),
    new Coordenada(2, 1),
    new Coordenada(3, 1),
    new Coordenada(4, 2),
    new Coordenada(3, 4),
    new Coordenada(2, 4),
    new Coordenada(1, 3),
}), inicial.siguienteGeneración());
```